
SOFTWARE ARCHITECTURE DOCUMENT

OpenS2P

Reference Implementation Guide

The Linux of Procurement AI — an open-source Source-to-Pay platform

1
.
2

(
P
u
b
l
i
c
a
t
i
o
n

D
r
a
f
t
)

D
r
a
f
t
—

U
n
d
e
r
R
e
v
i
e
w

I
n
t
e
r
n
a
l
/
P
a
r
t
n
e
r
D
i
s
t
r
i
b
u

t
i
o
n

J
u
l
y
2
0
2
6

K
a
n
n
a
n

—

O
p
e
n
S
2
P

P
r
o
j
e
c
t

Document Control

Revision History

Version	Date	Author	Summary of Changes
1.0	2026	Kannan	Initial implementation-ready draft covering 15 core services, APIs, events, and data models.
1.1	July 2026	Kannan	Structural review applied: document control, executive summary, architecture diagrams, service fact sheets, publication-quality formatting.
1.2	July 2026	Kannan	Added Catalog Service and Analytics Service as planned-service stubs (Chapters 32-33) to close the scope gap identified across multiple reviews; updated Service Inventory, Service Dependency Matrix, and Domain Ownership Matrix accordingly. Full architecture for both deferred to detailed design pending the BRD/SRS.

Review & Approval

Role	Name	Status
Author	Kannan	Complete
Technical Reviewer	TBD	Pending
Security Reviewer	TBD	Pending
Approving Authority	TBD	Pending

Related Documents

Document	Purpose
OpenS2P Architecture Vision v4.4	Strategic intent, principles, ADR catalog
ADR Catalog (ADR-001 – ADR-016)	Decision rationales

Document	Purpose
OpenAPI Specifications	API contract definitions
AsyncAPI Specifications	Event contract definitions
Technology Radar	Technology selection status
Deployment Profiles	Reference sizing configurations

Known Issues

Code snippet formatting (carried from v1.0 source): a number of SQL, YAML, and TypeScript listings in this document contain wrapped or concatenated lines inherited from the original source content. This publication pass has not rewritten code content — doing so without the ability to execute-test it risks introducing errors — so some listings may still need reformatting for readability before this reaches external developers.

Executive Summary

OpenS2P is an open-source, AI-native Source-to-Pay (S2P) platform positioned as a transparent, extensible alternative to incumbent suites such as SAP Ariba, Coupa, and Jaggaer. This document is the Software Architecture Document (SAD): the implementation-ready reference for every service, API, event contract, and data model that makes up the platform.

Business Goals

- Deliver end-to-end procurement functionality — sourcing, supplier management, contracts, purchasing, receiving, invoicing, and payment — as independently deployable services.
- Embed AI throughout the procurement lifecycle via a dedicated AI Orchestrator, Knowledge Service, and confidence-scored agent execution pipeline, rather than bolting AI on as an afterthought.
- Enable a plugin and connector ecosystem so implementation partners and customers can extend the platform without forking core services.
- Provide the same documentation and API rigor as Kubernetes or OpenTelemetry, so enterprise buyers can evaluate OpenS2P with the same diligence applied to incumbent platforms.

Major Architectural Decisions

- Fifteen core services organised into four layers — Presentation, Domain, AI, and Infrastructure — communicating over REST APIs and Kafka-based domain events.
- Domain-Driven Design throughout: each service owns clearly bounded aggregates (e.g. SourcingEvent, Supplier, Universal Contract, PurchaseOrder, Invoice) with explicit business rules and validation.
- AI capability is a first-class architectural layer (Knowledge Service, AI Orchestrator, Plugin Runtime, Connector Runtime) with human-in-the-loop gating and confidence scoring rather than unchecked automation.
- Multi-tenancy via schema-per-tenant PostgreSQL, with Keycloak for identity and JWT-based authorisation.

Deployment Model

Services are containerised and deployed on Kubernetes via Helm, with environment-specific value profiles for development, staging, and production. Observability is standardised on OpenTelemetry, with Prometheus-based alerting and defined performance targets per service. Full deployment topology, scaling guidance, and environment promotion practices are detailed in Section 25.

Table of Contents

1. Introduction

1.1 Document Purpose

This Software Architecture Document (SAD) provides the definitive, implementation-ready specification for every service, component, and integration point in the OpenS2P platform. Where the **Architecture Vision** defines *what* OpenS2P is and *why* decisions were made, this document defines *how* every service is built, deployed, and operated.

Audience:

- Service developers implementing new or existing services
- Platform engineers maintaining infrastructure
- Quality assurance teams validating implementations
- Security reviewers assessing service designs
- Technical leads reviewing architecture proposals

1.2 Document Scope

This document covers:

- Service contracts and APIs (both REST and event-driven)
- Data models and storage responsibilities
- Implementation patterns and code organisation
- Error handling, observability, and operational concerns
- Deployment-specific configuration and scaling

This document does NOT cover:

- Business capability strategy (see Architecture Vision)
- High-level architectural principles (see Architecture Vision)
- ADR rationales (see ADR Catalog)
- Developer workflow or contribution guidelines (see CONTRIBUTING.md)

1.3 Conventions

RFC 2119 Language: MUST, MUST NOT, SHOULD, SHOULD NOT, MAY as defined.

Service Naming Convention: Services are named as `{domain}-service` (e.g., `purchasing-service`).

API Versioning: All REST APIs follow `v{major}.{minor}` versioning. Breaking changes increment major version.

Event Naming Convention: Events are past-tense verbs: `{Aggregate}{Verb}ed` (e.g., `PurchaseOrderIssued`).

Path Conventions:

- Service source: `services/{service}/src/`
- Tests: `services/{service}/tests/`
- API specs: `api/openapi/` and `api/asynapi/`

1.4 Reference Documents

Document	Purpose
OpenS2P Architecture Vision v4.4	Strategic intent, principles, ADR catalog
ADR Catalog (ADR-001 through ADR-016)	Decision rationales
OpenAPI Specifications	API contract definitions
AsyncAPI Specifications	Event contract definitions
Technology Radar	Technology selection status
Deployment Profiles	Reference sizing configurations

1.5 Architecture Principles

OpenS2P's design choices throughout this document trace back to a small set of principles, summarised here from the Architecture Vision:

- **API First** — every capability is exposed through a versioned, documented API before any UI consumes it.
- **Event Driven** — cross-domain state changes are communicated as Kafka domain events, not synchronous coupling.
- **AI Native** — AI orchestration, knowledge retrieval, and confidence scoring are first-class architectural layers, not add-ons.
- **Zero Trust** — every request is authenticated and authorised at the service boundary; no implicit trust between services.
- **Multi-Tenant** — tenant isolation is enforced at the data layer (schema-per-tenant) and the authorisation layer (tenant-scoped tokens).
- **Domain-Driven Design** — each service owns a bounded context with explicit aggregates, invariants, and ubiquitous language.
- **Cloud Native** — services are stateless, containerised, and horizontally scalable on Kubernetes.
- **Open Standards** — OpenAPI, AsyncAPI, OpenTelemetry, and OAuth2/OIDC are used in preference to proprietary protocols.

2. Architecture Overview

The diagram below is the single-page view of OpenS2P: how users and external systems reach the platform through the API Gateway, how the domain, workflow, and AI layers relate, and how everything is backed by Kafka and the data layer.

2.1 Master Architecture

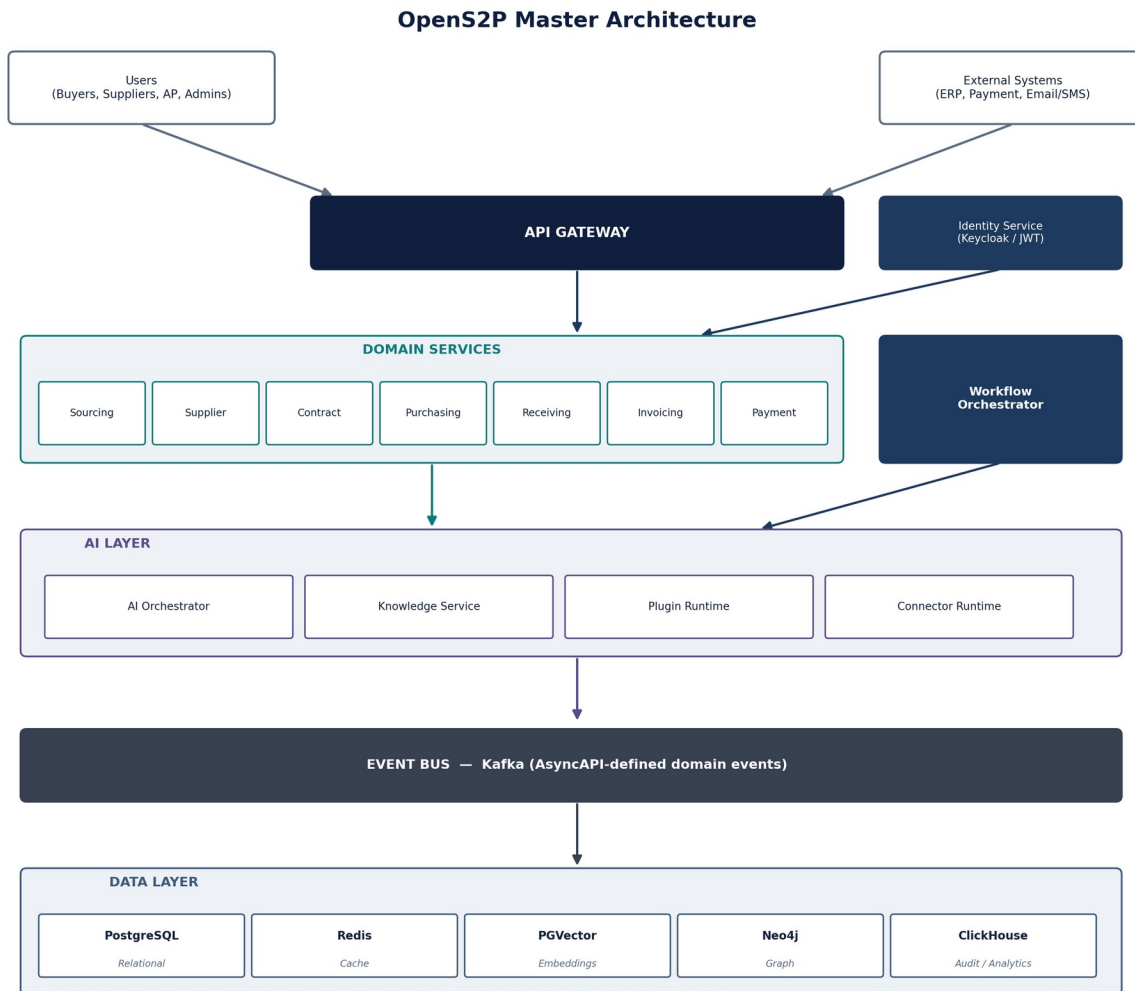


Figure 2.1 — OpenS2P Master Architecture

2.2 Service Topology

OpenS2P comprises 15 core services organised into four layers:

2.3 Service Communication Patterns

Pattern	When Used	Protocol
Event-Driven	Cross-domain state changes	Kafka + AsyncAPI
Request-Reply	Intra-domain queries, OHS calls	REST + OpenAPI
Event-Carried State Transfer	Self-contained event payloads (<1MB)	Kafka
Outbox Pattern	State change + event publishing	PostgreSQL + Debezium
CQRS	Complex read models	Domain events → Projections
Saga	Distributed transactions	Choreographed events

2.3.1 Choosing REST vs. Events

The patterns table above says *when* each pattern applies; this flowchart is the quick decision aid for a developer deciding how a new interaction should communicate.

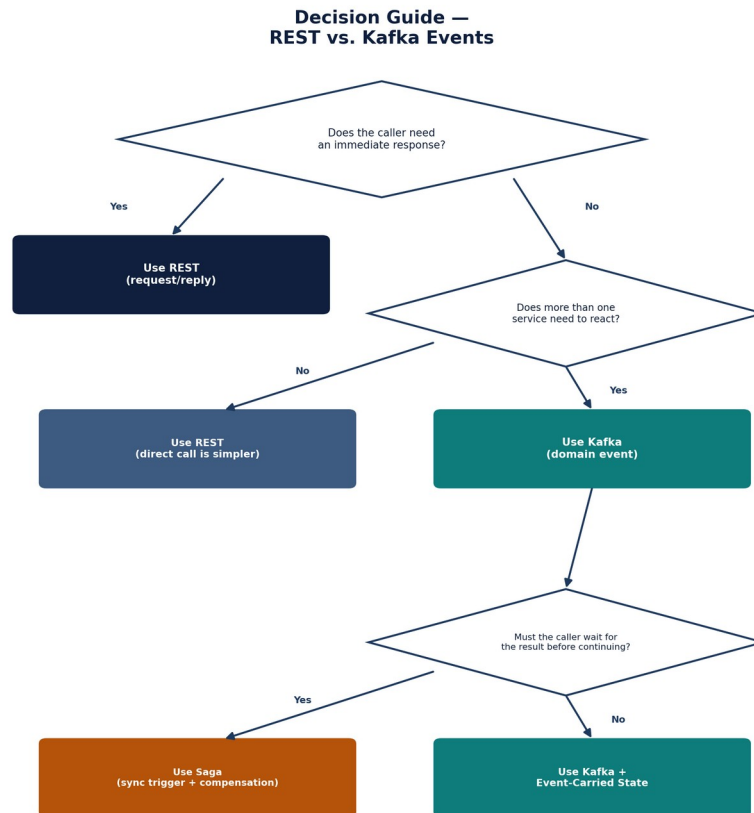


Figure 2.2 — Decision Guide: REST vs. Kafka Events

2.4 Technology Stack Summary

Layer	Technology	Version	ADR
Language (Domain)	TypeScript / Python	5.x / 3.11+	-
Orchestration	LangGraph	0.6+	ADR-003
REST API	FastAPI / Express	0.115+ / 4.x	ADR-013
Events	Kafka	3.5+	ADR-002
Relational Store	PostgreSQL	15+	ADR-004
Vector Store	PGVector	0.5+	ADR-004
Graph Store	Neo4j	5.x	ADR-004
Audit Store	ClickHouse	23.x+	ADR-009
Cache	Redis	7.x	-
Authorisation	OPA	1.0+	ADR-008

Layer	Technology	Version	ADR
Identity	Keycloak	22.x+	ADR-009
AI Inference	vLLM	0.6+	ADR-010
Observability	OpenTelemetry	1.20+	ADR-012
Container Runtime	Kubernetes	1.28+	C-01
Plugin Sandbox	Wasmtime	14.x+	ADR-007

2.4.1 Technology Lifecycle

Positioning each technology on a Trial → Adopt → Hold → Retire radar (the format used by the Technology Radar reference document) gives a one-glance view of platform maturity per component, separate from the version support question answered in Appendix I.

Technology	Radar Stage
Kubernetes, PostgreSQL, Kafka, Redis, Keycloak, OpenTelemetry	Adopt
LangGraph, vLLM, PGVector, Wasmtime, OPA	Adopt
Neo4j, ClickHouse	Trial
Distributed two-phase commit (explicitly not used, see 30.6)	Hold

2.5 Decision Traceability

Every major technology and architecture decision in this document is backed by a recorded ADR in the ADR Catalog. This table maps the most consequential decisions to their rationale record.

Decision	ADR	Summary
Kafka as the event backbone	ADR-002	Event-driven communication over synchronous coupling
LangGraph for agent orchestration	ADR-003	State-machine based agent workflow orchestration
PostgreSQL / PGVector / Neo4j / ClickHouse	ADR-004	Polyglot persistence per data shape (relational, vector, graph, analytical)
Wasmtime plugin sandbox	ADR-007	Safe, capability-scoped plugin execution
OPA for authorisation	ADR-008	Externalised, declarative policy evaluation
Keycloak for identity	ADR-009	Standards-based identity and session management

Decision	ADR	Summary
vLLM for AI inference	ADR-010	Self-hostable, high-throughput model serving
OpenTelemetry for observability	ADR-012	Vendor-neutral tracing, metrics, and logs
FastAPI / Express for REST APIs	ADR-013	Language-idiomatic API frameworks with OpenAPI generation

2.6 ADR-to-SAD Traceability Matrix

Section 2.5 maps decisions to their ADR. This table is the reverse lookup — given an ADR, where in this document is it actually implemented — which is what a governance reviewer typically needs.

ADR	Implemented In
ADR-002 (Kafka event backbone)	Section 20 (Events), Section 20.2 Event Catalog, Chapter 29 sequence diagrams
ADR-003 (LangGraph orchestration)	Section 9 (Workflow Service), Section 10 (AI Orchestrator)
ADR-004 (Polyglot persistence)	Section 18 (Data Model), Section 2.4 Technology Stack Summary
ADR-007 (Wasmtime plugin sandbox)	Section 12 (Plugin Runtime), Figure 29.8 Plugin Execution Sequence
ADR-008 (OPA authorisation)	Section 27.2 (Authorisation)
ADR-009 (Keycloak identity)	Section 15 (Identity Service), Section 27.1 (Authentication)
ADR-010 (vLLM inference)	Section 10 (AI Orchestrator), Section 31.2 (Model Registry & Fallback)
ADR-012 (OpenTelemetry observability)	Section 23 (Monitoring)
ADR-013 (FastAPI / Express APIs)	Section 19 (APIs), Section 19.1 (API Design Standards)

2.7 Vision Principle → ADR → SAD Section

Sections 2.5 and 2.6 trace decisions and ADRs independently. This table closes the loop back to the Architecture Principles in 1.5, so a reviewer can confirm every stated principle actually has a decision and an implementation behind it, not just a mention.

Vision Principle	ADR	SAD Section
API First	ADR-013	19 (APIs), 19.1 (API Design Standards), 19.3 (API Catalog)
Event Driven	ADR-002	20 (Events), 20.2 (Event Catalog), 2.3.1 (REST vs. Events)

Vision Principle	ADR	SAD Section
AI Native	ADR-003, ADR-010	10 (AI Orchestrator), 31 (AI Governance)
Zero Trust	ADR-008, ADR-009	27 (Security Architecture)
Multi-Tenant	ADR-004	27.3 (Multi-Tenancy & Data Isolation)
Domain-Driven Design	—	4–17 (each service's Core Aggregates section)
Cloud Native	—	25 (Deployment), 26.4 (C4 Deployment Diagram)
Open Standards	ADR-012, ADR-013	19 (APIs), 23 (Monitoring)

3. Service Catalogue

3.1 Service Inventory

Note on numbering: the service ID column below starts at 4 and runs to 19 in the original catalog; IDs 1–3 are not assigned to a service in this document. That range is presumed reserved for platform-level components documented elsewhere (e.g. UI, API Gateway, Identity infrastructure) rather than omitted services — this should be confirmed with the architecture team before publication, and the range either documented or the catalog renumbered to start at 1.

#	Service	Domain	Port	Description
4	Sourcing Service	Sourcing	8080	RFx lifecycle, bid management
5	Supplier Service	Supplier	8081	Supplier onboarding, qualification
6	Contract Service	Contract	8082	CLM lifecycle, obligation tracking
7	Purchasing Service	Purchasing	8083	PR/PO lifecycle, approvals
8	Receiving Service	Receiving	8084	Goods receipt, discrepancy handling
9	Invoicing Service	Invoicing	8085	Invoice capture, three-way match
10	Payment Service	Payment	8086	Payment scheduling, ERP sync
11	Workflow Service	Orchestration	8090	Workflow definition, execution
12	AI Orchestrator	AI	8091	Agent orchestration, confidence scoring
13	Knowledge Service	AI	8092	Document indexing, semantic search
14	Plugin Runtime	Extension	8093	Plugin sandbox execution
15	Connector Runtime	Integration	8094	External system integration
16	Marketplace Service	Ecosystem	8095	Plugin publication, trust services
17	Identity Service	Platform	8096	Authentication, session management
18	Notification Service	Platform	8097	Email, webhook, internal notifications

#	Service	Domain	Port	Description
19	Search Service	Platform	8098	Global search across all domains
20	Catalog Service (planned)	Purchasing Content	8099	Item/supplier catalogs, PunchOut, pricing — stub, see Chapter 32
21	Analytics Service (planned)	Intelligence	8100	Spend/supplier analytics, dashboards — stub, see Chapter 33

3.2 Service Dependencies

The diagram below shows which services call or are triggered by which others in the core procure-to-pay flow, plus the Workflow / AI Orchestrator relationship that spans domains.

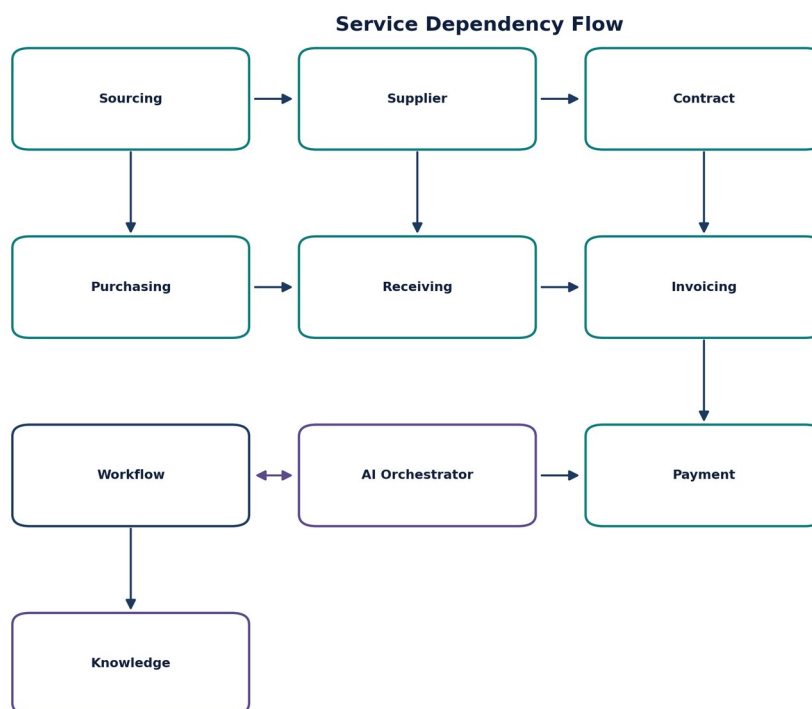


Figure 3.1 — Service Dependency Flow

3.3 Service Dependency Matrix

The same dependency information from Figure 3.1, in matrix form for easier scanning. Publishes/Consumes columns list the primary domain events per service; each service's own chapter has the complete event contract.

Service	Depends On	Publishes (primary)	Consumes (primary)
Sourcing	—	SourcingEventPublished, BidSubmitted, SourcingEventAwarded	SupplierQualified
Supplier	Sourcing	SupplierRegistered, SupplierQualified	SourcingEventAwarded
Contract	Supplier	ContractExecuted, ContractExpired	SupplierQualified, SourcingEventAwarded
Purchasing	Sourcing, Supplier	PurchaseRequisitionCreated, PurchaseOrderIssued	SourcingEventAwarded, SupplierQualified
Receiving	Purchasing, Supplier	GoodsReceiptRecorded	PurchaseOrderIssued
Invoicing	Receiving, Contract	InvoiceMatched, InvoiceApproved	GoodsReceiptRecorded, PurchaseOrderIssued
Payment	Invoicing	PaymentScheduled, PaymentIssued	InvoiceApproved
Workflow Orchestrator	AI Orchestrator	WorkflowStarted, WorkflowCompleted	(cross-domain, triggered by any service)
AI Orchestrator	Knowledge Service	AgentActionProposed, AgentActionApproved	WorkflowStarted
Catalog (planned)	Supplier, Contract, Purchasing, Search, AI Orchestrator	TBD — defined at detailed design	SupplierQualified, ContractExecuted
Analytics (planned)	Event streams (all domains), ClickHouse, Knowledge Service, AI Orchestrator	TBD — defined at detailed design	(consumes broadly; see SAD 20.2 Event Catalog)

3.4 Domain Ownership Matrix

One row per service, pulling together the owning team (from each service's factsheet), its API base path (19.3), its primary published events (20.2), and its primary data store — the single table a new contributor or a governance reviewer needs to orient themselves.

Domain	Owner	API Base Path	Primary Events Published	Primary Data Store
Sourcing	Sourcing Team	/api/v1/sourcing	SourcingEventPublished,	PostgreSQL

Domain	Owner	API Base Path	Primary Events Published	Primary Data Store
			SourcingEventAwarded	
Supplier	Supplier Team	/api/v1/suppliers	SupplierRegistered, SupplierQualified	PostgreSQL
Contract	Contract Team	/api/v1/contracts	ContractExecuted, ContractExpired	PostgreSQL
Purchasing	Purchasing Team	/api/v1/purchasing	PurchaseRequisitionCreated, PurchaseOrderIssued	PostgreSQL
Receiving	Purchasing Team	/api/v1/receiving	GoodsReceiptRecorded	PostgreSQL
Invoicing	Finance Team	/api/v1/invoicing	InvoiceMatched, InvoiceApproved	PostgreSQL
Payment	Finance Team	/api/v1/payments	PaymentScheduled, PaymentIssued	PostgreSQL
Workflow Orchestration	Platform Team	/api/v1/workflows	WorkflowStarted, WorkflowCompleted	PostgreSQL
AI Orchestrator	AI Team	/api/v1/ai	AgentActionProposed, AgentActionApproved	PostgreSQL + audit via ClickHouse
Knowledge	AI Team	/api/v1/knowledge	(read-heavy; indexing events internal)	PGVector + Neo4j
Plugin Runtime	Platform Team	/api/v1/plugins	PluginExecuted	PostgreSQL
Connector Runtime	Platform Team	/api/v1/connectors	ConnectorInvoked	PostgreSQL
Marketplace	Platform Team	/api/v1/marketplace	(publication/trust events, internal)	PostgreSQL
Identity	Platform Team	/api/v1/identity	(session events, internal)	Keycloak-managed store
Notification	Platform Team	/api/v1/notifications	(consumes broadly, publishes little)	PostgreSQL
Search	Platform Team	/api/v1/search	(read-only; no events published)	PostgreSQL + search index
Catalog (planned)	TBD	/api/v1/catalog (reserved)	TBD	TBD
Analytics (planned)	TBD	/api/v1/analytics (reserved)	(read-heavy; consumes broadly)	ClickHouse (existing audit store, extended scope TBD)

4. Sourcing Service

4.1 Service Overview

Domain	Sourcing Management
Owning Team	Sourcing Team
Primary Responsibility	RFx lifecycle, bid management, award decisions, savings tracking

4.2 Core Aggregates

4.2.1 SourcingEvent (Aggregate Root)

TYPESCRIPT

```
interface SourcingEvent {
  id: UUID;
  eventType: 'RFI' | 'RFQ' | 'RFP' | 'RFx';
  title: string;
  description: string;
  status: SourcingStatus;
  timeline: {
    issueDate: Date;
    responseDeadline: Date;
    evaluationPeriod: DateRange;
    awardDate?: Date;
  };
  organisationId: UUID;
  departmentId?: UUID;
  projectCode: string;
  createdBy: UserReference;
  assignedBuyer: UserReference;
  evaluationCriteria: EvaluationCriterion[];
  technicalRequirements: TechnicalRequirement[];
  commercialRequirements: CommercialRequirement[];
  bids: Bid[];
  attachments: DocumentReference[];
  metadata: Map<string, any>;
  createdAt: Date;
  updatedAt: Date;
  version: number;
}

type SourcingStatus =
  | 'DRAFT'
  | 'PUBLISHED'
  | 'UNDER_EVALUATION'
  | 'AWAITING_APPROVAL'
```

```
| 'AWARDED'  
| 'CANCELLED'  
| 'ARCHIVED';
```

4.2.2 Bid (Entity)

TYPESCRIPT

```
interface Bid {  
  id: UUID;  
  sourcingEventId: UUID;  
  supplierId: UUID;  
  submittedBy: UserReference;  
  submissionDate: Date;  
  technicalScore: number;  
  commercialScore: number;  
  totalScore: number;  
  lineItems: BidLineItem[];  
  documents: DocumentReference[];  
  terms: BidTerms;  
  status: BidStatus;  
  evaluationNotes: string;  
  isWinner: boolean;  
  createdAt: Date;  
  updatedAt: Date;  
}  
  
interface BidLineItem {  
  id: UUID;  
  description: string;  
  quantity: number;  
  unitPrice: number;  
  currency: string;  
  totalPrice: number;  
  deliveryDate: Date;  
  specifications: Map<string, any>;  
}
```

4.3 API Specification

4.3.1 OpenAPI Contract

File: api/openapi/sourcing-service.yaml

YAML

```
openapi: 3.1.0  
info:  
  title: Sourcing Service API
```

```
version: v1.0
description: RFx lifecycle, bid management, and award decisions

paths:
  /api/v1/sourcing/events:
    post:
      summary: Create a new sourcing event
      operationId: createSourcingEvent
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateSourcingEventRequest'
      responses:
        201:
          description: Sourcing event created
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SourcingEventResponse'
        400:
          $ref: '#/components/responses/BadRequest'
        401:
          $ref: '#/components/responses/Unauthorised'

  /api/v1/sourcing/events/{eventId}:
    get:
      summary: Get sourcing event by ID
      operationId: getSourcingEvent
      parameters:
        - name: eventId
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        200:
          description: Sourcing event found
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SourcingEventResponse'
        404:
          $ref: '#/components/responses/NotFound'

    patch:
      summary: Update sourcing event
      operationId: updateSourcingEvent
      parameters:
        - name: eventId
```

```
      in: path
      required: true
      schema:
        type: string
        format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UpdateSourcingEventRequest'
    responses:
      200:
        description: Sourcing event updated
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SourcingEventResponse'
      409:
        $ref: '#/components/responses/Conflict'

/api/v1/sourcing/events/{eventId}/publish:
  post:
    summary: Publish sourcing event to suppliers
    operationId: publishSourcingEvent
    parameters:
      - name: eventId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      202:
        description: Publish initiated
      403:
        $ref: '#/components/responses/Forbidden'

/api/v1/sourcing/events/{eventId}/bids:
  post:
    summary: Submit a bid
    operationId: submitBid
    parameters:
      - name: eventId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
```



```
    application/json:
      schema:
        $ref: '#/components/schemas/SubmitBidRequest'
  responses:
    201:
      description: Bid submitted
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/BidResponse'
    409:
      description: Bid deadline passed

/api/v1/sourcing/events/{eventId}/evaluate:
  post:
    summary: Evaluate all bids for an event
    operationId: evaluateBids
    parameters:
      - name: eventId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/EvaluateBidsRequest'
    responses:
      202:
        description: Evaluation initiated
      403:
        $ref: '#/components/responses/Forbidden'

/api/v1/sourcing/events/{eventId}/award:
  post:
    summary: Award the sourcing event
    operationId: awardSourcingEvent
    parameters:
      - name: eventId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
```

```
        $ref: '#/components/schemas/AwardRequest'
responses:
  202:
    description: Award approval requested
  403:
    $ref: '#/components/responses/Forbidden'

components:
  schemas:
    SourcingEventResponse:
      type: object
      properties:
        id:
          type: string
          format: uuid
        eventType:
          type: string
          enum: [RFI, RFQ, RFP, RFx]
        title:
          type: string
        description:
          type: string
        status:
          type: string
          enum: [DRAFT, PUBLISHED, UNDER_EVALUATION, AWAITING_APPROVAL, AWARDED,
CANCELLED, ARCHIVED]
        timeline:
          $ref: '#/components/schemas/Timeline'
        organisationId:
          type: string
          format: uuid
        createdBy:
          $ref: '#/components/schemas/UserReference'
        evaluationCriteria:
          type: array
          items:
            $ref: '#/components/schemas/EvaluationCriterion'
        bids:
          type: array
          items:
            $ref: '#/components/schemas/BidSummary'
        createdAt:
          type: string
          format: date-time
        updatedAt:
          type: string
          format: date-time
        version:
          type: integer

    CreateSourcingEventRequest:
      type: object
```

```
required: [eventType, title, projectCode, timeline]
properties:
  eventType:
    type: string
    enum: [RFI, RFQ, RFP, RFx]
  title:
    type: string
  description:
    type: string
  projectCode:
    type: string
  departmentId:
    type: string
    format: uuid
  timeline:
    $ref: '#/components/schemas/TimelineInput'
  evaluationCriteria:
    type: array
    items:
      $ref: '#/components/schemas/EvaluationCriterionInput'
  technicalRequirements:
    type: array
    items:
      $ref: '#/components/schemas/TechnicalRequirementInput'
  commercialRequirements:
    type: array
    items:
      $ref: '#/components/schemas/CommercialRequirementInput'

Timeline:
  type: object
  properties:
    issueDate:
      type: string
      format: date-time
    responseDeadline:
      type: string
      format: date-time
    evaluationPeriod:
      $ref: '#/components/schemas/DateRange'
    awardDate:
      type: string
      format: date-time

BidResponse:
  type: object
  properties:
    id:
      type: string
      format: uuid
    supplierId:
      type: string
```

```

    format: uuid
  submissionDate:
    type: string
    format: date-time
  lineItems:
    type: array
    items:
      $ref: '#/components/schemas/BidLineItemResponse'
  totalAmount:
    type: number
  currency:
    type: string
  status:
    type: string
    enum: [SUBMITTED, UNDER_REVIEW, REJECTED, ACCEPTED, WINNER]

```

4.4 Events Published

4.4.1 AsyncAPI Specification

File: `api/asynccapi/sourcing-events.yaml`

YAML

```

asynccapi: 3.0.0
info:
  title: Sourcing Service Events
  version: v1.0
  description: Domain events emitted by Sourcing Service

channels:
  sourcing:
    address: sourcing-events
    messages:
      RFxPublished:
        $ref: '#/components/messages/RFxPublished'
      BidSubmitted:
        $ref: '#/components/messages/BidSubmitted'
      BidEvaluated:
        $ref: '#/components/messages/BidEvaluated'
      AwardApproved:
        $ref: '#/components/messages/AwardApproved'
      AwardRejected:
        $ref: '#/components/messages/AwardRejected'
      SourcingCancelled:
        $ref: '#/components/messages/SourcingCancelled'

components:
  messages:
    RFxPublished:
      summary: A sourcing event has been published to suppliers

```

```
payload:
  type: object
  required: [eventId, eventType, title, responseDeadline, organisationId]
  properties:
    eventId:
      type: string
      format: uuid
    eventType:
      type: string
      enum: [RFI, RFQ, RFP, RFx]
    title:
      type: string
    description:
      type: string
    responseDeadline:
      type: string
      format: date-time
    organisationId:
      type: string
      format: uuid
    publishedAt:
      type: string
      format: date-time
    traceId:
      type: string
    correlationId:
      type: string
```

```
BidSubmitted:
  summary: A supplier has submitted a bid
  payload:
    type: object
    required: [bidId, eventId, supplierId, submittedBy]
    properties:
      bidId:
        type: string
        format: uuid
      eventId:
        type: string
        format: uuid
      supplierId:
        type: string
        format: uuid
      submittedBy:
        $ref: '#/components/schemas/UserReference'
      submittedAt:
        type: string
        format: date-time
      bidSummary:
        type: object
        properties:
          totalAmount:
```

```
        type: number
    lineItems:
        type: integer
    proposedDeliveryDate:
        type: string
        format: date
    traceId:
        type: string
    correlationId:
        type: string
```

4.5 Data Model

4.5.1 PostgreSQL Schema

SQL

```
-- Source: services/sourcing/src/infrastructure/schema/001_initial.sql

CREATE TABLE sourcing_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_type VARCHAR(20) NOT NULL CHECK (event_type IN ('RFI', 'RFQ', 'RFP', 'RFx')),
    title VARCHAR(255) NOT NULL,
    description TEXT,
    status VARCHAR(30) NOT NULL DEFAULT 'DRAFT',
    timeline JSONB NOT NULL,
    organisation_id UUID NOT NULL,
    department_id UUID,
    project_code VARCHAR(50) NOT NULL,
    created_by JSONB NOT NULL,
    assigned_buyer JSONB,
    evaluation_criteria JSONB DEFAULT '[]',
    technical_requirements JSONB DEFAULT '[]',
    commercial_requirements JSONB DEFAULT '[]',
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    version INTEGER DEFAULT 1,
    tenant_id VARCHAR(50) NOT NULL
);

CREATE INDEX idx_sourcing_events_organisation ON sourcing_events (organisation_id);
CREATE INDEX idx_sourcing_events_status ON sourcing_events (status);
CREATE INDEX idx_sourcing_events_created_at ON sourcing_events (created_at DESC);
CREATE INDEX idx_sourcing_events_project_code ON sourcing_events (project_code);

CREATE TABLE bids (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sourcing_event_id UUID NOT NULL REFERENCES sourcing_events(id) ON DELETE CASCADE,
    supplier_id UUID NOT NULL,
```

```
submitted_by JSONB NOT NULL,  
submission_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
technical_score DECIMAL(5,2),  
commercial_score DECIMAL(5,2),  
total_score DECIMAL(5,2),  
line_items JSONB NOT NULL,  
documents JSONB DEFAULT '[]',  
terms JSONB,  
status VARCHAR(20) NOT NULL DEFAULT 'SUBMITTED',  
evaluation_notes TEXT,  
is_winner BOOLEAN DEFAULT FALSE,  
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
tenant_id VARCHAR(50) NOT NULL  
);  
  
CREATE INDEX idx_bids_sourcing_event ON bids (sourcing_event_id);  
CREATE INDEX idx_bids_supplier ON bids (supplier_id);  
CREATE INDEX idx_bids_status ON bids (status);  
  
-- RLS policies  
ALTER TABLE sourcing_events ENABLE ROW LEVEL SECURITY;  
ALTER TABLE bids ENABLE ROW LEVEL SECURITY;  
  
CREATE POLICY tenant_isolation_sourcing_events ON sourcing_events  
    USING (tenant_id = current_setting('app.current_tenant'));  
  
CREATE POLICY tenant_isolation_bids ON bids  
    USING (tenant_id = current_setting('app.current_tenant'));
```

4.6 Implementation Structure

CODE

```
services/sourcing/  
├── src/  
│   ├── domain/  
│   │   ├── aggregates/  
│   │   │   ├── SourcingEvent.ts  
│   │   │   └── Bid.ts  
│   │   ├── value-objects/  
│   │   │   ├── Timeline.ts  
│   │   │   ├── EvaluationCriterion.ts  
│   │   │   └── UserReference.ts  
│   │   ├── events/  
│   │   │   ├── RFxPublished.ts  
│   │   │   ├── BidSubmitted.ts  
│   │   │   └── BidEvaluated.ts  
│   │   └── repositories/  
│   │       ├── ISourcingEventRepository.ts  
│   │       └── IBidRepository.ts  
│   ├── application/  
│   │   ├── commands/  
│   │   │   ├── CreateSourcingEvent.ts  
│   │   │   ├── PublishSourcingEvent.ts  
│   │   │   ├── SubmitBid.ts  
│   │   │   └── AwardSourcingEvent.ts  
│   │   ├── queries/  
│   │   │   ├── GetSourcingEvent.ts  
│   │   │   ├── ListSourcingEvents.ts  
│   │   │   └── GetBidsForEvent.ts  
│   │   └── handlers/  
│   │       ├── CreateSourcingEventHandler.ts  
│   │       ├── PublishSourcingEventHandler.ts  
│   │       └── SubmitBidHandler.ts  
│   ├── infrastructure/  
│   │   ├── repositories/  
│   │   │   ├── PostgresSourcingEventRepository.ts  
│   │   │   └── PostgresBidRepository.ts  
│   │   ├── messaging/  
│   │   │   ├── KafkaEventPublisher.ts  
│   │   │   └── SourcingEventSerializer.ts  
│   │   ├── storage/  
│   │   │   └── S3DocumentStore.ts  
│   │   ├── config/  
│   │   │   ├── database.ts  
│   │   │   ├── kafka.ts  
│   │   │   └── opentelemetry.ts  
│   └── presentation/  
│       ├── controllers/  
│       │   ├── SourcingEventController.ts  
│       │   └── BidController.ts  
│       ├── dto/  
│       │   ├── CreateSourcingEventDTO.ts  
│       │   └── SubmitBidDTO.ts  
│       └── middleware/  
└──
```



```

├── authentication.ts
├── authorisation.ts
├── tenant-context.ts
├── tests/
│   ├── unit/
│   │   ├── domain/
│   │   │   └── SourcingEvent.test.ts
│   │   └── application/
│   │       └── CreateSourcingEvent.test.ts
│   ├── integration/
│   │   ├── repositories/
│   │   │   └── SourcingEventRepository.test.ts
│   │   └── messaging/
│   │       └── EventPublishing.test.ts
│   └── contract/
│       ├── pact/
│       │   └── SourcingServicePact.test.ts
│       └── asyncapi/
│           └── EventSchemaValidation.test.ts
├── openapi.yaml
├── asyncapi.yaml
├── Dockerfile
├── helm/
│   └── values.yaml
└── package.json

```

4.7 Business Rules

4.7.1 Validation Rules

TYPESCRIPT

```

// Source: services/sourcing/src/domain/rules/SourcingRules.ts

export class SourcingRules {
  /**
   * Ensure sourcing event is valid for publication
   * @throws DomainError if validation fails
   */
  static validateForPublication(event: SourcingEvent): void {
    // Must have a response deadline
    if (!event.timeline.responseDeadline) {
      throw new DomainError('RESPONSE_DEADLINE_REQUIRED', 'Response deadline must be set before publication');
    }

    // Response deadline must be in the future
    if (event.timeline.responseDeadline <= new Date()) {
      throw new DomainError('RESPONSE_DEADLINE_PAST', 'Response deadline must be in the future');
    }
  }
}

```

```
}

// Must have at least one evaluation criterion
if (event.evaluationCriteria.length === 0) {
  throw new DomainError('EVALUATION_CRITERIA_REQUIRED', 'At least one evaluation
criterion must be defined');
}

// Weighted scores must sum to 100%
const totalWeight = event.evaluationCriteria.reduce((sum, c) => sum + c.weight, 0);
if (Math.abs(totalWeight - 100) > 0.01) {
  throw new DomainError('INVALID_WEIGHT_TOTAL',
    `Evaluation criteria weights must sum to 100%, currently ${totalWeight}%`);
}

// Required fields
if (!event.title || event.title.length < 3) {
  throw new DomainError('INVALID_TITLE', 'Title must be at least 3 characters');
}

if (!event.projectCode) {
  throw new DomainError('PROJECT_CODE_REQUIRED', 'Project code is required for
publication');
}
}

/**
 * Validate bid submission
 */
static validateBidSubmission(event: SourcingEvent, bid: Bid): void {
  // Event must be published
  if (event.status !== 'PUBLISHED' && event.status !== 'UNDER_EVALUATION') {
    throw new DomainError('EVENT_NOT_OPEN',
      `Event must be published to accept bids. Current status: ${event.status}`);
  }

  // Must be within response deadline
  if (bid.submissionDate > event.timeline.responseDeadline) {
    throw new DomainError('BID_DEADLINE_PASSED',
      `Bid must be submitted by ${event.timeline.responseDeadline.toISOString()}`);
  }

  // Bid must have at least one line item
  if (!bid.lineItems || bid.lineItems.length === 0) {
    throw new DomainError('BID_LINE_ITEMS_REQUIRED', 'Bid must include at least one
line item');
  }

  // All line items must have positive quantity and price
  for (const item of bid.lineItems) {
    if (item.quantity <= 0) {
      throw new DomainError('INVALID_QUANTITY', `Line item ${item.id}: quantity must be`

```

```

    positive`);
    }
    if (item.unitPrice < 0) {
        throw new DomainError('INVALID_PRICE', `Line item ${item.id}: price cannot be
negative`);
    }
}

// Supplier cannot have existing winning bid for this event
// (Checked by repository)
}

/**
 * Validate award decision
 */
static validateAward(event: SourcingEvent, winningBidId: UUID): void {
    // Event must have all bids evaluated
    if (event.status !== 'UNDER_EVALUATION') {
        throw new DomainError('INVALID_AWARD_STATE',
            `Event must be under evaluation to award. Current status: ${event.status}`);
    }

    // Winning bid must exist
    const winningBid = event.bids.find(b => b.id === winningBidId);
    if (!winningBid) {
        throw new DomainError('BID_NOT_FOUND', `Bid ${winningBidId} not found in event`);
    }

    // Winning bid must be evaluated
    if (winningBid.status !== 'EVALUATED') {
        throw new DomainError('BID_NOT_EVALUATED', 'Winning bid must be evaluated before
award');
    }
}
}
}

```

4.7.2 Scoring Rules

TYPESCRIPT

```

// Source: services/sourcing/src/domain/rules/ScoringRules.ts

export class ScoringRules {
    /**
     * Calculate total score for a bid based on evaluation criteria
     */
    static calculateBidScore(bid: Bid, criteria: EvaluationCriterion[]): BidScore {
        let totalScore = 0;
        const details: ScoringDetail[] = [];

        for (const criterion of criteria) {

```

```

        const score = this.evaluateCriterion(bid, criterion);
        const weightedScore = (score / 100) * criterion.weight;
        totalScore += weightedScore;

        details.push({
            criterionId: criterion.id,
            criterionName: criterion.name,
            rawScore: score,
            weight: criterion.weight,
            weightedScore: weightedScore,
            justification: criterion.autoScore ? 'Automated evaluation' : 'Manual evaluation
required'
        });
    }

    return {
        totalScore: totalScore,
        maxScore: 100,
        details: details,
        confidence: this.calculateConfidence(details)
    };
}

/**
 * Evaluate a single criterion against the bid
 */
private static evaluateCriterion(bid: Bid, criterion: EvaluationCriterion): number {
    switch (criterion.type) {
        case 'PRICE':
            return this.evaluatePriceCriterion(bid, criterion);
        case 'TECHNICAL':
            return this.evaluateTechnicalCriterion(bid, criterion);
        case 'DELIVERY':
            return this.evaluateDeliveryCriterion(bid, criterion);
        case 'QUALITY':
            return this.evaluateQualityCriterion(bid, criterion);
        case 'COMPLIANCE':
            return this.evaluateComplianceCriterion(bid, criterion);
        default:
            return 0;
    }
}

/**
 * Price scoring: lowest price gets maximum points
 */
private static evaluatePriceCriterion(bid: Bid, criterion: EvaluationCriterion): number
{
    const prices = bid.lineItems.map(item => item.totalPrice);
    const totalPrice = prices.reduce((sum, p) => sum + p, 0);
    // Scale: cheapest bid gets 100, most expensive gets 0
    // This requires reference to all bids - handled at aggregate level

```

```
// Returns score 0-100
return 0; // Implementation details in full code
}
}
```

4.8 Error Handling

TYPESCRIPT

```
// Source: services/sourcing/src/infrastructure/errors/ErrorHandler.ts

export enum SourcingErrorCode {
  // Domain errors
  EVENT_NOT_FOUND = 'SRC-001',
  BID_NOT_FOUND = 'SRC-002',
  INVALID_EVENT_STATUS = 'SRC-003',
  RESPONSE_DEADLINE_PASSED = 'SRC-004',
  DUPLICATE_BID = 'SRC-005',
  EVALUATION_CRITERIA_INCOMPLETE = 'SRC-006',

  // Business rule errors
  WEIGHT_TOTAL_ERROR = 'SRC-101',
  INVALID_PROJECT_CODE = 'SRC-102',
  BID_LINE_ITEMS_EMPTY = 'SRC-103',

  // Authorization errors
  UNAUTHORISED_TO_AWARD = 'SRC-201',
  UNAUTHORISED_TO_EVALUATE = 'SRC-202',
  UNAUTHORISED_TO_PUBLISH = 'SRC-203',

  // Infrastructure errors
  EVENT_PUBLISH_FAILED = 'SRC-301',
  DOCUMENT_STORE_FAILURE = 'SRC-302',
  REPOSITORY_ERROR = 'SRC-303'
}

export class SourcingError extends Error {
  constructor(
    public code: SourcingErrorCode,
    message: string,
    public details?: Record<string, any>
  ) {
    super(message);
    this.name = 'SourcingError';
  }
}

export const errorHandler: ErrorRequestHandler = (err, req, res, next) => {
  if (err instanceof SourcingError) {
    return res.status(400).json({
```

```
        error: {
            code: err.code,
            message: err.message,
            details: err.details,
            timestamp: new Date().toISOString(),
            requestId: req.headers['x-request-id']
        }
    });
}

// Log unexpected errors
logger.error('Unhandled error', { error: err, requestId: req.headers['x-request-id'] });

return res.status(500).json({
    error: {
        code: 'INTERNAL_ERROR',
        message: 'An unexpected error occurred',
        timestamp: new Date().toISOString(),
        requestId: req.headers['x-request-id']
    }
});
};
```

5. Supplier Service

5.1 Service Overview

Domain	Supplier Management
Owning Team	Supplier Team
Primary Responsibility	Supplier onboarding, qualification, performance scoring, financial health monitoring

5.2 Core Aggregates

Suppliers move from registration through qualification to active status, with explicit rejection and suspension paths so a supplier can be taken out of an active sourcing pool without deleting its history.

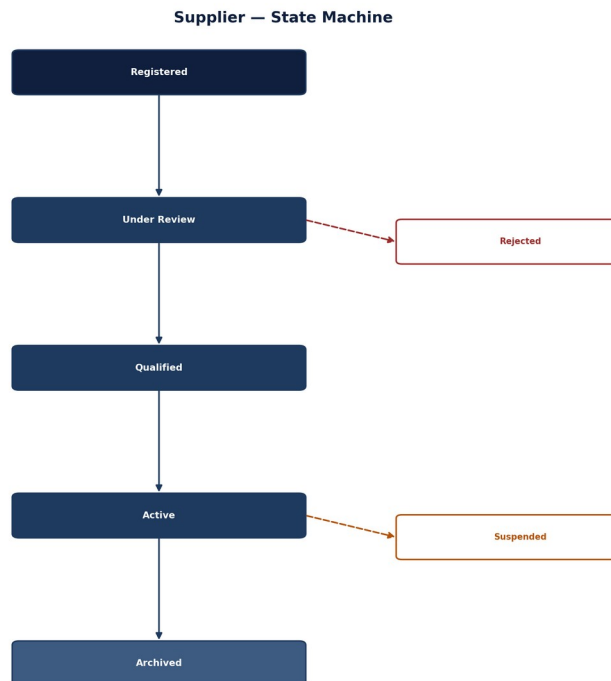


Figure 5.1 — Supplier State Machine

5.2.1 Supplier (Aggregate Root)

TYPESCRIPT

```
interface Supplier {  
  id: UUID;  
  legalName: string;  
  tradingName?: string;  
  registrationNumber: string;  
}
```

```
taxId: string;
addresses: Address[];
contacts: Contact[];
classifications: SupplierClassification[];
certifications: Certification[];
financialProfile: FinancialProfile;
performanceScore: PerformanceScore;
riskProfile: RiskProfile;
status: SupplierStatus;
createdBy: UserReference;
createdAt: Date;
updatedAt: Date;
version: number;
}

interface Address {
  id: UUID;
  type: 'REGISTERED' | 'OPERATIONAL' | 'BILLING' | 'SHIPPING';
  line1: string;
  line2?: string;
  city: string;
  state: string;
  country: string;
  postalCode: string;
  isDefault: boolean;
}

interface Contact {
  id: UUID;
  firstName: string;
  lastName: string;
  email: string;
  phone: string;
  role: 'PRIMARY' | 'FINANCE' | 'TECHNICAL' | 'OPERATIONS' | 'LEGAL';
  isDefault: boolean;
}

interface FinancialProfile {
  annualRevenue: number;
  currency: string;
  employeeCount: number;
  fiscalYearEnd: string; // MM-DD
  creditScore?: {
    provider: 'DUN_BRADSTREET' | 'CREDITSAFE' | 'OTHER';
    score: number;
    scoreDate: Date;
    rating: string;
  };
  bankruptcyFlag: boolean;
  litigations: LitigationRecord[];
}
```


5.2.2 Qualification (Entity)

TYPESCRIPT

```
interface Qualification {
  id: UUID;
  supplierId: UUID;
  type: 'FINANCIAL' | 'TECHNICAL' | 'COMPLIANCE' | 'ESG' | 'SECURITY';
  status: 'PENDING' | 'IN_PROGRESS' | 'QUALIFIED' | 'DISQUALIFIED' | 'EXPIRED';
  validFrom: Date;
  validUntil: Date;
  assessedBy: UserReference;
  assessmentDate: Date;
  criteria: QualificationCriterion[];
  documents: DocumentReference[];
  notes: string;
  createdAt: Date;
  updatedAt: Date;
}
```

5.3 Events Published

YAML

```
api/asynccapi/supplier-events.yaml
channels:
  supplier:
    address: supplier-events
    messages:
      SupplierRegistered:
        summary: A new supplier has been registered
        payload:
          type: object
          required: [supplierId, legalName, registrationNumber]
          properties:
            supplierId:
              type: string
              format: uuid
            legalName:
              type: string
            registrationNumber:
              type: string
            taxId:
              type: string
            classification:
              type: array
              items:
                type: string
            registeredAt:
              type: string
              format: date-time
```

```
    registeredBy:
      $ref: '#/components/schemas/UserReference'
    traceId:
      type: string

QualificationApproved:
  summary: A supplier qualification has been approved
  payload:
    type: object
    required: [qualificationId, supplierId, type]
    properties:
      qualificationId:
        type: string
        format: uuid
      supplierId:
        type: string
        format: uuid
      type:
        type: string
        enum: [FINANCIAL, TECHNICAL, COMPLIANCE, ESG, SECURITY]
      validFrom:
        type: string
        format: date-time
      validUntil:
        type: string
        format: date-time
      approvedBy:
        $ref: '#/components/schemas/UserReference'
      approvedAt:
        type: string
        format: date-time
      traceId:
        type: string

RiskFlagRaised:
  summary: A risk flag has been raised for a supplier
  payload:
    type: object
    required: [supplierId, riskType, severity]
    properties:
      supplierId:
        type: string
        format: uuid
      riskType:
        type: string
        enum: [FINANCIAL, COMPLIANCE, REPUTATIONAL, OPERATIONAL, ESG]
      severity:
        type: string
        enum: [LOW, MEDIUM, HIGH, CRITICAL]
      description:
        type: string
      source:
```

```
    type: string
  detectedAt:
    type: string
    format: date-time
  traceId:
    type: string
```

5.4 Data Model

SQL

```
-- services/supplier/src/infrastructure/schema/001_initial.sql

CREATE TABLE suppliers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  legal_name VARCHAR(255) NOT NULL,
  trading_name VARCHAR(255),
  registration_number VARCHAR(50) NOT NULL,
  tax_id VARCHAR(50),
  addresses JSONB NOT NULL DEFAULT '[]',
  contacts JSONB NOT NULL DEFAULT '[]',
  classifications JSONB DEFAULT '[]',
  certifications JSONB DEFAULT '[]',
  financial_profile JSONB,
  performance_score JSONB,
  risk_profile JSONB,
  status VARCHAR(30) NOT NULL DEFAULT 'ACTIVE',
  created_by JSONB NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  version INTEGER DEFAULT 1,
  tenant_id VARCHAR(50) NOT NULL,
  UNIQUE(tenant_id, registration_number)
);

CREATE TABLE qualifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  supplier_id UUID NOT NULL REFERENCES suppliers(id) ON DELETE CASCADE,
  type VARCHAR(20) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  valid_from TIMESTAMP WITH TIME ZONE,
  valid_until TIMESTAMP WITH TIME ZONE,
  assessed_by JSONB,
  assessment_date TIMESTAMP WITH TIME ZONE,
  criteria JSONB NOT NULL,
  documents JSONB DEFAULT '[]',
  notes TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  tenant_id VARCHAR(50) NOT NULL
```

```
);

CREATE INDEX idx_suppliers_status ON suppliers (status);
CREATE INDEX idx_suppliers_legal_name ON suppliers (legal_name);
CREATE INDEX idx_suppliers_tenant ON suppliers (tenant_id);
CREATE INDEX idx_qualifications_supplier ON qualifications (supplier_id);
CREATE INDEX idx_qualifications_status ON qualifications (status);

-- Neo4j integration for supplier network
-- CREATE (s:Supplier {id: $id, name: $name, status: $status})
-- Relationship: (s)-[:HAS_CONTRACT]->(c:Contract)
-- Relationship: (s)-[:SUPPLIES]->(p:Product)
```

6. Contract Service

6.1 Service Overview

Domain	Contract Lifecycle Management
Owning Team	Contract Team
Primary Responsibility	Universal contract aggregate, obligation tracking, renewal management, risk scoring

6.2 Core Aggregates

A contract's lifecycle runs from drafting through negotiation, execution, and eventual expiry, with an explicit termination path distinct from natural expiry.

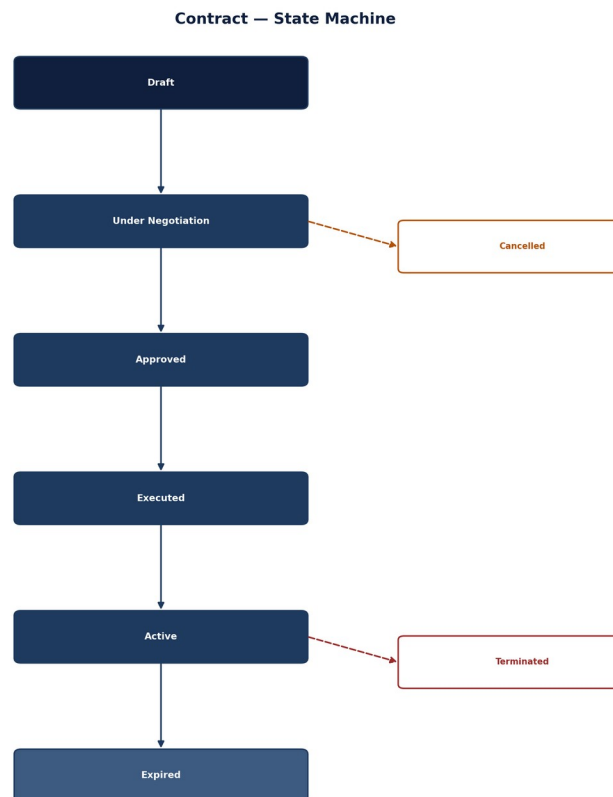


Figure 6.1 — Contract State Machine

6.2.1 Universal Contract (Aggregate Root)

TYPESCRIPT

```
interface Contract {
  id: UUID;
  contractType: ContractType;
  title: string;
  reference: string;
  parties: ContractParty[];
  effectiveDate: Date;
  expiryDate: Date;
  terms: ContractTerms;
  obligations: Obligation[];
  amendments: Amendment[];
  documents: DocumentReference[];
  riskScore: ContractRiskScore;
  status: ContractStatus;
  aiAnalysis?: ContractAIAnalysis;
  version: number;
  createdAt: Date;
  updatedAt: Date;
}

type ContractType =
  | 'MSA'           // Master Services Agreement
  | 'SOW'           // Statement of Work
  | 'NDA'           // Non-Disclosure Agreement
  | 'PURCHASE'      // Purchase Agreement
  | 'SALE'          // Sales Agreement
  | 'LICENSE'       // License Agreement
  | 'SERVICE'      // Service Agreement
  | 'EMPLOYMENT'    // Employment Contract
  | 'FRAMEWORK';   // Framework Agreement

interface ContractTerms {
  paymentTerms: PaymentTerms;
  deliveryTerms: DeliveryTerms;
  serviceLevels: ServiceLevelAgreement[];
  terminationClause: TerminationClause;
  governingLaw: string;
  jurisdiction: string;
  confidentialityClause: ConfidentialityClause;
  indemnityClause: IndemnityClause;
  insuranceRequirements: InsuranceRequirement[];
  customClauses: CustomClause[];
}

interface Obligation {
  id: UUID;
  type: ObligationType;
  description: string;
  responsibleParty: UUID; // Party ID
  dueDate: Date;
  status: ObligationStatus;
  completionEvidence?: DocumentReference;
```

```

    reminderSentAt: Date[];
    escalatedAt: Date[];
    aiDetectedSource: string; // Clause reference
    createdAt: Date;
    updatedAt: Date;
  }

  type ObligationType =
    | 'RENEWAL'
    | 'PAYMENT'
    | 'DELIVERY'
    | 'SERVICE_LEVEL'
    | 'REPORTING'
    | 'COMPLIANCE'
    | 'INSURANCE'
    | 'AUDIT'
    | 'INDEMNITY'
    | 'CONFIDENTIALITY';

```

6.3 Events Published

YAML

```

api/asynccapi/contract-events.yaml
channels:
  contract:
    address: contract-events
    messages:
      ContractDrafted:
        summary: A new contract has been drafted
        payload:
          type: object
          required: [contractId, contractType, title, parties]
          properties:
            contractId:
              type: string
              format: uuid
            contractType:
              type: string
              enum: [MSA, SOW, NDA, PURCHASE, SALE, LICENSE, SERVICE, EMPLOYMENT,
FRAMEWORK]
            title:
              type: string
            reference:
              type: string
            parties:
              type: array
              items:
                type: object
                properties:

```

```
    partyId:
      type: string
      format: uuid
    role:
      type: string
      enum: [BUYER, SUPPLIER, COUNCIL]
    name:
      type: string
  effectiveDate:
    type: string
    format: date
  expiryDate:
    type: string
    format: date
  draftedBy:
    $ref: '#/components/schemas/UserReference'
  traceId:
    type: string

ContractSigned:
  summary: A contract has been signed by all parties
  payload:
    type: object
    required: [contractId, signedAt]
    properties:
      contractId:
        type: string
        format: uuid
      signedAt:
        type: string
        format: date-time
      signedBy:
        type: array
        items:
          $ref: '#/components/schemas/UserReference'
      signatureMethod:
        type: string
        enum: [ESIGN, WET_SIGN, DIGITAL]
      traceId:
        type: string

ObligationDue:
  summary: A contract obligation is due
  payload:
    type: object
    required: [obligationId, contractId, dueDate]
    properties:
      obligationId:
        type: string
        format: uuid
      contractId:
        type: string
```



```

        format: uuid
      type:
        type: string
        enum: [RENEWAL, PAYMENT, DELIVERY, SERVICE_LEVEL, REPORTING, COMPLIANCE,
INSURANCE, AUDIT, INDEMNITY, CONFIDENTIALITY]
      description:
        type: string
      dueDate:
        type: string
        format: date-time
      responsiblePartyId:
        type: string
        format: uuid
      traceId:
        type: string

```

6.4 Data Model

SQL

```

-- services/contract/src/infrastructure/schema/001_initial.sql

CREATE TABLE contracts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contract_type VARCHAR(20) NOT NULL,
  title VARCHAR(255) NOT NULL,
  reference VARCHAR(50),
  parties JSONB NOT NULL,
  effective_date DATE NOT NULL,
  expiry_date DATE,
  terms JSONB NOT NULL,
  obligations JSONB DEFAULT '[]',
  amendments JSONB DEFAULT '[]',
  documents JSONB DEFAULT '[]',
  risk_score JSONB,
  ai_analysis JSONB,
  status VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
  version INTEGER DEFAULT 1,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  tenant_id VARCHAR(50) NOT NULL
);

CREATE TABLE obligations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
  type VARCHAR(30) NOT NULL,
  description TEXT NOT NULL,
  responsible_party_id UUID NOT NULL,
  due_date TIMESTAMP WITH TIME ZONE NOT NULL,

```

```

status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
completion_evidence JSONB,
reminder_sent_at JSONB DEFAULT '[]',
escalated_at JSONB DEFAULT '[]',
ai_detected_source TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
tenant_id VARCHAR(50) NOT NULL
);

CREATE INDEX idx_contracts_status ON contracts (status);
CREATE INDEX idx_contracts_effective_date ON contracts (effective_date);
CREATE INDEX idx_contracts_expiry_date ON contracts (expiry_date);
CREATE INDEX idx_obligations_contract ON obligations (contract_id);
CREATE INDEX idx_obligations_due_date ON obligations (due_date);
CREATE INDEX idx_obligations_status ON obligations (status);

```

6.5 Vector Store Integration

TYPESCRIPT

```

// services/contract/src/infrastructure/vector/ContractEmbedding.ts

import { embeddingService } from '@opens2p/knowledge-service';

export class ContractEmbeddingService {
  /**
   * Generate embeddings for contract clauses
   */
  async embedContract(contractId: UUID): Promise<void> {
    const contract = await this.repository.findById(contractId);
    if (!contract) {
      throw new ContractError('CONTRACT_NOT_FOUND', `Contract ${contractId} not found`);
    }

    // Extract clauses for embedding
    const clauses = this.extractClauses(contract);
    const chunks = await this.prepareEmbeddingChunks(clauses);

    // Generate embeddings
    const embeddings = await embeddingService.embedBatch(chunks);

    // Store in PGVector
    await this.vectorRepository.storeEmbeddings({
      contractId,
      embeddings: chunks.map((chunk, index) => ({
        chunkId: chunk.id,
        content: chunk.content,
        embedding: embeddings[index],
        metadata: {

```

```
        contractId,
        contractType: contract.contractType,
        section: chunk.section,
        clauseType: chunk.clauseType
    }
    })))
    });
}

/**
 * Search for similar clauses across contracts
 */
async searchSimilarClauses(
    query: string,
    contractType?: ContractType,
    limit: number = 10
): Promise<SimilarClauseResult[]> {
    const queryEmbedding = await embeddingService.embed(query);
    return this.vectorRepository.search(queryEmbedding, {
        filter: contractType ? { contractType } : undefined,
        limit
    });
}

private extractClauses(contract: Contract): ExtractedClause[] {
    // Implementation: parse contract text, identify clauses
    return [];
}

private async prepareEmbeddingChunks(
    clauses: ExtractedClause[]
): Promise<EmbeddingChunk[]> {
    // Implementation: prepare chunks for embedding
    return [];
}
}
```

7. Purchasing Service

7.1 Service Overview

Domain	Procurement Operations
Owning Team	Procurement Team
Primary Responsibility	Purchase requisition and order lifecycle, catalogue management, approvals

7.2 Core Aggregates

A purchase order moves through a fixed set of states from creation to closure; rejection and cancellation are explicit exits rather than implicit dead ends.

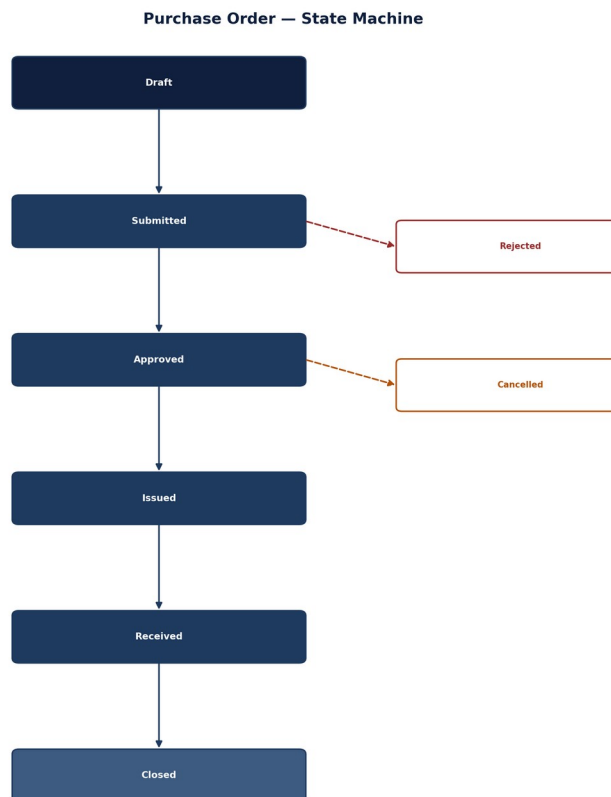


Figure 7.1 — Purchase Order State Machine

7.2.1 PurchaseRequisition (Aggregate Root)

TYPESCRIPT

```
interface PurchaseRequisition {
```

```
id: UUID;
requisitionNumber: string;
requester: UserReference;
departmentId: UUID;
costCentre: string;
budgetCode: string;
description: string;
lineItems: RequisitionLineItem[];
estimatedTotal: number;
currency: string;
requiredDate: Date;
deliveryLocation: Address;
supplierPreferences: SupplierPreference[];
attachments: DocumentReference[];
approvalStatus: ApprovalStatus;
approvalFlow: ApprovalFlow;
status: RequisitionStatus;
closedAt?: Date;
createdAt: Date;
updatedAt: Date;
version: number;
}

interface RequisitionLineItem {
  id: UUID;
  catalogueItemId?: UUID;
  description: string;
  quantity: number;
  unitOfMeasure: string;
  estimatedUnitPrice: number;
  estimatedTotal: number;
  budgetCode: string;
  costAllocation: CostAllocation[];
  specifications: Map<string, any>;
}
```

7.2.2 PurchaseOrder (Aggregate Root)

TYPESCRIPT

```
interface PurchaseOrder {
  id: UUID;
  orderNumber: string;
  requisitionId: UUID;
  supplierId: UUID;
  supplierReference?: string;
  orderDate: Date;
  deliveryDate: Date;
  lineItems: OrderLineItem[];
  totalAmount: number;
  currency: string;
```

```
    paymentTerms: PaymentTerms;
    deliveryTerms: DeliveryTerms;
    status: PurchaseOrderStatus;
    approvals: ApprovalRecord[];
    documents: DocumentReference[];
    issuedBy: UserReference;
    issuedAt: Date;
    acknowledgedAt?: Date;
    cancelledAt?: Date;
    version: number;
    createdAt: Date;
    updatedAt: Date;
  }

  interface OrderLineItem {
    id: UUID;
    requisitionLineId: UUID;
    catalogueItemId?: UUID;
    description: string;
    quantity: number;
    unitPrice: number;
    totalPrice: number;
    taxRate: number;
    taxAmount: number;
    netAmount: number;
    grossAmount: number;
    deliveryDate: Date;
    status: OrderLineStatus;
    receivedQuantity: number;
    invoicedQuantity: number;
  }

  type PurchaseOrderStatus =
    | 'DRAFT'
    | 'PENDING_APPROVAL'
    | 'APPROVED'
    | 'ISSUED'
    | 'ACKNOWLEDGED'
    | 'PARTIALLY_RECEIVED'
    | 'COMPLETED'
    | 'CANCELLED'
    | 'CLOSED';
```

7.3 Events Published

YAML

```
api/asynccapi/purchasing-events.yaml
channels:
  purchasing:
```

```
address: purchasing-events
messages:
  PRCreated:
    summary: A purchase requisition has been created
    payload:
      type: object
      required: [prId, requisitionNumber, requester, estimatedTotal]
      properties:
        prId:
          type: string
          format: uuid
        requisitionNumber:
          type: string
        requester:
          $ref: '#/components/schemas/UserReference'
        departmentId:
          type: string
          format: uuid
        estimatedTotal:
          type: number
        currency:
          type: string
        requiredDate:
          type: string
          format: date
        createdAt:
          type: string
          format: date-time
        traceId:
          type: string

  PRApproved:
    summary: A purchase requisition has been approved
    payload:
      type: object
      required: [prId, approvedBy, approvedAt]
      properties:
        prId:
          type: string
          format: uuid
        approvedBy:
          $ref: '#/components/schemas/UserReference'
        approvedAt:
          type: string
          format: date-time
        approvalLevel:
          type: integer
        traceId:
          type: string

  POIssued:
    summary: A purchase order has been issued
```

```
payload:
  type: object
  required: [poId, orderNumber, supplierId, totalAmount]
  properties:
    poId:
      type: string
      format: uuid
    orderNumber:
      type: string
    supplierId:
      type: string
      format: uuid
    requisitionId:
      type: string
      format: uuid
    orderDate:
      type: string
      format: date
    deliveryDate:
      type: string
      format: date
    totalAmount:
      type: number
    currency:
      type: string
    lineItems:
      type: array
      items:
        type: object
        properties:
          lineId:
            type: string
            format: uuid
          description:
            type: string
          quantity:
            type: number
          unitPrice:
            type: number
          totalPrice:
            type: number
    issuedBy:
      $ref: '#/components/schemas/UserReference'
    issuedAt:
      type: string
      format: date-time
    traceId:
      type: string

POAmended:
  summary: A purchase order has been amended
  payload:
```



```
type: object
required: [poId, amendmentReason]
properties:
  poId:
    type: string
    format: uuid
  amendmentReason:
    type: string
  amendedFields:
    type: array
    items:
      type: string
  amendedBy:
    $ref: '#/components/schemas/UserReference'
  amendedAt:
    type: string
    format: date-time
  traceId:
    type: string
```

7.4 Data Model

SQL

```
-- services/purchasing/src/infrastructure/schema/001_initial.sql

CREATE TABLE purchase_requisitions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  requisition_number VARCHAR(50) NOT NULL UNIQUE,
  requester JSONB NOT NULL,
  department_id UUID NOT NULL,
  cost_centre VARCHAR(50) NOT NULL,
  budget_code VARCHAR(50),
  description TEXT,
  line_items JSONB NOT NULL,
  estimated_total DECIMAL(19,4) NOT NULL,
  currency VARCHAR(3) NOT NULL DEFAULT 'USD',
  required_date DATE NOT NULL,
  delivery_location JSONB,
  supplier_preferences JSONB DEFAULT '[]',
  attachments JSONB DEFAULT '[]',
  approval_status JSONB NOT NULL,
  approval_flow JSONB NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
  closed_at TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  version INTEGER DEFAULT 1,
  tenant_id VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE purchase_orders (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  order_number VARCHAR(50) NOT NULL UNIQUE,  
  requisition_id UUID REFERENCES purchase_requisitions(id),  
  supplier_id UUID NOT NULL,  
  supplier_reference VARCHAR(50),  
  order_date DATE NOT NULL DEFAULT CURRENT_DATE,  
  delivery_date DATE NOT NULL,  
  line_items JSONB NOT NULL,  
  total_amount DECIMAL(19,4) NOT NULL,  
  currency VARCHAR(3) NOT NULL DEFAULT 'USD',  
  payment_terms JSONB NOT NULL,  
  delivery_terms JSONB NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'DRAFT',  
  approvals JSONB DEFAULT '[]',  
  documents JSONB DEFAULT '[]',  
  issued_by JSONB,  
  issued_at TIMESTAMP WITH TIME ZONE,  
  acknowledged_at TIMESTAMP WITH TIME ZONE,  
  cancelled_at TIMESTAMP WITH TIME ZONE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  version INTEGER DEFAULT 1,  
  tenant_id VARCHAR(50) NOT NULL  
);  
  
CREATE INDEX idx_pr_requester ON purchase_requisitions (requester);  
CREATE INDEX idx_pr_status ON purchase_requisitions (status);  
CREATE INDEX idx_pr_required_date ON purchase_requisitions (required_date);  
CREATE INDEX idx_po_supplier ON purchase_orders (supplier_id);  
CREATE INDEX idx_po_status ON purchase_orders (status);  
CREATE INDEX idx_po_order_date ON purchase_orders (order_date);  
CREATE INDEX idx_po_delivery_date ON purchase_orders (delivery_date);
```

7.5 Approval Workflow

TYPESCRIPT

```
// services/purchasing/src/domain/workflow/ApprovalWorkflow.ts

export class ApprovalWorkflow {
  /**
   * Determine the approval workflow based on requisition attributes
   */
  static determineWorkflow(pr: PurchaseRequisition): ApprovalFlow {
    const workflow: ApprovalFlow = {
      stages: [],
      currentStage: 0
    };

    // Basic rule: approval based on estimated total
    if (pr.estimatedTotal < 5000) {
      // Low value: manager approval only
      workflow.stages.push({
        level: 1,
        requiredRoles: ['PROCUREMENT_MANAGER'],
        autoApprove: false,
        timeoutHours: 48,
        escalationAfter: 72
      });
    } else if (pr.estimatedTotal < 50000) {
      // Medium value: manager + director
      workflow.stages.push({
        level: 1,
        requiredRoles: ['PROCUREMENT_MANAGER'],
        autoApprove: false,
        timeoutHours: 48,
        escalationAfter: 72
      });
      workflow.stages.push({
        level: 2,
        requiredRoles: ['PROCUREMENT_DIRECTOR'],
        autoApprove: false,
        timeoutHours: 24,
        escalationAfter: 48
      });
    } else {
      // High value: manager + director + CFO
      workflow.stages.push({
        level: 1,
        requiredRoles: ['PROCUREMENT_MANAGER'],
        autoApprove: false,
        timeoutHours: 48,
        escalationAfter: 72
      });
      workflow.stages.push({
        level: 2,
        requiredRoles: ['PROCUREMENT_DIRECTOR'],
        autoApprove: false,
        timeoutHours: 24,
```

```
        escalationAfter: 48
    });
    workflow.stages.push({
        level: 3,
        requiredRoles: ['CF0'],
        autoApprove: false,
        timeoutHours: 24,
        escalationAfter: 48
    });
}

// Additional checks
if (this.hasSensitiveCategory(pr)) {
    // Add compliance approval for sensitive categories
    workflow.stages.push({
        level: 4,
        requiredRoles: ['COMPLIANCE_OFFICER'],
        autoApprove: false,
        timeoutHours: 48,
        escalationAfter: 72
    });
}

return workflow;
}

private static hasSensitiveCategory(pr: PurchaseRequisition): boolean {
    const sensitiveCategories = ['IT', 'LEGAL', 'CONSULTING', 'SECURITY'];
    return pr.lineItems.some(item =>
        sensitiveCategories.some(cat => item.description.includes(cat))
    );
}
}
```

8. Invoice Service

8.1 Service Overview

Domain	Invoice Management
Owning Team	AP Team
Primary Responsibility	Invoice capture, three-way matching, exception management, early payment programs

8.2 Core Aggregates

An invoice moves from receipt through three-way matching to payment; a matching exception or a post-approval dispute are modelled as explicit states rather than error codes alone.

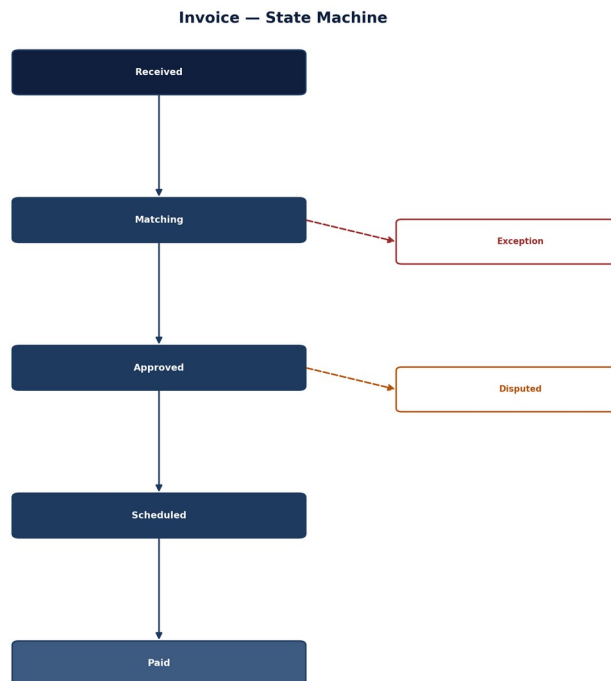


Figure 8.1 — Invoice State Machine

8.2.1 Invoice (Aggregate Root)

TYPESCRIPT

```
interface Invoice {  
  id: UUID;  
  invoiceNumber: string;  
  supplierId: UUID;  
  purchaseOrderId: UUID;  
}
```

```
    invoiceDate: Date;
    dueDate: Date;
    lineItems: InvoiceLineItem[];
    subtotal: number;
    taxAmount: number;
    totalAmount: number;
    currency: string;
    status: InvoiceStatus;
    matchingResult: ThreeWayMatchResult;
    exceptionFlags: ExceptionFlag[];
    payments: PaymentAllocation[];
    documents: DocumentReference[];
    createdAt: Date;
    updatedAt: Date;
    version: number;
}

interface InvoiceLineItem {
    id: UUID;
    poLineId: UUID;
    description: string;
    quantity: number;
    unitPrice: number;
    totalPrice: number;
    taxRate: number;
    taxAmount: number;
    matchStatus: MatchStatus;
}

type InvoiceStatus =
    | 'RECEIVED'
    | 'PROCESSING'
    | 'MATCHING'
    | 'MATCHED'
    | 'EXCEPTION'
    | 'APPROVED'
    | 'REJECTED'
    | 'PAID'
    | 'CANCELLED';
```

8.2.2 ThreeWayMatch

TYPESCRIPT

```
interface ThreeWayMatchResult {
    status: MatchResultStatus;
    poMatch: MatchDetail;
    receiptMatch: MatchDetail;
    overallMatch: 'COMPLETE' | 'PARTIAL' | 'NONE';
    discrepancies: Discrepancy[];
    matchDate: Date;
```

```
  matchedBy: 'AUTO' | 'MANUAL';
  confidence: number; // 0-1 for AI-assisted matching
}

interface MatchDetail {
  status: 'MATCH' | 'PARTIAL' | 'MISMATCH';
  toleranceThreshold: number;
  variance: number;
  lineMatches: LineMatch[];
}

interface Discrepancy {
  type: 'QUANTITY' | 'PRICE' | 'TAX' | 'ITEM' | 'SUPPLIER' | 'DATE';
  severity: 'LOW' | 'MEDIUM' | 'HIGH';
  description: string;
  expectedValue: number | string;
  actualValue: number | string;
  resolution?: string;
}
```

8.3 Events Published

YAML

```
api/asynccapi/invoice-events.yaml
channels:
  invoice:
    address: invoice-events
    messages:
      InvoiceReceived:
        summary: A new invoice has been received
        payload:
          type: object
          required: [invoiceId, invoiceNumber, supplierId, totalAmount]
          properties:
            invoiceId:
              type: string
              format: uuid
            invoiceNumber:
              type: string
            supplierId:
              type: string
              format: uuid
            purchaseOrderId:
              type: string
              format: uuid
            invoiceDate:
              type: string
              format: date
            dueDate:
              type: string
              format: date
            totalAmount:
              type: number
            currency:
              type: string
            receivedAt:
              type: string
              format: date-time
            source:
              type: string
              enum: [EMAIL, EDI, OCR, API, PORTAL]
            traceId:
              type: string

      MatchFailed:
        summary: A three-way match has failed
        payload:
          type: object
          required: [invoiceId, discrepancies]
          properties:
            invoiceId:
              type: string
              format: uuid
            purchaseOrderId:
              type: string
```



```
    format: uuid
  discrepancies:
    type: array
  items:
    type: object
  properties:
    type:
      type: string
    severity:
      type: string
    description:
      type: string
    expectedValue:
      type: string
    actualValue:
      type: string
  exceptionRaisedAt:
    type: string
    format: date-time
  traceId:
    type: string

ExceptionRaised:
  summary: An exception has been raised on an invoice
  payload:
    type: object
    required: [invoiceId, exceptionType]
  properties:
    invoiceId:
      type: string
      format: uuid
    exceptionType:
      type: string
      enum: [MATCH_FAILURE, MISSING_PO, DUPLICATE, APPROVAL_REQUIRED,
PRICE_DIFFERENCE, QUANTITY_DIFFERENCE, TAX_MISMATCH, SUPPLIER_MISMATCH]
    description:
      type: string
    severity:
      type: string
      enum: [LOW, MEDIUM, HIGH]
    escalatedTo:
      type: string
      format: uuid
    traceId:
      type: string
```

8.4 Three-Way Matching Implementation

TYPESCRIPT

```
// services/invoicing/src/domain/rules/ThreeWayMatchingRules.ts

export class ThreeWayMatchingRules {
  /**
   * Execute three-way match between invoice, PO, and goods receipt
   */
  static executeMatch(
    invoice: Invoice,
    po: PurchaseOrder,
    receipt: GoodsReceipt
  ): ThreeWayMatchResult {
    const lineMatches: LineMatch[] = [];
    const discrepancies: Discrepancy[] = [];
    let overallMatch: 'COMPLETE' | 'PARTIAL' | 'NONE' = 'COMPLETE';

    for (const invoiceLine of invoice.lineItems) {
      const poLine = po.lineItems.find(l => l.id === invoiceLine.poLineId);
      const receiptLine = receipt.lineItems.find(l => l.poLineId ===
invoiceLine.poLineId);

      const match = this.matchLine(invoiceLine, poLine, receiptLine);
      lineMatches.push(match);

      if (!match.isMatch) {
        overallMatch = 'PARTIAL';
        discrepancies.push(...match.discrepancies);
      }
    }

    // Determine overall match result
    const poMatch = this.calculatePoMatch(lineMatches);
    const receiptMatch = this.calculateReceiptMatch(lineMatches);

    const result: ThreeWayMatchResult = {
      status: this.determineResultStatus(discrepancies),
      poMatch,
      receiptMatch,
      overallMatch: this.determineOverallMatch(lineMatches),
      discrepancies,
      matchDate: new Date(),
      matchedBy: 'AUTO',
      confidence: this.calculateConfidence(lineMatches)
    };

    return result;
  }

  private static matchLine(
    invoiceLine: InvoiceLineItem,
    poLine: OrderLineItem,
    receiptLine: ReceiptLineItem
  ): LineMatch {
```

```
const discrepancies: Discrepancy[] = [];

// Check quantity
const qtyTolerance = 0.05; // 5% tolerance
const qtyVariance = Math.abs(invoiceLine.quantity - poLine.quantity) /
poLine.quantity;
const qtyMatch = qtyVariance <= qtyTolerance;

if (!qtyMatch) {
  discrepancies.push({
    type: 'QUANTITY',
    severity: 'HIGH',
    description: `Invoice quantity ${invoiceLine.quantity} does not match PO quantity
${poLine.quantity}`,
    expectedValue: poLine.quantity,
    actualValue: invoiceLine.quantity
  });
}

// Check price
const priceTolerance = 0.02; // 2% tolerance
const priceVariance = Math.abs(invoiceLine.unitPrice - poLine.unitPrice) /
poLine.unitPrice;
const priceMatch = priceVariance <= priceTolerance;

if (!priceMatch) {
  discrepancies.push({
    type: 'PRICE',
    severity: 'HIGH',
    description: `Invoice price ${invoiceLine.unitPrice} does not match PO price $
${poLine.unitPrice}`,
    expectedValue: poLine.unitPrice,
    actualValue: invoiceLine.unitPrice
  });
}

// Check receipt if available
let receiptMatch = false;
if (receiptLine) {
  const receiptVariance = Math.abs(invoiceLine.quantity -
receiptLine.receivedQuantity) / receiptLine.receivedQuantity;
  receiptMatch = receiptVariance <= qtyTolerance;
  if (!receiptMatch) {
    discrepancies.push({
      type: 'QUANTITY',
      severity: 'MEDIUM',
      description: `Invoice quantity ${invoiceLine.quantity} does not match receipt
quantity ${receiptLine.receivedQuantity}`,
      expectedValue: receiptLine.receivedQuantity,
      actualValue: invoiceLine.quantity
    });
  }
}
```

```
}

return {
  invoiceLineId: invoiceLine.id,
  poLineId: poLine.id,
  receiptLineId: receiptLine?.id,
  isMatch: qtyMatch && priceMatch && (receiptLine ? receiptMatch : true),
  qtyMatch,
  priceMatch,
  receiptMatch: receiptLine ? receiptMatch : false,
  discrepancies
};
}
}
```

9. Workflow Service

9.1 Service Overview

Domain	Workflow Orchestration
Owning Team	Platform Team
Primary Responsibility	Workflow definition management, execution orchestration, state persistence

9.2 Core Components

Every workflow instance — regardless of which business process it's driving — follows this same state machine, which is what lets the Workflow Orchestrator manage approvals generically across services.

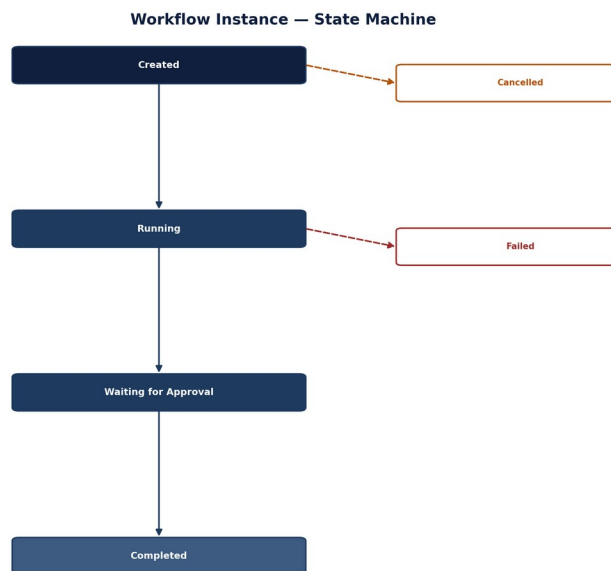


Figure 9.1 — Workflow Instance State Machine

9.2.1 Workflow Definition

TYPESCRIPT

```
interface WorkflowDefinition {
  id: UUID;
  name: string;
  version: string;
  description: string;
  graph: WorkflowGraph;
  metadata: WorkflowMetadata;
  status: 'DRAFT' | 'ACTIVE' | 'DEPRECATED' | 'ARCHIVED';
}
```

```
    createdAt: Date;
    updatedAt: Date;
  }

  interface WorkflowGraph {
    nodes: WorkflowNode[];
    edges: WorkflowEdge[];
  }

  interface WorkflowNode {
    id: string;
    type: 'START' | 'PROCESS' | 'DECISION' | 'GATE' | 'PARALLEL' | 'END';
    label: string;
    config: NodeConfig;
    position?: { x: number; y: number };
  }

  interface WorkflowEdge {
    id: string;
    source: string;
    target: string;
    condition?: EdgeCondition;
    label?: string;
  }

  interface NodeConfig {
    // Based on node type
    process?: {
      handler: string; // Service/function to call
      timeout: number;
      retry: {
        maxAttempts: number;
        backoff: number;
      };
    };
    decision?: {
      expression: string;
      branches: {
        condition: string;
        target: string;
      }[];
    };
    gate?: {
      type: 'HUMAN' | 'AUTO';
      threshold?: number;
      approvers?: string[];
    };
    parallel?: {
      branches: string[]; // Node IDs to execute in parallel
    };
  }
}
```

9.2.2 Workflow Execution

TYPESCRIPT

```
interface WorkflowExecution {
  id: UUID;
  workflowId: UUID;
  workflowVersion: string;
  status: ExecutionStatus;
  context: ExecutionContext;
  currentNodes: string[];
  history: ExecutionStep[];
  state: Record<string, any>;
  startedAt: Date;
  completedAt?: Date;
  error?: WorkflowError;
}

interface ExecutionContext {
  tenantId: string;
  userId: UUID;
  correlationId: string;
  inputs: Record<string, any>;
  variables: Record<string, any>;
}
```

9.3 LangGraph Integration

TYPESCRIPT

```
// services/workflow/src/application/langgraph/WorkflowEngine.ts

import { StateGraph, MemorySaver, State } from '@langchain/langgraph';
import { ChatOpenAI } from '@langchain/openai';

export class WorkflowEngine {
  private graphRegistry: Map<string, StateGraph<any>> = new Map();
  private memorySaver = new MemorySaver();

  /**
   * Register a workflow definition as a LangGraph
   */
  async registerWorkflow(definition: WorkflowDefinition): Promise<void> {
    const graph = new StateGraph({
      channels: {
        // Define state schema
      }
    });

    // Add nodes
    for (const node of definition.graph.nodes) {
```

```

    switch (node.type) {
      case 'PROCESS':
        graph.addNode(node.id, this.createProcessNode(node));
        break;
      case 'DECISION':
        graph.addNode(node.id, this.createDecisionNode(node));
        break;
      case 'GATE':
        graph.addNode(node.id, this.createGateNode(node));
        break;
      case 'PARALLEL':
        graph.addNode(node.id, this.createParallelNode(node));
        break;
    }
  }

  // Add edges
  for (const edge of definition.graph.edges) {
    if (edge.condition) {
      graph.addConditionalEdges(edge.source, (state: State) => {
        return this.evaluateCondition(edge.condition, state);
      });
    } else {
      graph.addEdge(edge.source, edge.target);
    }
  }

  graph.setEntryPoint(definition.graph.nodes.find(n => n.type === 'START')?.id || '');
  graph.setFinishPoint(definition.graph.nodes.find(n => n.type === 'END')?.id || '');

  this.graphRegistry.set(`${definition.id}:${definition.version}`, graph.compile({
    checkpointer: this.memorySaver
  }));
}

/**
 * Execute a workflow
 */
async executeWorkflow(
  workflowId: UUID,
  version: string,
  inputs: Record<string, any>
): Promise<WorkflowExecution> {
  const key = `${workflowId}:${version}`;
  const graph = this.graphRegistry.get(key);

  if (!graph) {
    throw new WorkflowError('WORKFLOW_NOT_FOUND', `Workflow ${key} not found`);
  }

  const execution: WorkflowExecution = {
    id: uuidv4(),

```



```
        workflowId,
        workflowVersion: version,
        status: 'RUNNING',
        context: {
            tenantId: this.getCurrentTenant(),
            userId: this.getCurrentUser(),
            correlationId: this.getCorrelationId(),
            inputs,
            variables: {}
        },
        currentNodes: [],
        history: [],
        state: {},
        startedAt: new Date()
    };

    try {
        const result = await graph.invoke(
            {
                ...inputs,
                __execution: execution
            },
            {
                configurable: {
                    thread_id: execution.id
                }
            }
        );

        execution.status = 'COMPLETED';
        execution.completedAt = new Date();
        execution.state = result;

        return execution;
    } catch (error) {
        execution.status = 'FAILED';
        execution.error = {
            code: 'EXECUTION_ERROR',
            message: error.message,
            stack: error.stack
        };
        throw error;
    }
}

/**
 * Resume a suspended workflow (HITL gate)
 */
async resumeWorkflow(
    executionId: UUID,
    decision: GateDecision
): Promise<WorkflowExecution> {
```

```
    // Resume the workflow with the gate decision
    // Implementation details
  }
}
```

10. AI Orchestrator

10.1 Service Overview

Domain	AI Agent Orchestration
Owning Team	AI Platform Team
Primary Responsibility	Agent lifecycle management, confidence scoring, HITL gating, evaluation

10.2 Core Components

The state view of the same pipeline shown in Figures 10.1 and 10.2: an agent run is either completed automatically, routed to human approval, or marked failed — there is no silent partial-completion state.

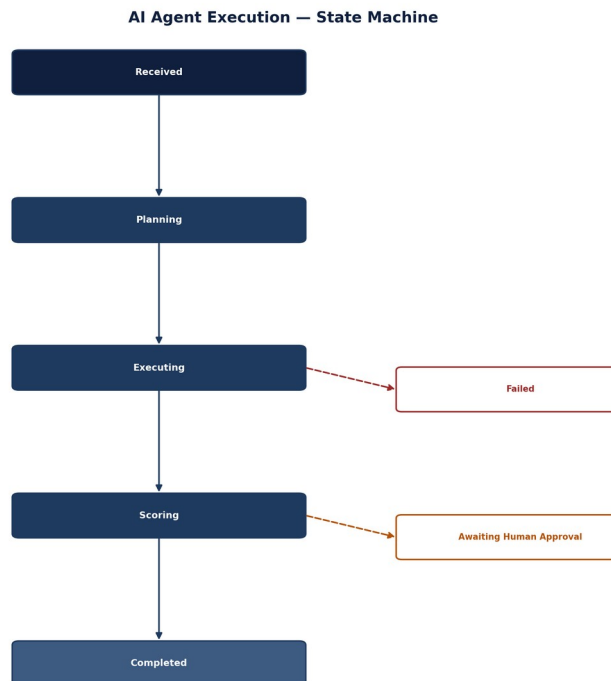


Figure 10.3 — AI Agent Execution State Machine

10.2.1 Agent Configuration

TYPESCRIPT

```
interface AgentConfig {  
  id: UUID;  
  name: string;  
  type: AgentType;  
  description: string;  
}
```

```

    systemPrompt: PromptReference;
    tools: ToolDefinition[];
    confidenceThreshold: number; // 0-1
    memoryConfig: MemoryConfig;
    modelConfig: ModelConfig;
    evaluationConfig: EvaluationConfig;
    status: 'DRAFT' | 'ACTIVE' | 'DEPRECATED';
    version: string;
    createdAt: Date;
    updatedAt: Date;
  }

  type AgentType =
    | 'SOURCING_ASSISTANT'
    | 'CONTRACT_ANALYZER'
    | 'SUPPLIER_DISCOVERY'
    | 'PO_GENERATOR'
    | 'INVOICE_MATCHER'
    | 'RISK_ANALYZER'
    | 'SPEND_ANALYZER';

  interface ToolDefinition {
    id: string;
    name: string;
    description: string;
    parameters: JSONSchema;
    handler: string; // Service:method
    allowedScopes: string[];
  }

```

10.2.2 Agent Invocation

TYPESCRIPT

```

interface AgentInvocation {
  id: UUID;
  agentId: UUID;
  input: Record<string, any>;
  confidence: number;
  status: 'PENDING' | 'PROCESSING' | 'COMPLETED' | 'GATE_WAITING' | 'FAILED';
  result?: AgentResult;
  traceId: string;
  startedAt: Date;
  completedAt?: Date;
}

interface AgentResult {
  output: Record<string, any>;
  confidence: number;
  citations: Citation[];
  toolCalls: ToolCall[];
}

```

```
    evaluation: EvaluationResult;
    metadata: Record<string, any>;
  }

  interface Citation {
    sourceId: string;
    sourceType: 'CONTRACT' | 'SUPPLIER' | 'POLICY' | 'REGULATION' | 'PO' | 'INVOICE';
    reference: string;
    excerpt: string;
    confidence: number;
  }
```

10.3 Agent Execution Pipeline

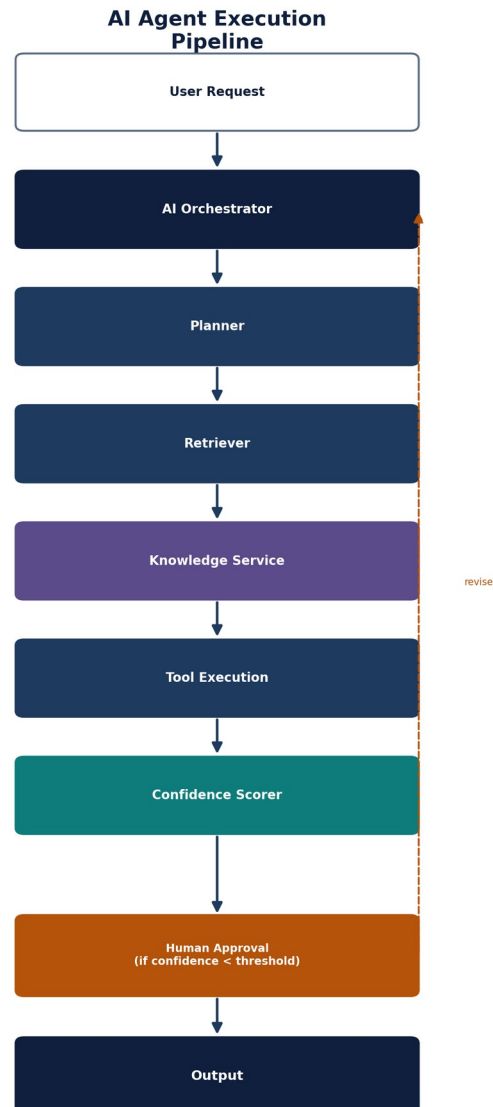


Figure 10.1 — AI Agent Execution Pipeline

TYPESCRIPT

```
// services/ai-orchestrator/src/application/agent/AgentExecutionPipeline.ts

export class AgentExecutionPipeline {
  private agentRegistry: AgentRegistry;
  private promptService: PromptService;
  private confidenceScorer: ConfidenceScorer;
  private gateService: GateService;
```

```
private auditService: AuditService;

/**
 * Execute an agent invocation through the 9-stage pipeline
 */
async execute(invocation: AgentInvocation): Promise<AgentResult> {
  // Stage 1: Input Trigger (already handled)
  this.auditService.record('AGENT_STARTED', invocation);

  // Stage 2: Planner
  const plan = await this.plan(invocation);

  // Stage 3: Retriever
  const context = await this.retrieve(plan);

  // Stage 4: Reasoner
  const reasoning = await this.reason(plan, context);

  // Stage 5: Tool Calls
  const toolResults = await this.executeTools(reasoning);

  // Stage 6: Validator
  const validation = await this.validate(toolResults);

  // Stage 7: Confidence Scorer
  const confidence = await this.scoreConfidence(validation);

  // Stage 8: Approval Gate
  const gateResult = await this.applyGate(confidence, invocation);

  // Stage 9: Output & Audit
  const result = await this.produceOutput(gateResult, invocation);

  this.auditService.record('AGENT_COMPLETED', {
    invocationId: invocation.id,
    result
  });

  return result;
}

private async plan(invocation: AgentInvocation): Promise<Plan> {
  const agent = await this.agentRegistry.findById(invocation.agentId);
  const prompt = await this.promptService.getSystemPrompt(agent.systemPrompt);

  // Decompose task and select tools
  const plan = await this.llmService.invoke({
    prompt: prompt,
    input: invocation.input,
    tools: agent.tools
  });
}
```

```
    return plan;
  }

  private async retrieve(plan: Plan): Promise<Context> {
    // Vector search, graph traversal, document fetch
    const context = await this.knowledgeService.query({
      query: plan.searchQuery,
      filters: plan.filters,
      maxResults: 20
    });

    return context;
  }

  private async scoreConfidence(validation: ValidationResult): Promise<ConfidenceScore> {
    const score = await this.confidenceScorer.calculate({
      evidenceQuality: validation.evidenceQuality,
      modelUncertainty: validation.modelUncertainty,
      policyCompliance: validation.policyCompliance,
      riskSignals: validation.riskSignals
    });

    return score;
  }

  private async applyGate(
    confidence: ConfidenceScore,
    invocation: AgentInvocation
  ): Promise<GateResult> {
    const agent = await this.agentRegistry.findById(invocation.agentId);

    if (confidence.score >= agent.confidenceThreshold) {
      return { approved: true, confidence };
    }

    // Below threshold - escalate to human
    const gateRequest = await this.gateService.createGateRequest({
      invocationId: invocation.id,
      agentId: invocation.agentId,
      confidence: confidence.score,
      output: invocation.result,
      reason: confidence.reasons
    });

    return { approved: false, gateRequest };
  }
}
```

10.4 Confidence Scorer

TYPESCRIPT


```
// services/ai-orchestrator/src/domain/confidence/ConfidenceScorer.ts

export class ConfidenceScorer {
  /**
   * Calculate aggregate confidence score
   */
  async calculate(
    factors: ConfidenceFactors
  ): Promise<ConfidenceScore> {
    const scores: ScoreComponent[] = [];

    // Evidence quality
    const evidenceScore = this.scoreEvidence(factors.evidenceQuality);
    scores.push({
      component: 'EVIDENCE',
      weight: 0.30,
      score: evidenceScore,
      reasoning: this.getEvidenceReasoning(factors.evidenceQuality)
    });

    // Model uncertainty
    const modelScore = this.scoreModelUncertainty(factors.modelUncertainty);
    scores.push({
      component: 'MODEL',
      weight: 0.25,
      score: modelScore,
      reasoning: this.getModelReasoning(factors.modelUncertainty)
    });

    // Policy compliance
    const policyScore = this.scorePolicyCompliance(factors.policyCompliance);
    scores.push({
      component: 'POLICY',
      weight: 0.25,
      score: policyScore,
      reasoning: this.getPolicyReasoning(factors.policyCompliance)
    });

    // Risk signals
    const riskScore = this.scoreRiskSignals(factors.riskSignals);
    scores.push({
      component: 'RISK',
      weight: 0.20,
      score: riskScore,
      reasoning: this.getRiskReasoning(factors.riskSignals)
    });

    // Aggregate
    const aggregate = scores.reduce((sum, s) => sum + (s.score * s.weight), 0);

    return {
      score: aggregate,
    };
  }
}
```

```

        components: scores,
        threshold: this.determineThreshold(aggregate),
        reasons: this.generateReasons(scores)
    };
}

private scoreEvidence(evidence: EvidenceQuality): number {
    // Count citations, assess relevance
    const citationCount = evidence.citations.length;
    const relevanceScore = evidence.relevanceScore;
    const completeness = evidence.completeness;

    return (citationCount * 0.3 + relevanceScore * 0.4 + completeness * 0.3);
}

private scoreModelUncertainty(uncertainty: ModelUncertainty): number {
    // Convert uncertainty to confidence
    return 1 - uncertainty.entropy;
}

private scorePolicyCompliance(compliance: PolicyCompliance): number {
    // Check policy adherence
    const violations = compliance.violations.length;
    return Math.max(0, 1 - (violations * 0.2));
}

private scoreRiskSignals(signals: RiskSignal[]): number {
    // Assess risk signals
    const highRisk = signals.filter(s => s.severity === 'HIGH').length;
    const mediumRisk = signals.filter(s => s.severity === 'MEDIUM').length;

    return Math.max(0, 1 - (highRisk * 0.5 + mediumRisk * 0.2));
}
}

```

10.5 AI Architecture Overview

Figure 10.1 shows the pipeline as a linear sequence; the diagram below shows the same components arranged by relationship rather than by step order, including the knowledge stores and audit trail. This is the view worth putting in front of a customer or investor to explain what makes OpenS2P's AI layer different from a chatbot bolted onto a procurement UI.

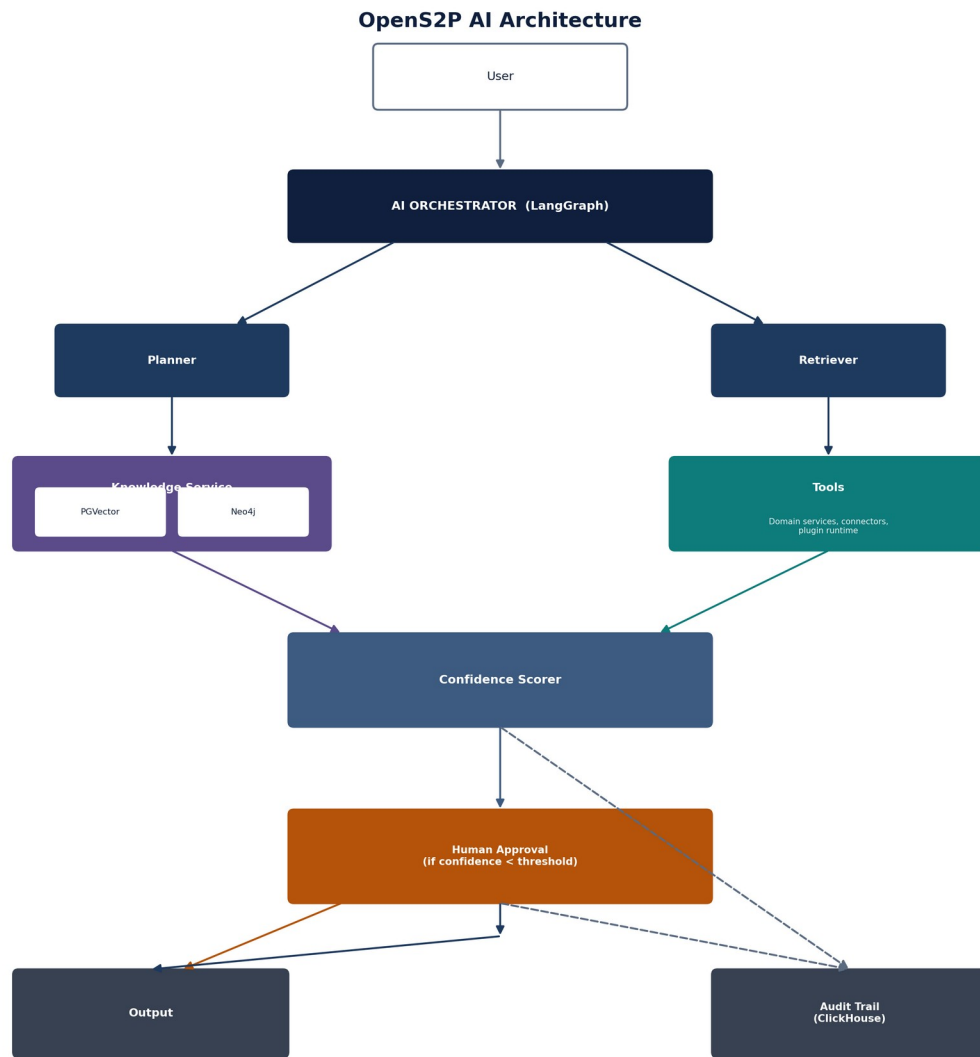


Figure 10.2 — OpenS2P AI Architecture

11. Knowledge Service

11.1 Service Overview

Domain	Knowledge Management
Owning Team	AI Platform Team
Primary Responsibility	Document indexing, semantic search, organisational memory

11.2 Core Components

11.2.1 Document Indexing

TYPESCRIPT

```
interface Document {
  id: UUID;
  source: DocumentSource;
  type: DocumentType;
  title: string;
  content: string;
  metadata: DocumentMetadata;
  embeddings?: Embedding[];
  status: 'PENDING' | 'INDEXED' | 'FAILED';
  createdAt: Date;
  updatedAt: Date;
}

type DocumentSource =
  | 'CONTRACT'
  | 'INVOICE'
  | 'PO'
  | 'SUPPLIER_PROFILE'
  | 'POLICY'
  | 'REGULATION'
  | 'EMAIL'
  | 'NOTE'
  | 'UPLOADED';

interface DocumentMetadata {
  tenantId: string;
  ownerId: UUID;
  departmentId: UUID;
  tags: string[];
  classification: DataClassification;
  sourceId: string; // Original entity ID
  sourceType: string;
  version: number;
```

```
    processedAt: Date;
  }

  interface Embedding {
    id: UUID;
    chunkId: UUID;
    embedding: number[];
    metadata: Record<string, any>;
  }
```

11.2.2 Search

TYPESCRIPT

```
interface SearchRequest {
  query: string;
  filters?: SearchFilter[];
  limit?: number;
  minScore?: number;
  searchType?: 'SEMANTIC' | 'KEYWORD' | 'HYBRID';
}

interface SearchResult {
  documents: SearchHit[];
  total: number;
  took: number;
}

interface SearchHit {
  documentId: UUID;
  score: number;
  content: string;
  metadata: DocumentMetadata;
  highlights: string[];
}
```

11.3 Implementation

TYPESCRIPT

```
// services/knowledge/src/application/search/SearchService.ts

import { EmbeddingService } from './EmbeddingService';
import { VectorStore } from '../infrastructure/vector/VectorStore';

export class SearchService {
  private embeddingService: EmbeddingService;
  private vectorStore: VectorStore;
  private elasticsearch: ElasticsearchClient;

  /**
   * Execute hybrid search
   */
  async search(request: SearchRequest): Promise<SearchResult> {
    const [semanticResults, keywordResults] = await Promise.all([
      this.semanticSearch(request),
      this.keywordSearch(request)
    ]);

    // Merge and rank results
    return this.mergeResults(semanticResults, keywordResults, request);
  }

  private async semanticSearch(request: SearchRequest): Promise<SearchHit[]> {
    const embedding = await this.embeddingService.embed(request.query);
    const results = await this.vectorStore.search(embedding, {
      filter: request.filters,
      limit: request.limit || 20
    });

    return results.map(r => ({
      documentId: r.id,
      score: r.score,
      content: r.content,
      metadata: r.metadata,
      highlights: []
    }));
  }

  private async keywordSearch(request: SearchRequest): Promise<SearchHit[]> {
    const results = await this.elasticsearch.search({
      index: 'documents',
      body: {
        query: {
          bool: {
            must: [
              { match: { content: request.query } }
            ],
            filter: this.buildFilters(request.filters)
          }
        },
        size: request.limit || 20
      }
    });
  }
}
```

```
    }
  });

  return results.hits.hits.map(hit => ({
    documentId: hit._id,
    score: hit._score,
    content: hit._source.content,
    metadata: hit._source.metadata,
    highlights: hit.highlight?.content || []
  }));
}

private mergeResults(
  semantic: SearchHit[],
  keyword: SearchHit[],
  request: SearchRequest
): SearchResult {
  // Reciprocal Rank Fusion (RRF)
  const scores: Map<string, number> = new Map();
  const documents: Map<string, SearchHit> = new Map();

  for (const [rank, result] of semantic.entries()) {
    const id = result.documentId;
    scores.set(id, (scores.get(id) || 0) + 1 / (rank + 60));
    documents.set(id, result);
  }

  for (const [rank, result] of keyword.entries()) {
    const id = result.documentId;
    scores.set(id, (scores.get(id) || 0) + 1 / (rank + 60));
    if (!documents.has(id)) {
      documents.set(id, result);
    }
  }

  // Sort by combined score
  const combined = Array.from(scores.entries())
    .sort((a, b) => b[1] - a[1])
    .map(([id, score]) => ({
      ...documents.get(id)!,
      score
    }));

  return {
    documents: combined.slice(0, request.limit || 20),
    total: combined.length,
    took: 0 // Calculate actual time
  };
}
```

11.4 Embedding Pipeline

TYPESCRIPT

```
// services/knowledge/src/application/indexing/EmbeddingPipeline.ts

export class EmbeddingPipeline {
  private embeddingService: EmbeddingService;
  private vectorStore: VectorStore;
  private chunkingService: ChunkingService;

  /**
   * Process a document for embedding
   */
  async indexDocument(document: Document): Promise<void> {
    // 1. Chunk the document
    const chunks = await this.chunkingService.chunk(document.content, {
      chunkSize: 512,
      overlap: 50,
      minSize: 100
    });

    // 2. Generate embeddings
    const embeddings = await this.embeddingService.embedBatch(
      chunks.map(c => c.content)
    );

    // 3. Store in vector store
    await this.vectorStore.upsert(
      chunks.map((chunk, index) => ({
        id: chunk.id,
        content: chunk.content,
        embedding: embeddings[index],
        metadata: {
          ...document.metadata,
          chunkIndex: index,
          totalChunks: chunks.length
        }
      })))
    );

    // 4. Update document status
    await this.updateDocumentStatus(document.id, 'INDEXED');
  }

  /**
   * Refresh embeddings for changed documents
   */
  async refreshEmbeddings(documentId: UUID): Promise<void> {
    const document = await this.getDocument(documentId);
    if (!document) {
      throw new Error(`Document ${documentId} not found`);
    }
  }
}
```



```
// Delete old embeddings
await this.vectorStore.delete(documentId);

// Re-index
await this.indexDocument(document);
}
}
```

12. Plugin Runtime

12.1 Service Overview

Domain	Plugin Execution
Owning Team	Platform Team
Primary Responsibility	Plugin sandbox execution, capability enforcement, resource isolation

12.2 Core Components

12.2.1 Plugin Manifest

TYPESCRIPT

```
interface PluginManifest {
  id: UUID;
  name: string;
  version: string; // SemVer
  description: string;
  publisher: PublisherInfo;
  capabilities: CapabilityDeclaration[];
  requirements: PluginRequirement[];
  permissions: PermissionDeclaration[];
  signature: PluginSignature;
  platformCompatibility: string; // SemVer range
  createdAt: Date;
  updatedAt: Date;
}

interface CapabilityDeclaration {
  type: 'TOOL' | 'WIDGET' | 'WORKFLOW_STEP' | 'APPROVAL_LOGIC' | 'INTEGRATION';
  name: string;
  description: string;
  entrypoint: string;
  parameters: JSONSchema;
}

interface PermissionDeclaration {
  resource: string;
  actions: ('READ' | 'WRITE' | 'EXECUTE')[];
  scope: 'TENANT' | 'USER' | 'SYSTEM';
}

interface PluginRequirement {
  type: 'STORAGE' | 'NETWORK' | 'MEMORY' | 'CPU';
  value: number | string;
}
```

12.2.2 Plugin Execution

TYPESCRIPT

```
interface PluginExecutionRequest {  
  pluginId: UUID;  
  capability: string;  
  input: Record<string, any>;  
  context: ExecutionContext;  
  timeout: number; // milliseconds  
}  
  
interface PluginExecutionResult {  
  success: boolean;  
  output: Record<string, any>;  
  executionTime: number;  
  memoryUsed: number;  
  errors?: PluginError[];  
}
```

12.3 Wasm Sandbox Implementation

TYPESCRIPT

```
// services/plugin-runtime/src/infrastructure/sandbox/WasmSandbox.ts

import { WasmRuntime, WasmInstance } from '@opens2p/wasm-runtime';

export class WasmSandbox {
  private runtime: WasmRuntime;
  private memoryLimit: number = 256 * 1024 * 1024; // 256MB
  private cpuLimit: number = 10000; // 10 seconds
  private allowedEgress: Set<string> = new Set();

  /**
   * Execute a plugin in the Wasm sandbox
   */
  async execute(request: PluginExecutionRequest): Promise<PluginExecutionResult> {
    const startTime = Date.now();

    // 1. Load plugin
    const plugin = await this.loadPlugin(request.pluginId);
    const instance = await this.createInstance(plugin, request);

    // 2. Validate permissions
    await this.validatePermissions(plugin, request);

    // 3. Execute with timeout
    const result = await this.runWithTimeout(instance, request, this.cpuLimit);

    const executionTime = Date.now() - startTime;

    return {
      success: result.success,
      output: result.output,
      executionTime,
      memoryUsed: instance.getMemoryUsage(),
      errors: result.errors
    };
  }

  private async createInstance(
    plugin: Plugin,
    request: PluginExecutionRequest
  ): Promise<WasmInstance> {
    // Configure Wasm instance with permissions
    return this.runtime.createInstance({
      module: plugin.wasmModule,
      memory: {
        maxPages: Math.ceil(this.memoryLimit / 65536)
      },
      imports: {
        // Provide limited host functions
        'host': {
          'log': (message: string) => {
            // Cap log size

```

```
    },
    'http_request': (url: string, method: string, body: Uint8Array) => {
        // Validate URL against allowed egress
        if (!this.isAllowedEgress(url)) {
            throw new Error(`Egress to ${url} not allowed`);
        }
        // Execute HTTP request with timeout
        return this.executeHttpRequest(url, method, body);
    }
}
});
}

private async runWithTimeout(
    instance: WasmInstance,
    request: PluginExecutionRequest,
    timeoutMs: number
): Promise<PluginResult> {
    return new Promise((resolve, reject) => {
        const timeout = setTimeout(() => {
            instance.terminate();
            reject(new Error('Plugin execution timed out'));
        }, timeoutMs);

        try {
            const result = instance.execute(request.capability, request.input);
            clearTimeout(timeout);
            resolve(result);
        } catch (error) {
            clearTimeout(timeout);
            reject(error);
        }
    });
}

private isAllowedEgress(url: string): boolean {
    // Check against declared egress URLs
    for (const allowed of this.allowedEgress) {
        if (url.startsWith(allowed)) {
            return true;
        }
    }
    return false;
}
}
```

13. Connector Runtime

13.1 Service Overview

Domain	Integration
Owning Team	Platform Team
Primary Responsibility	External system integration, adapter management, data transformation

13.2 Core Components

13.2.1 Connector Configuration

TYPESCRIPT

```
interface ConnectorConfig {
  id: UUID;
  name: string;
  type: ConnectorType;
  version: string;
  description: string;
  protocol: 'HTTP' | 'SOAP' | 'JDBC' | 'FILE' | 'MQ' | 'EMAIL';
  config: Record<string, any>;
  credentials: CredentialReference;
  rateLimit: RateLimitConfig;
  retryPolicy: RetryPolicy;
  mapping: DataMapping[];
  status: 'DRAFT' | 'ACTIVE' | 'ERROR' | 'DISABLED';
  createdAt: Date;
  updatedAt: Date;
}

type ConnectorType =
  | 'ERP' // SAP, Oracle, Microsoft
  | 'CRM' // Salesforce, HubSpot
  | 'SUPPLIER' // Supplier portals
  | 'PAYMENT' // Payment rails
  | 'BANK' // Banking systems
  | 'REGULATORY' // Sanctions, AML
  | 'CREDIT' // Credit bureaus
  | 'DOCUMENT' // Document management
  | 'WORKFLOW' // External workflow engines;

interface DataMapping {
  sourceField: string;
  targetField: string;
  transformation: 'COPY' | 'MAP' | 'CONCAT' | 'SPLIT' | 'FORMAT' | 'CUSTOM';
  defaultValue?: any;
```

```
    required: boolean;  
  }
```

13.2.2 Connector Execution

TYPESCRIPT

```
interface ConnectorRequest {  
  connectorId: UUID;  
  operation: 'READ' | 'WRITE' | 'EXECUTE' | 'POLL' | 'SUBSCRIBE';  
  payload: Record<string, any>;  
  options: RequestOptions;  
}  
  
interface ConnectorResponse {  
  success: boolean;  
  data: any;  
  statusCode: number;  
  headers: Record<string, string>;  
  duration: number;  
  errors?: ConnectorError[];  
}
```

13.3 Implementation

TYPESCRIPT

```
// services/connector-runtime/src/application/ConnectorService.ts  
  
export class ConnectorService {  
  private connectorRegistry: ConnectorRegistry;  
  private credentialService: CredentialService;  
  private rateLimiter: RateLimiter;  
  private retryHandler: RetryHandler;  
  
  /**  
   * Execute a connector request  
   */  
  async execute(request: ConnectorRequest): Promise<ConnectorResponse> {  
    const connector = await this.connectorRegistry.findById(request.connectorId);  
    if (!connector) {  
      throw new ConnectorError('CONNECTOR_NOT_FOUND', `Connector ${request.connectorId}  
not found`);  
    }  
  
    // Get credentials  
    const credentials = await  
this.credentialService.getCredentials(connector.credentials);
```

```
// Build adapter
const adapter = this.buildAdapter(connector);

// Rate limiting
await this.rateLimiter.check(connector.rateLimit);

// Execute with retry
const response = await this.retryHandler.execute(
  () => adapter.execute(request, credentials),
  connector.retryPolicy
);

// Transform response
const transformedData = this.transformResponse(response.data, connector.mapping);

return {
  success: true,
  data: transformedData,
  statusCode: response.statusCode,
  headers: response.headers,
  duration: response.duration
};
}

private buildAdapter(connector: ConnectorConfig): ConnectorAdapter {
  switch (connector.protocol) {
    case 'HTTP':
      return new HttpAdapter(connector.config);
    case 'SOAP':
      return new SoapAdapter(connector.config);
    case 'JDBC':
      return new JdbcAdapter(connector.config);
    case 'FILE':
      return new FileAdapter(connector.config);
    default:
      throw new ConnectorError('UNSUPPORTED_PROTOCOL', `Protocol ${connector.protocol}
not supported`);
  }
}
}
```


14. Marketplace Service

14.1 Service Overview

Domain	Plugin Marketplace
Owning Team	Platform Team
Primary Responsibility	Plugin publication, trust services, distribution, tenant installation

14.2 Core Components

14.2.1 Plugin Publication

TYPESCRIPT

```
interface PluginSubmission {
  id: UUID;
  pluginId: UUID;
  publisherId: UUID;
  manifest: PluginManifest;
  packageUrl: string;
  checksum: string;
  status: 'SUBMITTED' | 'REVIEWING' | 'SCANNING' | 'APPROVED' | 'REJECTED' | 'PUBLISHED';
  reviews: ReviewRecord[];
  securityScan: SecurityScanResult;
  submittedAt: Date;
  updatedAt: Date;
}

interface SecurityScanResult {
  status: 'PASSED' | 'FAILED' | 'WARNING';
  issues: SecurityIssue[];
  score: number;
  scannedAt: Date;
}

interface SecurityIssue {
  id: string;
  severity: 'CRITICAL' | 'HIGH' | 'MEDIUM' | 'LOW';
  type: 'VULNERABILITY' | 'MALWARE' | 'SECURITY' | 'CODE_QUALITY';
  description: string;
  recommendation: string;
  cveReference?: string;
}
```

14.2.2 Trust Model

TYPESCRIPT

```
interface TrustEvaluation {
  pluginId: UUID;
  overallScore: number;
  scores: {
    codeQuality: number;
    security: number;
    reliability: number;
    publisherReputation: number;
    communityTrust: number;
  };
  trustTier: 'GOLD' | 'SILVER' | 'BRONZE' | 'UNTRUSTED';
  evaluations: EvaluationRecord[];
  expiresAt: Date;
}
```

14.3 Signing, Versioning & Dependency Resolution

This section covers the parts of the plugin lifecycle not shown in the publication and trust-model code in 14.2: how a package is verified as untampered, how plugin versions are resolved against a manifest's declared dependencies, and where billing fits (or doesn't yet).

- **Signing:** each published plugin package is signed by the publisher's registered key at submission time; the Marketplace verifies the signature (not just the checksum shown in 14.2) before a plugin can move from **SCANNING** to **APPROVED**, and the Plugin Runtime re-verifies the signature at install time on the tenant's cluster.
- **Version management:** plugins follow semantic versioning; a tenant installs a version range (e.g. `^2.1.0`), and the Marketplace resolves the highest compatible published version at install and upgrade time.
- **Dependency resolution:** a plugin manifest declares its required OpenS2P API capabilities and minimum platform version; the Marketplace rejects installation if the target tenant's platform version doesn't satisfy the manifest, rather than allowing a runtime failure.
- **Billing:** not yet implemented. The **PluginSubmission** and trust-tier model in 14.2 are billing-agnostic by design so a billing layer (usage-based or listing-fee) can be added later without a schema migration, but no billing component exists in this version of the platform.

15. Identity Service

15.1 Service Overview

Domain	Identity Management
Owning Team	Platform Team
Primary Responsibility	Authentication, session management, user provisioning

15.2 Core Components

15.2.1 User Schema

TYPESCRIPT

```
interface User {
  id: UUID;
  username: string;
  email: string;
  firstName: string;
  lastName: string;
  tenantId: string;
  roles: string[];
  groups: string[];
  attributes: Record<string, any>;
  status: 'ACTIVE' | 'INACTIVE' | 'LOCKED';
  lastLogin: Date;
  createdAt: Date;
  updatedAt: Date;
}
```

15.2.2 JWT Claims

TYPESCRIPT

```
interface JWTClaims {
  sub: string; // User ID
  iss: string; // Issuer
  aud: string[]; // Audience
  exp: number; // Expiry
  iat: number; // Issued at
  tenant: string;
  roles: string[];
  groups: string[];
  permissions: string[];
  name: string;
```

```
    email: string;  
  }
```

15.3 Integration with Keycloak

TYPESCRIPT

```
// services/identity/src/infrastructure/keycloak/KeycloakService.ts  
  
export class KeycloakService {  
  private adminClient: KeycloakAdminClient;  
  
  /**  
   * Create user in Keycloak  
   */  
  async createUser(user: User): Promise<User> {  
    const keycloakUser = {  
      username: user.username,  
      email: user.email,  
      firstName: user.firstName,  
      lastName: user.lastName,  
      enabled: true,  
      attributes: {  
        tenantId: user.tenantId  
      }  
    };  
  
    const id = await this.adminClient.users.create(keycloakUser);  
  
    // Assign roles  
    await this.adminClient.users.addRealmRoleMappings({  
      id: id.id,  
      roles: user.roles.map(role => ({ name: role }))  
    });  
  
    return {  
      ...user,  
      id: id.id  
    };  
  }  
  
  /**  
   * Validate JWT token  
   */  
  async validateToken(token: string): Promise<JWTClaims> {  
    const payload = await this.adminClient.jwt.verify(token);  
    return {  
      sub: payload.sub,  
      iss: payload.iss,  
      aud: payload.aud,  

```

```
    exp: payload.exp,  
    iat: payload.iat,  
    tenant: payload.tenant,  
    roles: payload.realm_access?.roles || [],  
    groups: payload.groups || [],  
    permissions: payload.permissions || [],  
    name: payload.name,  
    email: payload.email  
  };  
}  
}
```

16. Notification Service

16.1 Service Overview

Domain	Notification Management
Owning Team	Platform Team
Primary Responsibility	Email, webhook, internal notifications

16.2 Core Components

TYPESCRIPT

```
interface Notification {
  id: UUID;
  type: 'EMAIL' | 'WEBHOOK' | 'PUSH' | 'SMS';
  recipient: string | string[];
  subject: string;
  body: string;
  template?: string;
  data: Record<string, any>;
  priority: 'LOW' | 'MEDIUM' | 'HIGH';
  status: 'PENDING' | 'SENT' | 'FAILED';
  attempts: number;
  createdAt: Date;
  sentAt?: Date;
}
```

17. Search Service

17.1 Service Overview

Domain	Global Search
Owning Team	Platform Team
Primary Responsibility	Unified search across all domains

17.2 Core Components

TYPESCRIPT

```
interface GlobalSearchRequest {
  query: string;
  types?: DocumentType[];
  filters?: SearchFilter[];
  sort?: SearchSort;
  limit?: number;
  offset?: number;
}

interface GlobalSearchResult {
  hits: SearchHit[];
  total: number;
  facets: SearchFacet[];
  took: number;
}

interface SearchHit {
  id: UUID;
  type: DocumentType;
  title: string;
  content: string;
  score: number;
  highlights: string[];
  metadata: Record<string, any>;
  url: string;
}
```

18. Data Model

18.1 Core Schema Definitions

18.1.1 Base Types

SQL

```
-- Infrastructure: shared base types
CREATE TYPE data_classification AS ENUM (
    'PUBLIC',
    'INTERNAL',
    'CONFIDENTIAL',
    'RESTRICTED'
);

CREATE TYPE approval_status AS ENUM (
    'PENDING',
    'APPROVED',
    'REJECTED',
    'REVIEWING'
);

CREATE TYPE user_reference AS (
    id UUID,
    name VARCHAR(255),
    email VARCHAR(255)
);
```

18.1.2 Audit Log

SQL

```
-- ClickHouse audit table
CREATE TABLE audit_log (
    timestamp DateTime64(9) DEFAULT now64(9),
    event_type String,
    tenant_id String,
    user_id UUID,
    session_id UUID,
    correlation_id UUID,
    trace_id String,
    service_name String,
    action String,
    resource_type String,
    resource_id UUID,
    request_id String,
    ip_address String,
    user_agent String,
    payload String,
```



```
status_code UInt16,  
duration_ms UInt32,  
is_error UInt8,  
error_code String,  
error_message String  
) ENGINE = MergeTree()  
ORDER BY (timestamp, tenant_id, event_type)  
PARTITION BY toYYYYMM(timestamp)  
TTL timestamp + INTERVAL 7 YEAR;
```

18.2 Schema-per-Tenant Implementation

SQL

```
-- Tenant isolation via schema per tenant  
CREATE SCHEMA IF NOT EXISTS tenant_${TENANT_ID};  
  
SET search_path TO tenant_${TENANT_ID};  
  
-- Tables within tenant schema  
CREATE TABLE sourcing_events ( ... );  
CREATE TABLE purchase_requisitions ( ... );  
-- etc.
```

19. APIs

19.1 API Design Standards

19.1.1 REST Standards

Aspect	Standard
URL Structure	/api/v{major}/{resource}
HTTP Methods	GET, POST, PUT, PATCH, DELETE
Status Codes	Standard HTTP semantics
Versioning	Path-based: /api/v1/
Pagination	limit, offset, total
Filtering	Query parameters
Sorting	sort parameter
Fields	fields for partial responses

19.1.2 Common Response Format

TYPESCRIPT

```
interface ApiResponse<T> {
  data: T;
  metadata: {
    timestamp: string;
    requestId: string;
    version: string;
  };
  links?: {
    self: string;
    next?: string;
    prev?: string;
  };
}

interface ApiError {
  error: {
    code: string;
    message: string;
    details?: Record<string, any>;
    timestamp: string;
    requestId: string;
  };
}
```

19.1.3 Error Codes by Service

Service	Prefix	Example
Sourcing	SRC	SRC-001: Event not found
Supplier	SUP	SUP-002: Supplier already exists
Contract	CTR	CTR-003: Contract expired
Purchasing	PUR	PUR-004: PO not found
Invoice	INV	INV-005: Three-way match failed
Workflow	WKF	WKF-006: Workflow not found
AI	AI	AI-007: Evaluation failed
Plugin	PLG	PLG-008: Plugin not signed
Connector	CON	CON-009: Connection failed

19.2.1 Gateway Responsibilities

- **Authentication:** validates the JWT on every request (27.1); requests without a valid token are rejected at the edge, never forwarded.
- **Authorisation (coarse-grained):** confirms the token's tenant and role claims are present and well-formed; fine-grained resource-level authorisation is delegated to OPA at the service (27.2) — the Gateway doesn't duplicate that logic.
- **Rate limiting & throttling:** per-tenant and per-route limits (see the `rateLimit` block in the routes configuration below) protect downstream services from both abuse and accidental overload.
- **API versioning:** routes requests to the correct service version per the `v{major}.{minor}` scheme (1.3), and attaches `Deprecation/Sunset` headers per the policy in 19.4.
- **Request validation:** rejects malformed requests (schema validation against the OpenAPI spec) before they consume a domain service's resources.
- **WAF (Web Application Firewall):** filters common attack patterns (injection, malformed headers, oversized payloads) at the edge, ahead of authentication — the one check that runs before token validation, since it protects the auth path itself.
- **Caching:** caches responses for explicitly cacheable, read-only routes (e.g. Search, read-only Knowledge queries) to reduce load on domain services; write routes are never cached.

Before the routing configuration below, this is what the API Gateway is architecturally responsible for — the contract every service can rely on being enforced before a request ever reaches it.

19.2 API Gateway Configuration

YAML

```
infrastructure/kubernetes/gateway/routes.yaml
routes:
  - path: /api/v1/sourcing
```

```

service: sourcing-service
port: 8080
methods: [GET, POST, PUT, PATCH, DELETE]
rateLimit:
  requestsPerSecond: 100
  burst: 200
cors:
  allowedOrigins: ["https://*.opens2p.io"]
  allowedMethods: ["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"]
  allowedHeaders: ["Authorization", "Content-Type", "X-Request-Id"]

- path: /api/v1/supplier
  service: supplier-service
  port: 8081
  methods: [GET, POST, PUT, PATCH]
  rateLimit:
    requestsPerSecond: 50
    burst: 100

... similar for all services

```

19.3 API Catalog

A service-level summary of every REST API surface. This is a navigation aid, not a replacement for the OpenAPI specification of each service (referenced in that service's "API Specification" section), which remains the source of truth for request/response schemas, idempotency keys, and full error codes.

Service	Base Path	Auth	Version	Idempotency
Sourcing	/api/v1/sourcing	JWT + OPA	v1	Yes — Idempotency-Key header on POST
Supplier	/api/v1/suppliers	JWT + OPA	v1	Yes
Contract	/api/v1/contracts	JWT + OPA	v1	Yes
Purchasing	/api/v1/purchasing	JWT + OPA	v1	Yes
Receiving	/api/v1/receiving	JWT + OPA	v1	Yes
Invoicing	/api/v1/invoicing	JWT + OPA	v1	Yes
Payment	/api/v1/payments	JWT + OPA	v1	Yes
Workflow Orchestrator	/api/v1/workflows	JWT + OPA	v1	Yes
AI Orchestrator	/api/v1/ai	JWT + OPA	v1	Yes (agent run ID)
Knowledge Service	/api/v1/knowledge	JWT + OPA	v1	N/A (read-heavy)
Plugin Runtime	/api/v1/plugins	JWT + OPA	v1	Yes
Connector Runtime	/api/v1/connectors	JWT + OPA	v1	Yes

Service	Base Path	Auth	Version	Idempotency
Marketplace	/api/v1/marketplace	JWT + OPA	v1	Yes
Identity	/api/v1/identity	OIDC / JWT	v1	N/A
Search	/api/v1/search	JWT + OPA	v1	N/A (read-only)
Notification	/api/v1/notifications	JWT + OPA	v1	Yes

19.4 Versioning & Deprecation Policy

All REST APIs follow the `v{major}.{minor}` convention set out in 1.3. This section adds the lifecycle rules that convention implies but doesn't spell out.

- A new major version is introduced only for breaking changes (removed fields, changed semantics, incompatible auth changes); additive changes ship as the same major version.
- A deprecated API version is supported for a minimum of 6 months after its replacement ships, with the deprecation date and sunset date both published in the OpenAPI spec's `deprecated` and `x-sunset` fields.
- Deprecated endpoints return a `Deprecation` and `Sunset` HTTP header (RFC 8594) so API consumers can detect deprecation programmatically, not just via changelog.
- Breaking changes are announced in the Technology Radar (see Related Documents) before the deprecation window starts, not at the start of it.

20. Events

20.1 Event Standards

20.1.1 Event Envelope

TYPESCRIPT

```
interface DomainEventEnvelope {  
  id: UUID;  
  type: string;  
  version: string;  
  tenantId: string;  
  correlationId: UUID;  
  causationId: UUID;  
  timestamp: string;  
  traceId: string;  
  payload: Record<string, any>;  
  metadata: Record<string, any>;  
}
```

20.1.2 Kafka Topic Naming

CODE

```
{domain}-events
```

Examples:

- sourcing-events
- supplier-events
- contract-events
- purchasing-events
- invoice-events

20.1.3 Event Schema Evolution

YAML

```
Event schema evolution rules  
schemaEvolution:  
  majorVersion:  
    - Backward incompatible  
    - Requires new topic  
    - Dual-publish period: 6 months  
  minorVersion:  
    - Backward compatible
```

- Optional new fields
 - Deprecated fields marked with @deprecated
- patchVersion:
- Documentation only
 - No schema changes

20.2 Event Catalog

The complete list of domain events referenced throughout this document, consolidated in one place. This is the events counterpart to the API Catalog in 19.3 — a navigation aid backed by the AsyncAPI specification of each publishing service.

Event	Publisher	Primary Subscribers
SourcingEventPublished	Sourcing	Supplier
BidSubmitted	Sourcing	Sourcing (self), Notification
SourcingEventAwarded	Sourcing	Supplier, Purchasing, Contract
SupplierRegistered	Supplier	Workflow, Search
SupplierQualified	Supplier	Sourcing, Purchasing, Contract, Notification
SupplierSuspended	Supplier	Purchasing, Sourcing
ContractExecuted	Contract	Invoicing, Search
ContractExpired	Contract	Notification, Workflow
PurchaseRequisitionCreated	Purchasing	Workflow
PurchaseRequisitionApproved	Purchasing	Notification
PurchaseOrderIssued	Purchasing	Receiving, Invoicing, Supplier-facing Notification
GoodsReceiptRecorded	Receiving	Invoicing
InvoiceMatched	Invoicing	Payment
InvoiceApproved	Invoicing	Payment, Notification
PaymentScheduled	Payment	Notification
PaymentIssued	Payment	Invoicing, Notification
WorkflowStarted	Workflow Orchestrator	AI Orchestrator, Notification
WorkflowCompleted	Workflow Orchestrator	Originating domain service, Notification
AgentActionProposed	AI Orchestrator	Workflow Orchestrator
AgentActionApproved	AI Orchestrator	Originating domain service

Event	Publisher	Primary Subscribers
PluginExecuted	Plugin Runtime	Audit / ClickHouse sink
ConnectorInvoked	Connector Runtime	Audit / ClickHouse sink

Every event name follows the {Aggregate}{Verb}ed convention from 1.3; if you're adding a new event, checking this table first is the fastest way to confirm it doesn't already exist under a slightly different name.

21. Error Handling

21.1 Error Hierarchy

TYPESCRIPT

```
// Base error
class OpenS2PError extends Error {
  constructor(
    public code: string,
    message: string,
    public details?: Record<string, any>
  ) {
    super(message);
    this.name = this.constructor.name;
  }
}

// Domain errors
class DomainError extends OpenS2PError { }
class AggregateNotFoundError extends DomainError { }
class BusinessRuleViolationError extends DomainError { }

// Infrastructure errors
class InfrastructureError extends OpenS2PError { }
class DatabaseError extends InfrastructureError { }
class MessagingError extends InfrastructureError { }

// Security errors
class SecurityError extends OpenS2PError { }
class UnauthorisedError extends SecurityError { }
class ForbiddenError extends SecurityError { }
```

21.2 Error Handling Middleware

TYPESCRIPT

```
export function errorHandler(): ErrorRequestHandler {
  return (err: Error, req: Request, res: Response, next: NextFunction) => {
    // Log error
    logger.error('Request error', {
      error: err.message,
      stack: err.stack,
      code: err instanceof OpenS2PError ? err.code : 'INTERNAL_ERROR',
      path: req.path,
      method: req.method,
      requestId: req.headers['x-request-id']
    });
  };
}
```

```
// Determine response
if (err instanceof OpenS2PError) {
  const status = this.getStatusCode(err);
  return res.status(status).json({
    error: {
      code: err.code,
      message: err.message,
      details: err.details,
      timestamp: new Date().toISOString(),
      requestId: req.headers['x-request-id']
    }
  });
}

// Generic error
return res.status(500).json({
  error: {
    code: 'INTERNAL_ERROR',
    message: 'An unexpected error occurred',
    timestamp: new Date().toISOString(),
    requestId: req.headers['x-request-id']
  }
});
};
}
```

22. Logging

22.1 Logging Standards

22.1.1 Structured Logging

TYPESCRIPT

```
// Use structured logging throughout
logger.info('Purchase order created', {
  orderId: po.id,
  orderNumber: po.orderNumber,
  supplierId: po.supplierId,
  totalAmount: po.totalAmount,
  tenantId: context.tenantId,
  userId: context.userId,
  traceId: context.traceId
});
```

22.1.2 Log Levels

Level	Usage
ERROR	System errors, exceptions
WARN	Business rule violations, security events
INFO	Important business events, state changes
DEBUG	Detailed debugging information (not in production)
TRACE	Very detailed tracing (not in production)

22.1.3 Log Field Standards

TYPESCRIPT

```
interface LogEntry {
  timestamp: string;
  level: string;
  service: string;
  message: string;
  // Common fields
  tenantId?: string;
  userId?: UUID;
  correlationId?: UUID;
  traceId?: string;
  // Business fields
  entityType?: string;
  entityId?: UUID;
```

```
    action?: string;
    // Error fields
    errorCode?: string;
    errorStack?: string;
    // Performance fields
    durationMs?: number;
    // Custom fields
    [key: string]: any;
}
```

23. Monitoring

23.1 Observability Stack

Component	Technology	Purpose
Metrics	Prometheus	Time-series metrics collection
Traces	OpenTelemetry	Distributed tracing
Logs	Loki	Log aggregation
Visualisation	Grafana	Dashboards and alerting
Alerting	AlertManager	Alert routing

23.2 Key Metrics

23.2.1 Service Metrics

YAML

```
metrics:
  Request metrics
  - name: http_requests_total
    type: counter
    labels: [service, method, path, status]
  - name: http_request_duration_seconds
    type: histogram
    labels: [service, method, path]
  - name: http_requests_in_flight
    type: gauge
    labels: [service]

  Business metrics
  - name: orders_created_total
    type: counter
    labels: [service, type]
  - name: orders_processing_time_seconds
    type: histogram
    labels: [service]

  AI metrics
  - name: ai_invocations_total
    type: counter
    labels: [agent, model, success]
  - name: ai_confidence_score
    type: gauge
    labels: [agent]
  - name: ai_hallucination_rate
    type: gauge
```

```

labels: [agent]
- name: ai_token_usage_total
  type: counter
  labels: [agent, model, type]

```

23.2.2 Infrastructure Metrics

YAML

```

infrastructure_metrics:
- name: kubernetes_pod_status
  type: gauge
  labels: [namespace, pod, status]
- name: kubernetes_node_cpu_usage
  type: gauge
  labels: [node]
- name: kubernetes_node_memory_usage
  type: gauge
  labels: [node]
- name: kafka_consumer_lag
  type: gauge
  labels: [topic, consumer_group]
- name: database_connections
  type: gauge
  labels: [database, pool]

```

23.3 Alerting Rules

YAML

```

Prometheus alert rules
groups:
- name: service_alerts
  rules:
- alert: HighErrorRate
  expr: rate(http_requests_total{status=~"5.."}[5m]) /
rate(http_requests_total[5m]) > 0.05
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "High error rate on {{ $labels.service }}"
    description: "Error rate is {{ $value }} for {{ $labels.service }}"

- alert: ServiceDown
  expr: up{service!="identity"} == 0
  for: 1m
  labels:
    severity: critical

```

```
30      annotations:
        summary: "Service {{ $labels.service }} is down"
        description: "Service has been down for {{ $value }} minutes"

- alert: AILatencyHigh
  expr: histogram_quantile(0.95, rate(ai_invocation_duration_seconds_bucket[5m])) >
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "High AI latency on {{ $labels.agent }}"
    description: "p95 latency is {{ $value }}s"

- alert: KafkaConsumerLag
  expr: kafka_consumer_lag > 10000
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "High Kafka consumer lag"
    description: "Consumer lag is {{ $value }} for topic {{ $labels.topic }}"
```

24. Performance

24.1 Performance Targets

Metric	Target	Measurement
API p95 Latency	< 400ms	HTTP requests
API p99 Latency	< 1s	HTTP requests
AI Agent p95 Latency	< 30s	Agent invocations
Database Query p99	< 100ms	SQL queries
Event Processing	< 100ms	Kafka consumers
Plugin Startup	< 1s	Wasm instances
Page Load	< 2s	UI requests

24.2 Performance Testing

TYPESCRIPT

```
// k6 load test script
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 }, // Ramp up
    { duration: '5m', target: 100 }, // Steady state
    { duration: '2m', target: 200 }, // Spike
    { duration: '5m', target: 200 }, // Steady state
    { duration: '2m', target: 0 }, // Ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<400'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const url = `http://purchasing-service:8080/api/v1/purchase-orders`;

  const response = http.get(url, {
    headers: {
      Authorization: `Bearer ${__ENV.JWT_TOKEN}`,
      'X-Request-Id': `${Date.now()}-${Math.random()}`,
    },
  });
};
```



```

check(response, {
  'status is 200': (r) => r.status === 200,
  'response time < 400ms': (r) => r.timings.duration < 400,
});

sleep(1);
}

```

24.3 Scaling Guidelines

Component	Scaling Strategy
API Services	Horizontal pod autoscaling
Kafka	Partition count = max consumers
PostgreSQL	Read replicas for queries
ClickHouse	Distributed cluster
Neo4j	Read replicas for queries
Redis	Cluster mode for cache
AI GPU Pool	Horizontal scaling with GPU nodes
Plugin Runtime	Horizontal scaling for Wasm

24.4 Performance Budgets by Service

The platform-wide targets in 24.1 apply as a floor; some services have materially different latency and throughput profiles by nature of what they do (e.g. AI inference vs. a simple CRUD lookup), so this table sets service-specific budgets. These are design targets to validate under load testing (24.2), not measured production numbers.

Service	p95 Latency Target	Throughput Target
Purchasing	200 ms	500 req/s
Sourcing	250 ms	300 req/s
Supplier	200 ms	300 req/s
Contract	300 ms	150 req/s
Receiving	200 ms	400 req/s
Invoicing	300 ms	300 req/s
Payment	300 ms	200 req/s
Knowledge Service (hybrid search)	1.5 s	100 searches/s
AI Orchestrator (agent run)	30 s	50 concurrent runs

Service	p95 Latency Target	Throughput Target
Identity Service (token validation)	50 ms	2000 req/s

25. Deployment

25.1 Container Configuration

DOCKERFILE

```
Multi-stage Dockerfile
FROM node:20-slim AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

FROM gcr.io/distroless/nodejs20-debian12
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER nonroot
EXPOSE 8080
CMD ["dist/index.js"]
```

25.2 Kubernetes Deployment

YAML

```
services/purchasing/helm/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: purchasing-service
  namespace: opens2p
spec:
  replicas: 3
  selector:
    matchLabels:
      app: purchasing-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: purchasing-service
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "8080"
```

```
    prometheus.io/path: "/metrics"
spec:
  containers:
    - name: purchasing-service
      image: registry.opens2p.io/purchasing-service:{{ .Values.image.tag }}
      ports:
        - containerPort: 8080
          name: http
      env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: purchasing-db-secret
              key: url
        - name: KAFKA_BROKERS
          value: "kafka-cluster.opens2p.svc:9092"
        - name: REDIS_URL
          value: "redis-cluster.opens2p.svc:6379"
        - name: OTEL_ENDPOINT
          value: "otel-collector.opens2p.svc:4317"
        - name: LOG_LEVEL
          value: "{{ .Values.logLevel }}"
      resources:
        requests:
          memory: "256Mi"
          cpu: "250m"
        limits:
          memory: "512Mi"
          cpu: "500m"
      livenessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 30
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /ready
          port: 8080
        initialDelaySeconds: 10
        periodSeconds: 5
      securityContext:
        allowPrivilegeEscalation: false
        runAsUser: 1000
        runAsGroup: 1000
        readOnlyRootFilesystem: true
      serviceAccountName: purchasing-service
      securityContext:
        fsGroup: 1000
```

25.3 Helm Values

YAML

```
services/purchasing/helm/values.yaml
image:
  repository: registry.opens2p.io/purchasing-service
  tag: latest
  pullPolicy: IfNotPresent

replicas: 3
logLevel: info

database:
  host: postgres-primary
  port: 5432
  database: opens2p
  maxConnections: 20

kafka:
  brokers: kafka-cluster:9092
  consumerGroup: purchasing-service

redis:
  host: redis-cluster
  port: 6379

resources:
  requests:
    memory: 256Mi
    cpu: 250m
  limits:
    memory: 512Mi
    cpu: 500m

autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
  targetMemoryUtilizationPercentage: 80

service:
  type: ClusterIP
  port: 8080
  targetPort: 8080

ingress:
  enabled: false

monitoring:
  enabled: true
  serviceMonitor:
```

```
enabled: true
interval: 30s
```

25.4 Environment Profiles

YAML

```
values-dev.yaml
replicas: 1
logLevel: debug
resources:
  requests:
    memory: 128Mi
    cpu: 100m
  limits:
    memory: 256Mi
    cpu: 200m
autoscaling:
  enabled: false

values-staging.yaml
replicas: 2
logLevel: info
resources:
  requests:
    memory: 256Mi
    cpu: 200m
  limits:
    memory: 512Mi
    cpu: 500m
autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 5

values-prod.yaml
replicas: 3
logLevel: info
resources:
  requests:
    memory: 256Mi
    cpu: 250m
  limits:
    memory: 512Mi
    cpu: 500m
autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
```

25.5 Deployment Topology

For the visual end-to-end deployment topology — ingress through to the managed data tier — see Figure 26.4 (C4 Deployment Diagram) in Section 26.4, which is the canonical topology diagram for the platform rather than duplicating it here.

26. C4 Architecture Views

This chapter presents OpenS2P using the full C4 model — Context, Container, Component, and Deployment — to complement the layered views in Section 2.

26.1 Level 1 — System Context

The context diagram shows OpenS2P as a single system, the people who use it, and the external systems it integrates with. Actor and integration categories are illustrative of the platform's intended footprint; specific external system names (e.g. a given ERP or payment provider) are determined per deployment via the Connector Runtime.

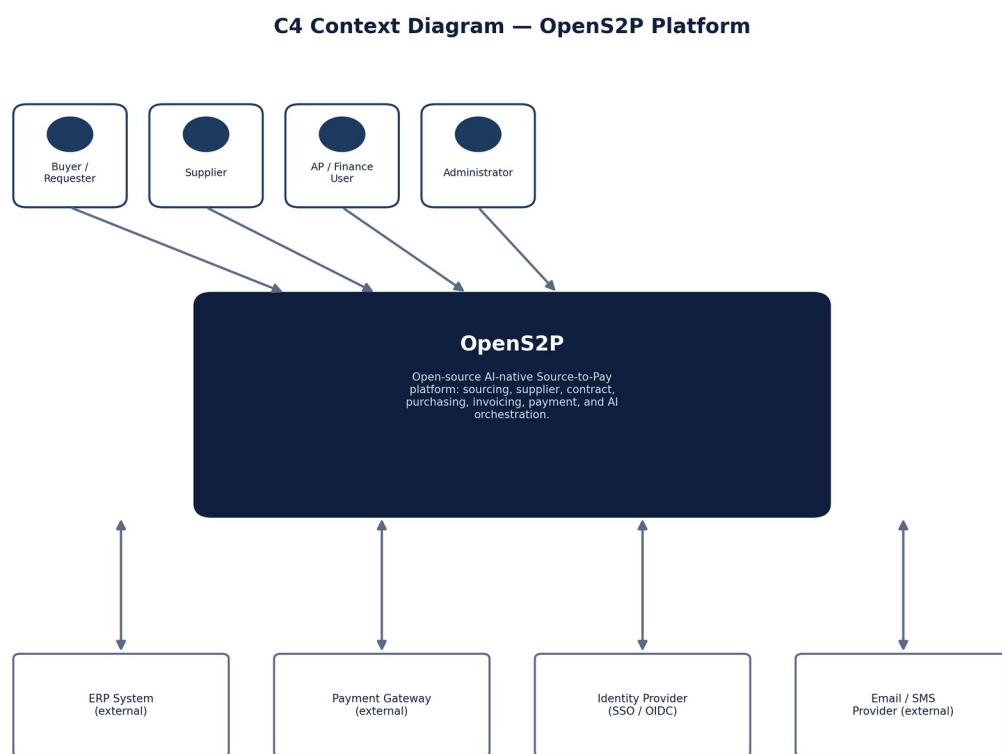


Figure 26.1 — C4 Context Diagram

26.2 Level 2 — Containers

The container diagram decomposes OpenS2P into its independently deployable units and the data stores each depends on, annotated with representative technology choices.

C4 Container Diagram — OpenS2P Platform

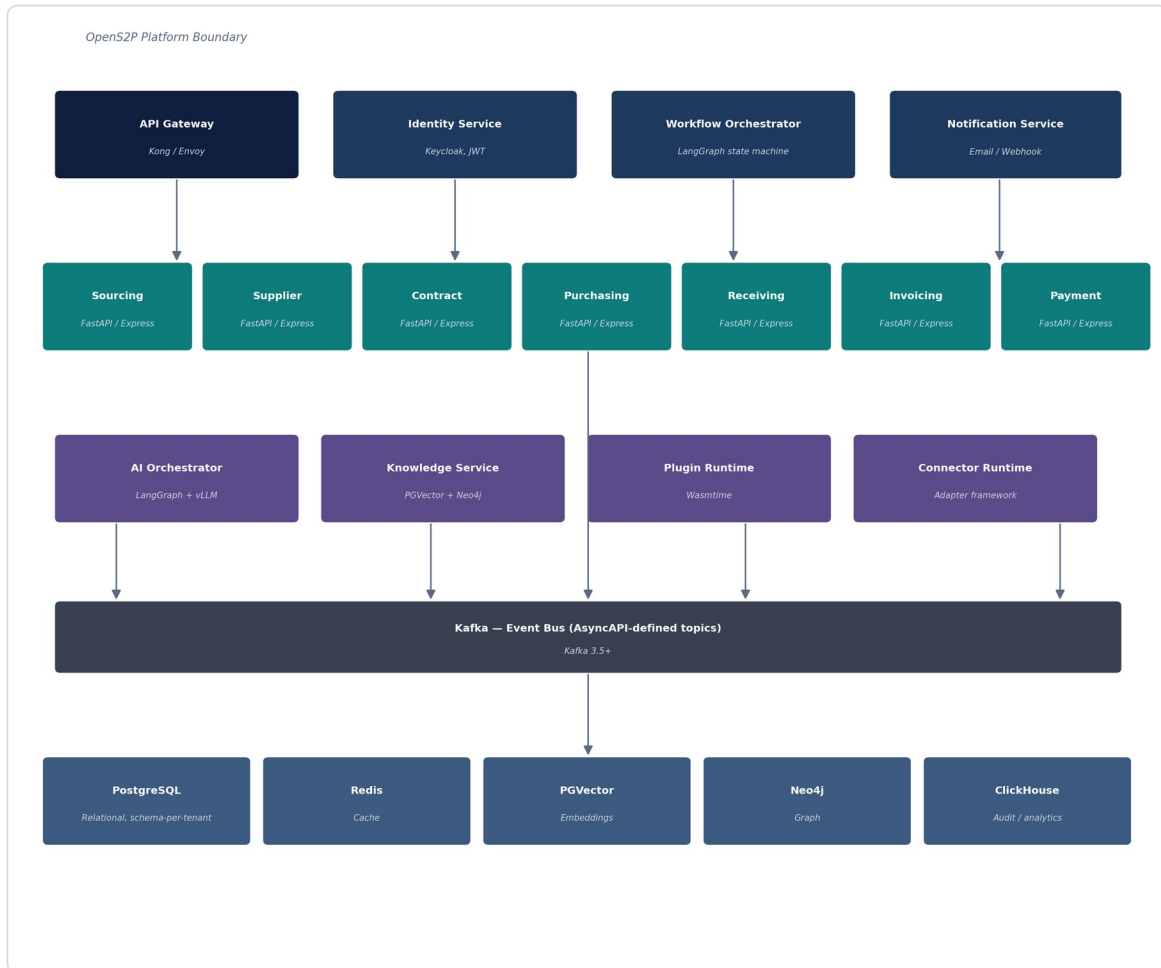


Figure 26.2 — C4 Container Diagram

26.3 Level 3 — Components

Rather than diagram all 16 services at component level, this section uses the Sourcing Service as a representative example — its internal structure (controllers, command/query handlers, aggregates, repositories) follows the same layered pattern used by every domain service in this document.

C4 Component Diagram — Sourcing Service

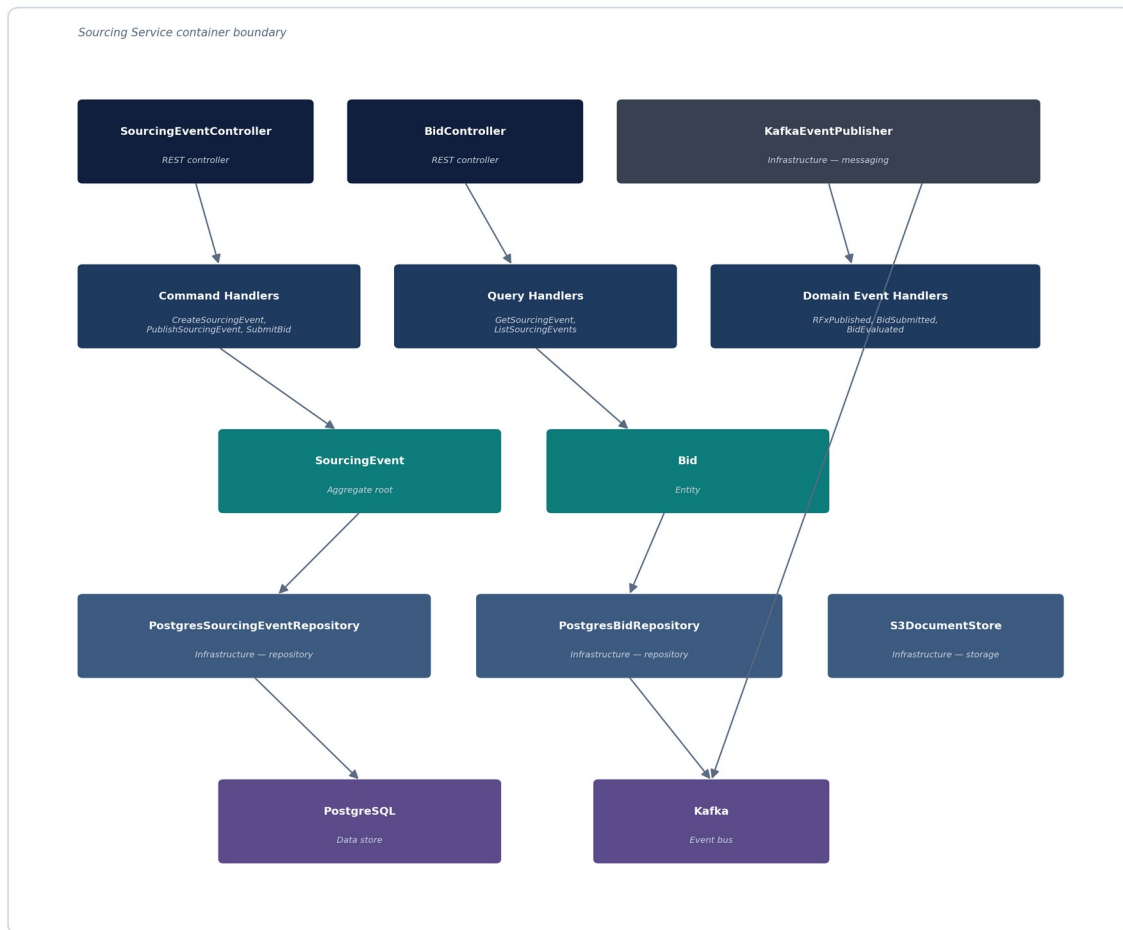


Figure 26.3 — C4 Component Diagram (Sourcing Service)

26.4 Level 4 — Deployment

The deployment diagram shows how containers map to Kubernetes: ingress and API Gateway at the edge, domain and AI/platform services grouped by namespace, Kafka as the event backbone, and the managed data tier. Section 25 (Deployment) covers the Helm and environment-profile detail behind this topology.

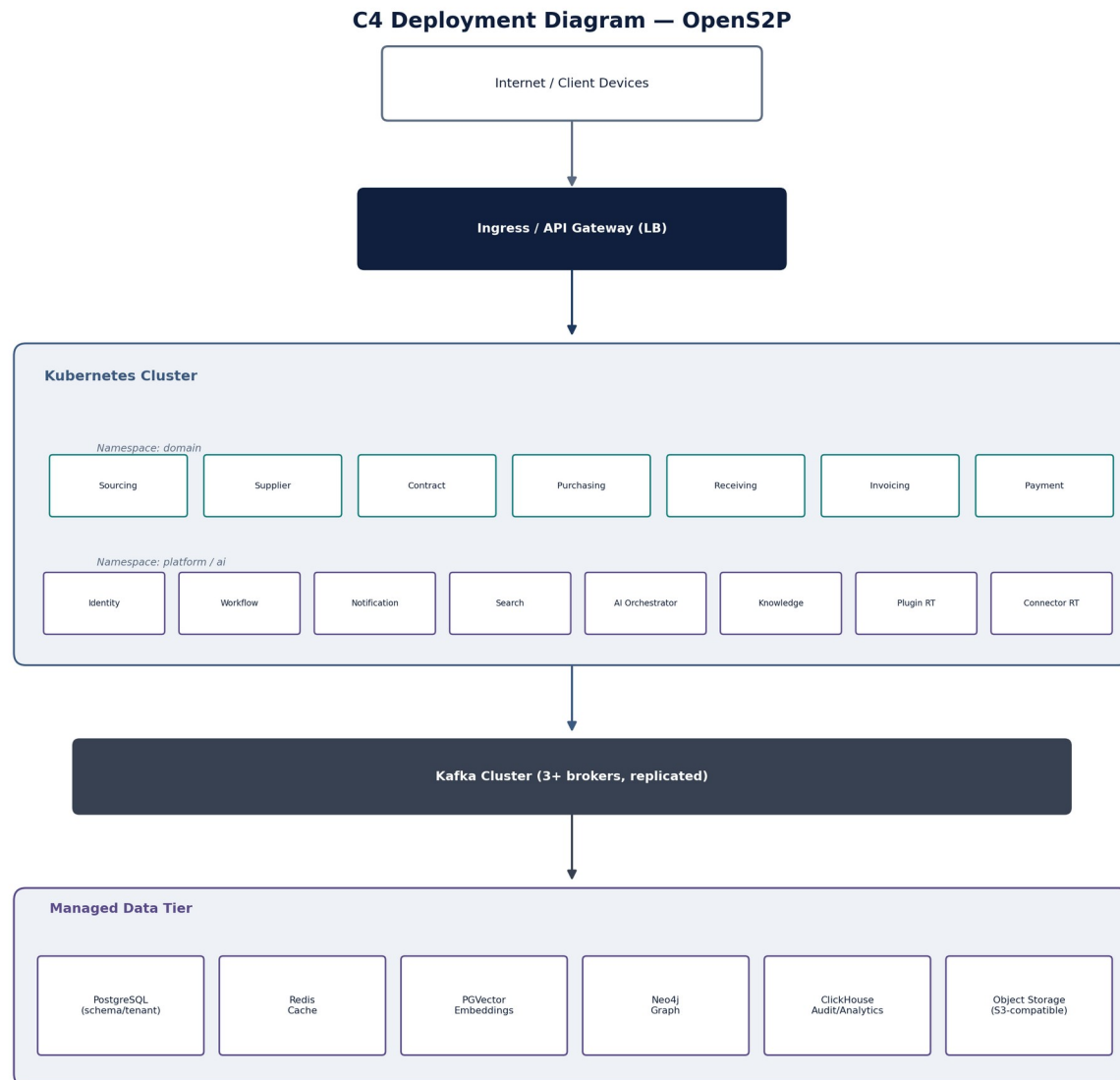


Figure 26.4 — C4 Deployment Diagram

27. Security Architecture

Security is addressed throughout this document at the service level; this chapter consolidates the platform-wide security model. Items marked (baseline, ratified) reflect decisions already established by the referenced ADRs. Items marked (recommended default — pending ratification) are proposed specifics, consistent with the stated technology stack, that still need sign-off from the security review rather than facts already decided; treat the numbers as a starting point for that discussion, not a commitment.

27.1 Authentication

- Recommended: service-to-service traffic inside the cluster uses mTLS (mesh-issued certificates) in addition to the JWT described above, so a compromised JWT alone is not sufficient to impersonate a service. (recommended default — pending ratification)
- Human user authentication uses the OAuth2 Authorization Code flow with PKCE via Keycloak; service accounts use the Client Credentials flow. Neither the Implicit flow nor Resource Owner Password Credentials flow is used, per current OAuth2 security best practice.
- Identity Service (Keycloak) issues OIDC-compliant tokens for all human users. (baseline, ratified — ADR-009)
- Service-to-service calls use short-lived JWTs validated at the API Gateway and re-validated by the receiving service. (baseline, ratified)
- Recommended token lifetimes: 15-minute access tokens, 7-day rotating refresh tokens, refresh reuse detection enabled. (recommended default — pending ratification)

27.2 Authorisation

- Fine-grained authorisation decisions are externalised to OPA (Open Policy Agent) rather than hard-coded per service. (baseline, ratified — ADR-008)
- Policies are evaluated per request using tenant, role, and resource attributes from the JWT claims. (baseline, ratified)
- Recommended policy workflow: Rego policies version-controlled alongside the service that owns them, reviewed via pull request, and rolled out through the same environment-promotion pipeline as code (Section 25.4). (recommended default — pending ratification)

27.3 Multi-Tenancy & Data Isolation

- Tenant data is isolated at the PostgreSQL schema level (schema-per-tenant). (baseline, ratified — ADR-004)
- All API requests carry a tenant context derived from the authenticated token; services must not accept a tenant identifier from an unauthenticated source. (baseline, ratified)

Schema-per-tenant covers relational isolation; the dimensions below make the full multi-tenant boundary explicit across every data store the platform uses.

- **Row-level isolation:** every table carries a `tenant_id`; application-layer repositories scope every query by the authenticated tenant, and PostgreSQL row-level security policies provide a second enforcement layer beneath the application code.
- **Cache partitioning:** Redis keys are namespaced by tenant (`{tenant_id}:{key}`) so a cache flush or eviction in one tenant cannot leak into or evict another's data.
- **Vector index isolation:** PGVector embeddings carry `tenant_id` as a filter predicate on every similarity search; the Knowledge Service does not run cross-tenant retrieval under any code path.
- **Audit isolation:** ClickHouse audit records are tenant-scoped and queried only with an explicit tenant filter; audit export/reporting tools enforce the same scoping.
- **Backup isolation:** because isolation is schema-level rather than database-level, backup and restore procedures (Appendix H) must support restoring a single tenant's schema without affecting others — this is a requirement on the backup tooling, not just the schema design.
- **Tenant-aware routing:** the API Gateway resolves tenant context from the authenticated token (not from the URL or a header a client could spoof) before any request reaches a domain service.
- **RBAC:** roles are tenant-scoped — a Buyer role in tenant A carries no permissions in tenant B, even for the same authenticated user, if that user belongs to multiple tenants.
- **Quotas:** per-tenant rate limits and resource quotas (API calls/min, AI agent runs/day, storage) are enforced at the API Gateway and AI Orchestrator respectively, so one tenant's usage spike cannot degrade service for others.

27.3.1 Tenant Lifecycle

- **Provisioning:** a new tenant is created by provisioning its PostgreSQL schema, a Redis key namespace, and a Keycloak realm (or realm partition) in one orchestrated operation — never as separate manual steps that could leave a tenant partially set up.
- **Onboarding / offboarding:** onboarding activates a tenant's schema and issues initial admin credentials via Identity Service; offboarding deactivates access immediately but retains data per the retention policy (Appendix H) rather than deleting it outright, pending an explicit data-deletion request.
- **Tenant encryption keys:** each tenant's data-at-rest encryption key is distinct (envelope encryption via the key management approach in 27.4), so revoking one tenant's key cannot be done accidentally when rotating another's.
- **Tenant migration:** moving a tenant between environments (e.g. staging to production, or between regions once multi-region exists — see Appendix H) is a schema export/import operation scoped to that tenant alone, consistent with the backup-isolation requirement in 27.3.

27.4 Secrets, Encryption & Key Management

- Data in transit is encrypted via TLS 1.2+ between all services and at the API Gateway edge. (baseline, ratified)
- Recommended secrets management: Kubernetes Secrets for local/dev profiles, an external secrets manager (e.g. Vault-compatible) for staging and production, mounted via a CSI driver rather than environment variables. (recommended default — pending ratification)

- Recommended key rotation: quarterly rotation for data-at-rest encryption keys, immediate rotation on suspected compromise, rotation events written to the audit store (Section 27.5). (recommended default — pending ratification)

27.5 Audit & Zero Trust

- Security-relevant events are written to the ClickHouse audit store referenced in Section 22 (Logging). (baseline, ratified)
- Zero-trust posture: no service trusts another purely on network location; every call is authenticated and authorised per the model above. (baseline principle, see Section 1.5)
- A starting threat model is provided in Appendix G; it should be extended into a full exercise (STRIDE per service, or equivalent) before public release. (recommended next step)

28. Quality Attribute Scenarios

Quality attribute scenarios make the non-functional requirements measurable. Each scenario follows the stimulus / response pattern used in enterprise architecture reviews. This is a representative starting set, not an exhaustive test plan.

28.1 Availability

- **Scenario:** A Purchasing Service pod fails during business hours. **Response:** Kubernetes detects the failed liveness probe and replaces the pod; in-flight requests to the failed pod are retried by the API Gateway against a healthy replica. **Target:** service restored within 30 seconds, zero data loss (state is persisted, not held in-memory).

28.2 Performance

- **Scenario:** 500 concurrent purchase order creations. **Target:** 95th-percentile API latency under 400ms, with the event publish to Kafka completing asynchronously and not blocking the API response.

28.3 Security

- **Scenario:** A request arrives with an expired JWT. **Response:** the API Gateway rejects the request with 401 Unauthorized before it reaches any domain service; no partial processing occurs.

28.4 Scalability

- **Scenario:** Sourcing event volume triples during a quarterly RFX cycle. **Response:** the Sourcing Service scales horizontally via Kubernetes HPA based on CPU and queue-depth metrics; PostgreSQL connection pooling (Appendix C) prevents connection exhaustion.

28.5 AI Governance

- **Scenario:** An AI agent proposes an action with a confidence score below the configured threshold. **Response:** the Agent Execution Pipeline (Figure 10.1) routes the action to Human Approval rather than executing it automatically.

28.6 NFR Traceability

This matrix maps each non-functional requirement category to where it is defined and measured in this document, so a reviewer can confirm coverage at a glance.

Requirement	Primary Section	Scenario / Detail
Availability	28.1	Pod failure / replacement scenario
Performance	28.2, 24	Latency targets, performance section
Security	27, 28.3	Security Architecture chapter + JWT expiry scenario

Requirement	Primary Section	Scenario / Detail
Scalability	28.4, 24	HPA scenario, performance section
Observability	23	Monitoring
AI Governance	28.5, 10.4	Confidence scorer scenario and component
Multi-Tenancy	27.3	Data isolation model
Disaster Recovery	Appendix H	Backup approach and recovery objectives per data store

29. Key Business Flows

The service and API chapters describe each capability in isolation. This chapter shows how four representative flows play out across services over time, as sequence diagrams, plus one data-flow view of the same underlying process for a complementary perspective.

29.0 Procure-to-Pay (End-to-End)

Before the individual flows below, this is the full procure-to-pay journey end to end — requisition through payment — showing how the individually-scoped sequences that follow fit together as one continuous process.

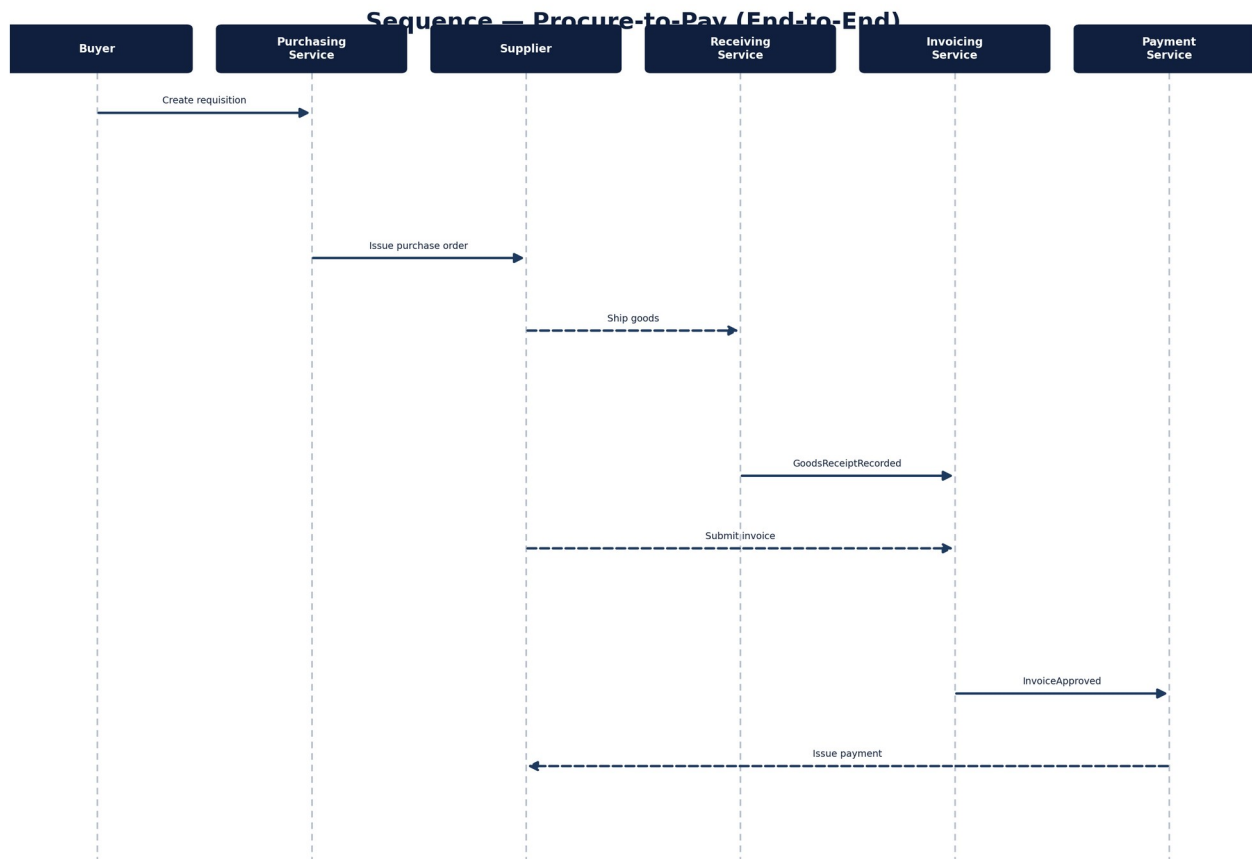


Figure 29.0 — Procure-to-Pay End-to-End Sequence

29.1 Purchase Requisition Approval

A buyer creates a purchase requisition; the Purchasing Service publishes a domain event and hands approval routing to the Workflow Orchestrator, which requests and receives approval before the requisition is marked approved.

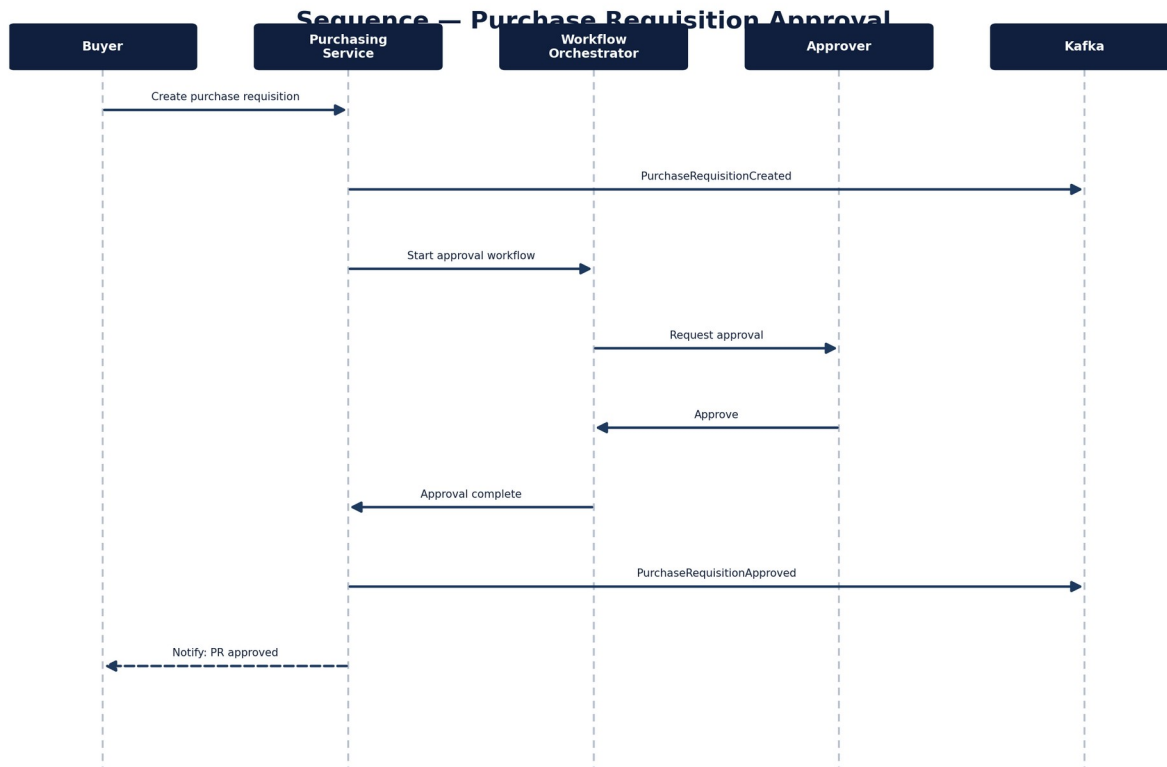
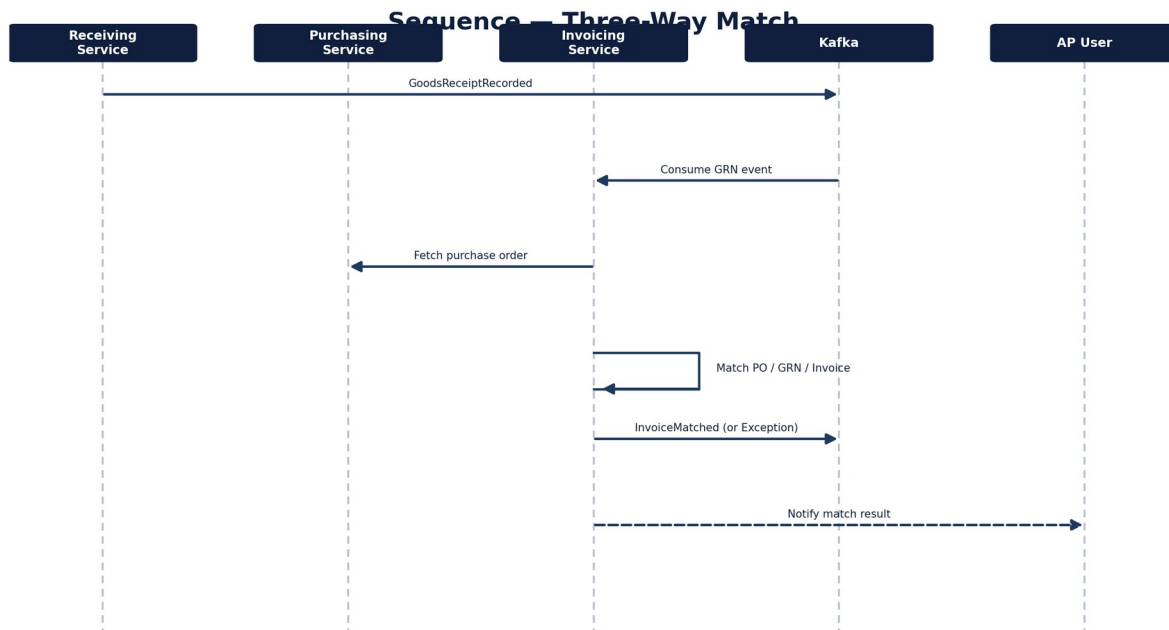


Figure 29.1 — Purchase Requisition Approval Sequence

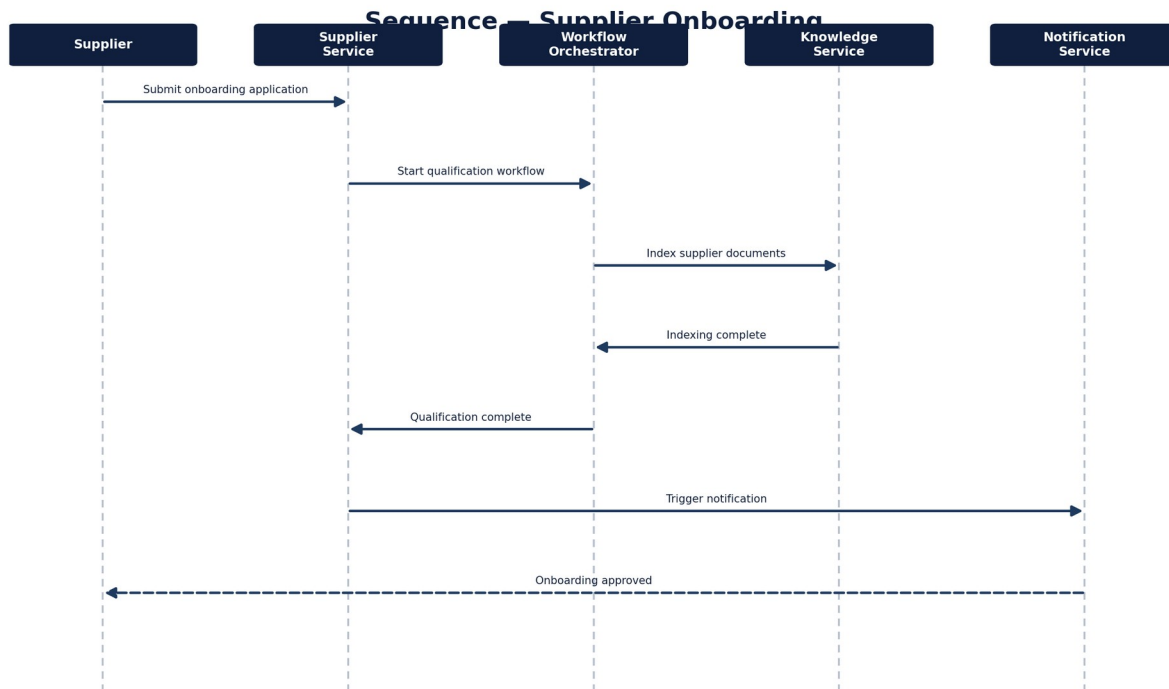
29.2 Three-Way Match

The Invoicing Service reconciles a purchase order, goods receipt, and invoice before payment is authorised — the core control that prevents paying for goods never ordered or never received.

*Figure 29.2 — Three-Way Match Sequence*

29.3 Supplier Onboarding

A prospective supplier submits an onboarding application; the Workflow Orchestrator drives qualification, including indexing supplier documents into the Knowledge Service, before the supplier is marked qualified and notified. The data-flow view below shows the same process from the perspective of where the data goes rather than which service calls which.

*Figure 29.3 — Supplier Onboarding Sequence*

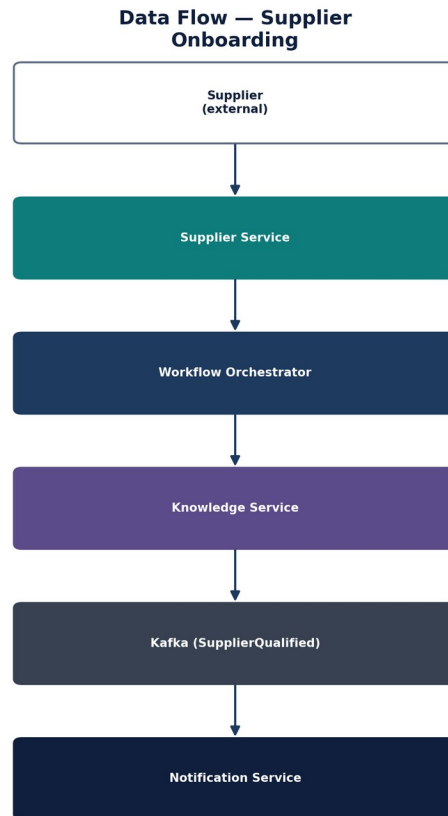


Figure 29.4 — Supplier Onboarding Data Flow

29.4 AI Agent Execution

A user request flows through the AI Orchestrator, which retrieves context from the Knowledge Service, proposes an action against a domain service, and routes low-confidence actions to a human approver before returning output — the same pipeline shown structurally in Figures 10.1 and 10.2, here shown as a time-ordered sequence.

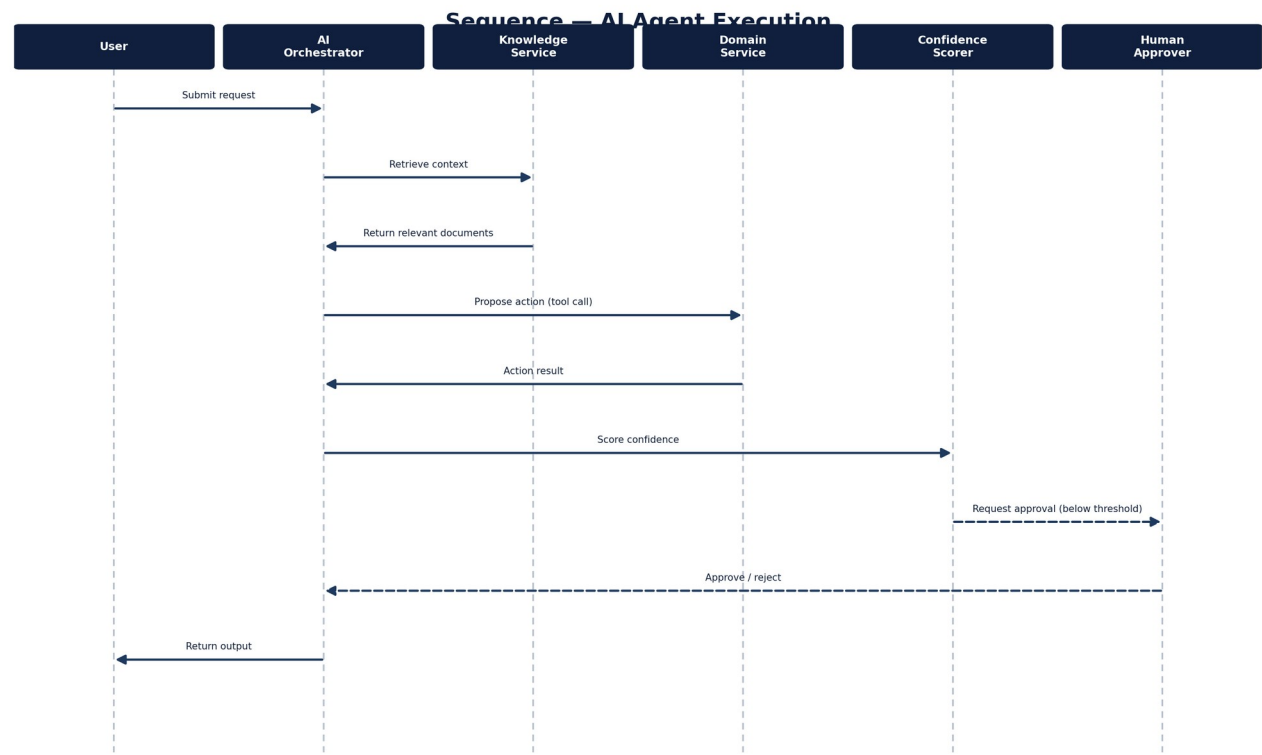


Figure 29.5 — AI Agent Execution Sequence

29.5 RFx Lifecycle

A buyer publishes an RFx; suppliers submit bids; the buyer evaluates and the Sourcing Service scores and ranks bids before recording an award.

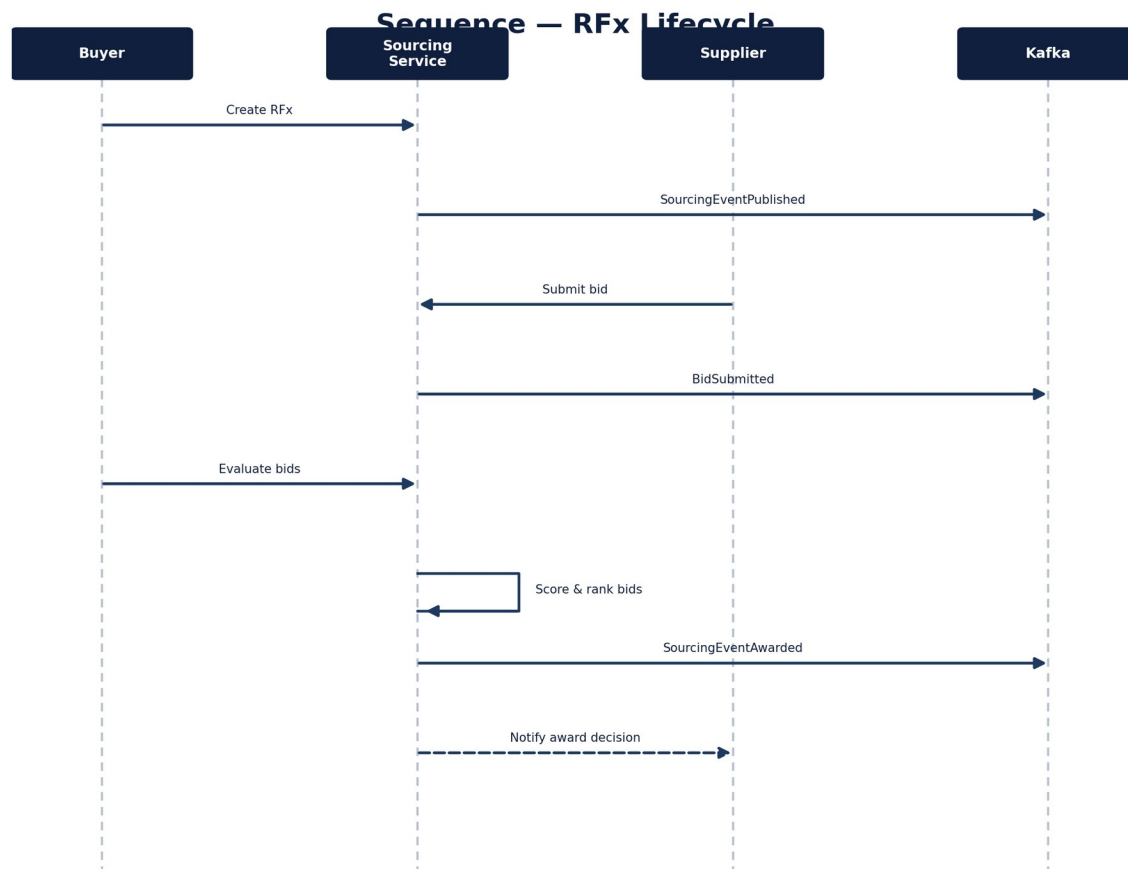


Figure 29.6 — RFX Lifecycle Sequence

29.6 AI-Assisted Contract Review

The Contract Service delegates clause analysis to the AI Orchestrator, which retrieves precedent clauses from the Knowledge Service before returning risk flags and suggestions for buyer review.

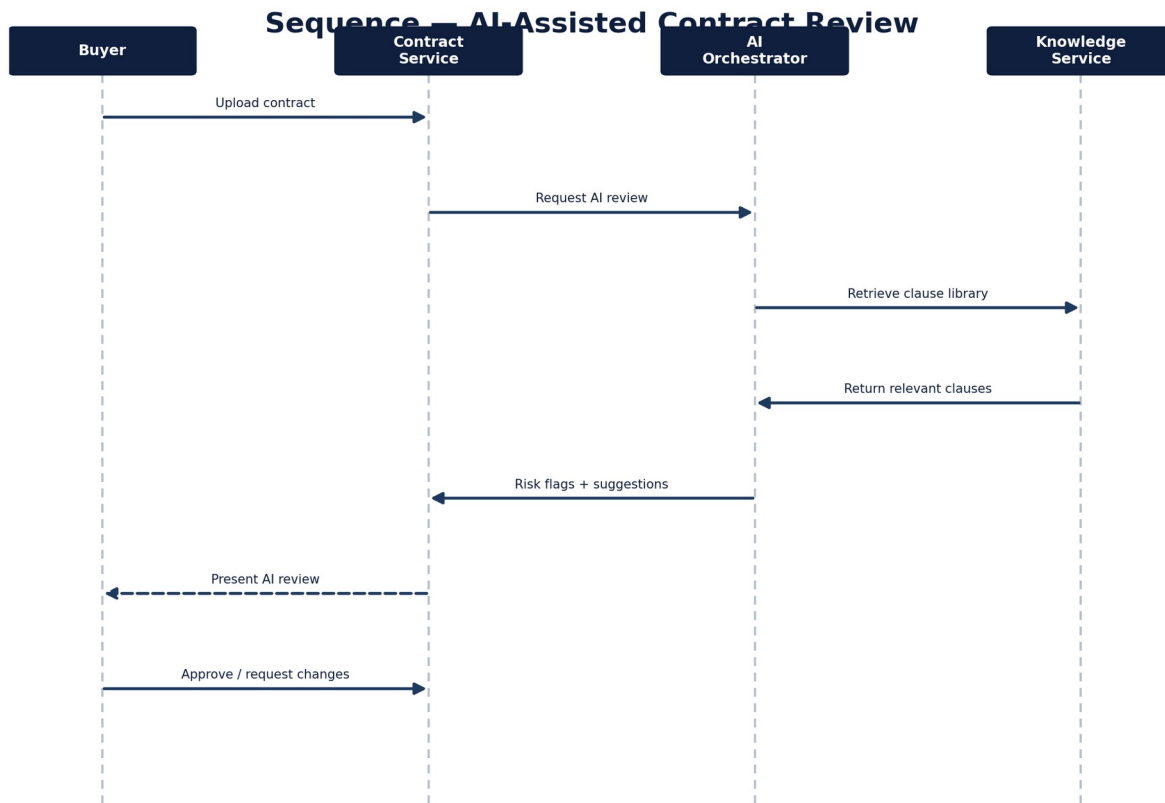
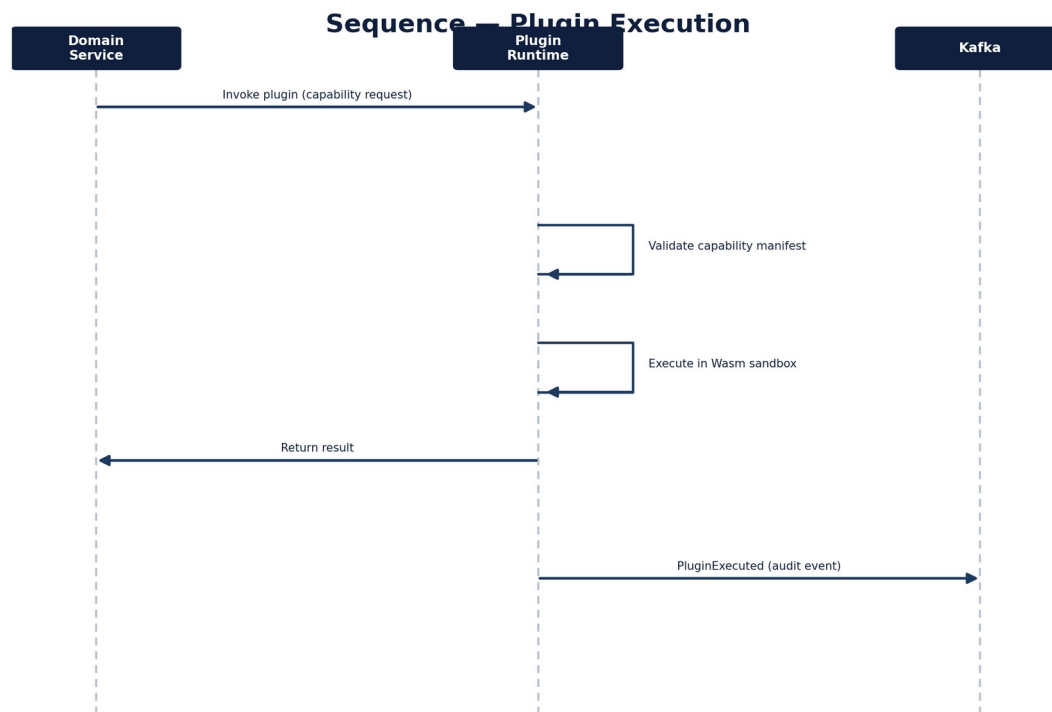


Figure 29.7 — AI-Assisted Contract Review Sequence

29.7 Plugin Execution

A domain service invokes a plugin by capability rather than by name; the Plugin Runtime validates the plugin's manifest before executing it inside a Wasmtime sandbox and recording the execution for audit.

*Figure 29.8 — Plugin Execution Sequence*

29.8 Connector Invocation

A domain service asks the Connector Runtime to synchronise with an external system; the runtime resolves the correct adapter configuration, makes the external call, maps the response back into OpenS2P's domain shape, and records the invocation for audit.

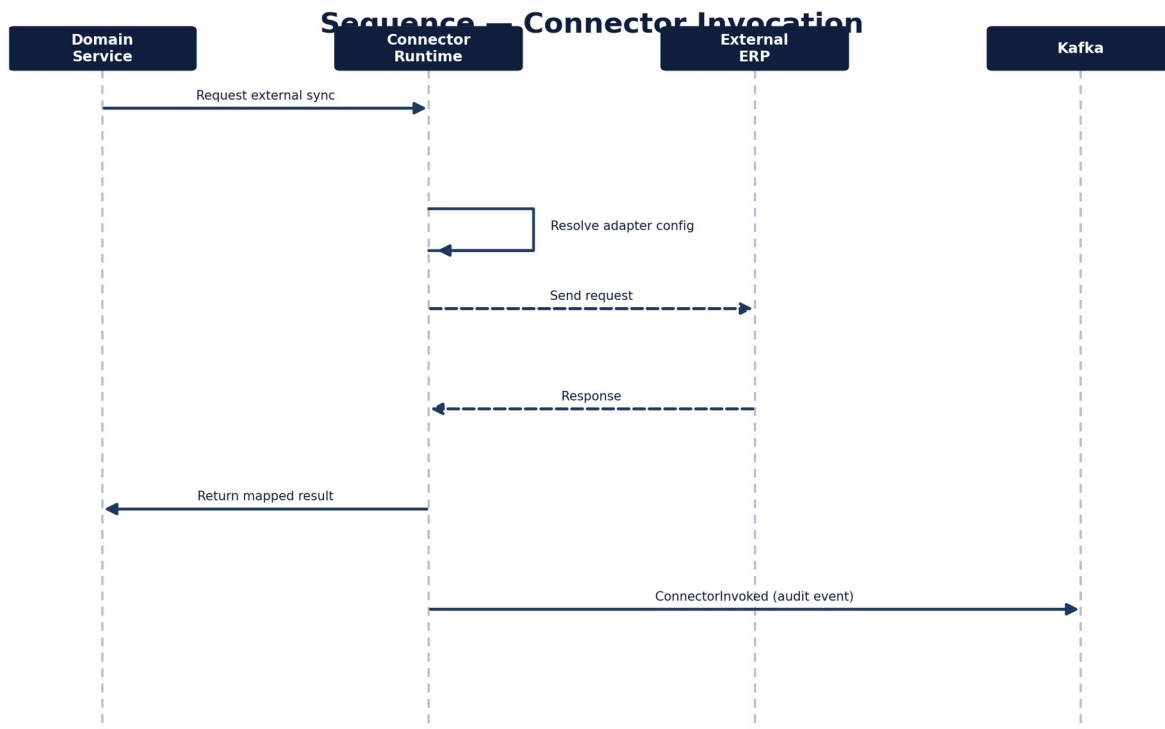


Figure 29.9 — Connector Invocation Sequence

30. Resilience & Failure Handling

Individual resilience mechanisms are implied throughout the service chapters (retry examples in code, the Saga row in the communication-patterns table at 2.3, dead-letter handling in event contracts) but not consolidated. This chapter is that consolidation — the patterns every service is expected to follow, not a new set of decisions.

30.1 Circuit Breakers

Synchronous calls between services (including to the API Gateway's upstream targets) should be wrapped in a circuit breaker that opens after a configurable failure threshold, so a struggling downstream service doesn't cascade load failures upstream. Recommended defaults: 50% failure rate over a 10-request rolling window opens the circuit; a 30-second half-open probe interval.

30.2 Retry Strategy

Retries apply to idempotent operations only (see 30.4). Recommended defaults: exponential backoff starting at 100ms, maximum 3 attempts, full jitter to avoid retry storms across concurrent requests.

30.3 Bulkheads

Each service isolates its outbound connection pools per downstream dependency (e.g. a separate PostgreSQL pool from the Kafka producer pool) so exhaustion in one dependency doesn't starve unrelated request paths.

30.4 Idempotency

State-changing API operations accept an idempotency key (typically the request ID); a service receiving a duplicate key returns the original result rather than re-executing the operation. This is what makes the retry strategy in 30.2 safe.

30.5 Dead-Letter Queues

Kafka consumers route events that fail processing after retry exhaustion to a per-topic dead-letter topic rather than blocking the consumer group or silently dropping the event. Dead-lettered events are visible to operators via the monitoring stack (Section 23) and require manual or automated replay.

30.6 Saga Compensation

Multi-service transactions (e.g. purchase order issuance touching Purchasing, Supplier, and Payment) use the choreographed saga pattern referenced in 2.3: each step publishes a domain event, and a failure at any step publishes a compensating event that prior steps listen for and react to (e.g. releasing a reserved budget). There is no distributed two-phase commit anywhere in the platform.

31. AI Governance

Chapter 10 describes how the AI Orchestrator executes an agent run; this chapter describes how that capability is governed — versioned, evaluated, monitored, and overridden — which is a distinct concern from execution mechanics.

31.1 Prompt & Model Versioning

Prompts are version-controlled artifacts, not inline strings: each prompt template used by the AI Orchestrator carries a version identifier, and an agent run's audit record (31.6) captures which prompt version and which model version produced a given action. This is what makes a past decision reproducible and explainable after the fact.

31.2 Model Registry & Fallback

- Models available to the AI Orchestrator are registered with their capability profile (context window, supported tool-calling format, cost tier) rather than hardcoded per call site.
- A fallback chain is defined per agent task: if the primary model is unavailable or exceeds a latency budget, the orchestrator falls back to a secondary model rather than failing the request outright — the fallback event is itself audited.

31.3 Evaluation & Hallucination Detection

- Agent behaviour is validated against a maintained evaluation dataset of representative procurement scenarios before a new prompt or model version is promoted to production.
- Retrieval-grounded responses (via the Knowledge Service) are checked for citation support — an answer that can't be traced to a retrieved document is treated as lower-confidence input to the Confidence Scorer (10.4), not presented as fact.

31.4 Guardrails

- Tool calls available to an agent are capability-scoped per task (an agent reviewing a contract cannot invoke a payment-issuing tool), independent of the confidence-score gate.
- Output is validated against the expected schema before being handed to a domain service — a malformed or out-of-range proposed action is rejected before it reaches the Confidence Scorer, not after.

31.5 Cost Monitoring

Token usage and inference cost are recorded per agent run and rolled up per tenant, feeding both the quota enforcement described in 27.3 and standard cost dashboards alongside the platform's other OpenTelemetry metrics (Section 23).

31.6 Human Override & AI Audit Log

- Every agent action — approved automatically or by a human — is written to the audit trail shown in Figure 10.2, including the confidence score, the prompt/model version (31.1), and the identity of the approving human where applicable.
- A human reviewer can override an agent's proposed action at the Human Approval gate (Figure 10.3); overrides are themselves audited and are the primary input to the evaluation dataset in 31.3 over time.

32. Catalog Service (Planned)

This is a planned-service placeholder, not a fully architected service like Chapters 4–17. Detailed domain model, aggregates, state machines, APIs, and events will be defined during SRS / detailed design, once the BRD and SRS exist to inform them. Designing the full architecture now, ahead of those documents, risks locking in a data model before requirements are mature — see the OpenS2P Product Requirements Document, Section 9.2, for the product-side context on this decision.

Domain	Purchasing Content
Owning Team	TBD
Primary Responsibility	Manage the buying experience — what can be purchased, from whom, and at what price.

32.1 Purpose

Give buyers a curated, priced set of purchasable items and services, sourced from suppliers, contracts, and punch-out catalogs, rather than requiring every purchase to start from a blank requisition.

32.2 Responsibilities

- Supplier catalogs — items a supplier offers, ingested or supplier-maintained
- Internal catalogs — buyer-curated item sets
- PunchOut catalogs — live redirect to a supplier's own e-commerce site with cart hand-back (cXML/OCI-style integration, pattern only — not yet designed)
- Contract catalogs — items and pricing tied to an executed contract (Chapter 6)
- Item master references
- Pricing and price-book management
- Catalog approval workflow (via the existing Workflow Orchestrator, Chapter 9 — not a new approval mechanism)
- Catalog search (via the existing Search Service, Chapter 17)
- Catalog recommendations (via the AI Orchestrator, Chapter 10)

32.3 Dependencies

Depends on: Supplier Service, Contract Service, Purchasing Service, Search Service, AI Orchestrator. Reuses existing platform capability (Workflow, Search, AI) rather than duplicating it — consistent with the pattern every other service in this document follows.

32.4 Priority

Should Have — see OpenS2P PRD Section 10.15 and 17 (Release Plan).

33. Analytics Service (Planned)

This is a planned-service placeholder, not a fully architected service like Chapters 4–17. Detailed domain model, aggregates, state machines, APIs, and events will be defined during SRS / detailed design, once the BRD and SRS exist to inform them. Designing the full architecture now, ahead of those documents, risks locking in a data model before requirements are mature — see the OpenS2P Product Requirements Document, Section 9.2, for the product-side context on this decision.

Domain	Procurement Intelligence
Owning Team	TBD
Primary Responsibility	Turn the event and audit data every other service already produces into decision-support reporting.

33.1 Purpose

Close the loop flagged throughout this document and the PRD (Section 16): operational, financial, supplier, and executive reporting, without each service building its own bespoke reporting layer.

33.2 Responsibilities

- Spend analytics — by category, supplier, cost centre
- Supplier performance analytics
- Savings tracking (from Sourcing awards, Chapter 4)
- Cycle-time analytics (procure-to-pay, Figure 29.0)
- Compliance dashboards
- Executive dashboards
- AI-generated insights (via AI Orchestrator, Chapter 10)
- Predictive analytics foundation — explicitly foundational only; forecasting itself is a future capability (PRD 17.1), not in scope for this stub

33.3 Dependencies

Depends on: event streams from every domain service (SAD 20.2 Event Catalog), ClickHouse (already provisioned as the audit store, Chapter 22 — this service would extend its use, not introduce a new store), Knowledge Service, AI Orchestrator.

33.4 Priority

Should Have — see OpenS2P PRD Section 10.16 and 17 (Release Plan).

33.5 Open Question: Dedicated Service vs. Reporting Layer

This chapter treats Analytics as its own service per the current decision, but the underlying architectural question — a dedicated service vs. a reporting layer over the existing ClickHouse store — noted in earlier drafts of this document is not fully closed by that decision alone. Whoever picks this up for detailed design should confirm event volume and query-pattern requirements before committing to a service boundary, rather than treating 'it's a service now' as having settled the implementation approach too.

Appendices

A. Service Port Allocation

Service	Port	Internal	External
Sourcing	8080	✓	✓
Supplier	8081	✓	✓
Contract	8082	✓	✓
Purchasing	8083	✓	✓
Receiving	8084	✓	✓
Invoicing	8085	✓	✓
Payment	8086	✓	✓
Workflow	8090	✓	✗
AI Orchestrator	8091	✓	✗
Knowledge	8092	✓	✗
Plugin Runtime	8093	✓	✗
Connector Runtime	8094	✓	✗
Marketplace	8095	✓	✓
Identity	8096	✓	✓
Notification	8097	✓	✗
Search	8098	✓	✓

B. Service Account Permissions

YAML

```

serviceAccounts:
  purchasing-service:
    secrets:
      - "purchasing-db-secret"
      - "kafka-credentials"
    permissions:
      - "get pods"
      - "list events"

  invoice-service:
    secrets:
      - "invoice-db-secret"
      - "kafka-credentials"

```

```
permissions:
  - "get pods"

workflow-service:
  secrets:
    - "workflow-db-secret"
    - "kafka-credentials"
  permissions:
    - "get pods"
    - "watch pods"
```

C. Database Connection Pools

TYPESCRIPT

```
// Connection pool configuration
const dbConfig = {
  postgres: {
    max: 20,
    idleTimeoutMillis: 30000,
    connectionTimeoutMillis: 2000,
    queue: { limit: 1000 },
    healthCheck: true
  },
  kafka: {
    producer: { maxInFlight: 5, acks: 1 },
    consumer: { maxConcurrent: 10 }
  },
  redis: {
    max: 50,
    idleTimeout: 30000
  }
};
```

D. Health Check Endpoints

TYPESCRIPT

```
// Standard health check implementation
interface HealthCheck {
  status: 'healthy' | 'unhealthy' | 'degraded';
  checks: {
    database: 'ok' | 'error';
    kafka: 'ok' | 'error' | 'warning';
    redis: 'ok' | 'error';
    version: string;
    uptime: number;
  };
};
```

```

    timestamp: string;
  }

  // Liveness: simple status
  GET /health -> { status: 'healthy', version: '1.0.0' }

  // Readiness: full checks
  GET /ready -> {
    status: 'healthy',
    checks: { ... },
    timestamp: '2027-01-15T10:30:00Z'
  }

```

End of Software Architecture Document

Document Status: Draft — Under Review | **Version:** 1.1 | **Effective Date:** TBD after approval

Maintained by: OpenS2P Core Team | **License:** Apache 2.0

E. Acronyms

Acronym	Meaning
ADR	Architecture Decision Record
API	Application Programming Interface
CLM	Contract Lifecycle Management
DDD	Domain-Driven Design
GRN	Goods Receipt Note
HITL	Human-in-the-Loop
JWT	JSON Web Token
OTEL	OpenTelemetry
PO	Purchase Order
RFx	Request for X (RFI / RFP / RFQ)
S2P	Source-to-Pay
SAD	Software Architecture Document
SLA	Service Level Agreement
SOW	Statement of Work
Wasm	WebAssembly

F. Glossary

Term	Definition
Aggregate	A cluster of domain objects treated as a single unit for data changes, per DDD conventions.
Aggregate Root	The entry-point entity of an aggregate through which all changes to that aggregate must pass.
Bounded Context	An explicit boundary within which a domain model applies consistently.
Confidence Score	A numeric estimate of an AI agent's certainty in a proposed action, used to decide whether human review is required.
Connector	A runtime adapter that integrates OpenS2P with an external system such as an ERP or payment gateway.
Domain Event	An immutable record that something meaningful happened within a service's domain, published for other services to consume.
Plugin	An extension executed in a sandboxed runtime that adds capability without modifying core services.
Schema-per-Tenant	A multi-tenancy pattern in which each tenant's data lives in its own PostgreSQL schema within a shared database.
Three-Way Match	Reconciliation of a purchase order, goods receipt, and invoice before payment is authorised.
Award	The decision to select a winning bid at the close of an RFx, recorded as SourcingEventAwarded.
Goods Receipt	Confirmation that ordered goods physically arrived, recorded by the Receiving Service.
Requisition	An internal request to purchase, which becomes a Purchase Order once approved.
RFx	Umbrella term for RFI (Request for Information), RFP (Request for Proposal), and RFQ (Request for Quotation).

G. Threat Model (Summary)

A STRIDE-style summary of the primary trust boundaries. This is a starting point for a full threat-modelling exercise, not a substitute for one.

Trust Boundary	Primary Threats	Mitigation (baseline / planned)
External client → API Gateway	Spoofing, tampering, DoS	TLS, OIDC token validation, rate limiting (planned)
API Gateway → Domain services	Elevation of privilege	OPA policy evaluation per request

Trust Boundary	Primary Threats	Mitigation (baseline / planned)
Service → Kafka	Tampering, information disclosure	Broker ACLs and TLS (to be defined in deployment profile)
Service → PostgreSQL	Information disclosure across tenants	Schema-per-tenant isolation, least-privilege DB roles
Plugin Runtime → Host	Elevation of privilege, resource exhaustion	Wasmtime sandbox, capability-scoped manifests (Section 12)

H. Disaster Recovery & Backup Strategy

Recovery objectives and backup approach, to be finalised with the infrastructure team ahead of production launch.

Data Store	Backup Approach	Recovery Objective (proposed)
PostgreSQL	Continuous WAL archiving + daily base backup	RPO ≤ 15 min / RTO ≤ 1 hour
Kafka	Multi-broker replication (replication factor ≥ 3)	No single-broker data loss
PGVector / Neo4j	Daily snapshot	RPO ≤ 24 hours
ClickHouse (audit)	Daily snapshot to object storage	RPO ≤ 24 hours

Availability target: 99.9% platform availability (roughly 8.7 hours of downtime budget per year), consistent with the pod-replacement scenario in 28.1. A 99.95% target is achievable with multi-zone deployment but has not been ratified as a commitment; treat it as a stretch target pending infrastructure sign-off.

Multi-region strategy: not yet defined. The schema-per-tenant model (27.3) is compatible with a future active-passive multi-region design, but no region-failover mechanism exists in this version of the platform — flagged here rather than implied.

Data retention: operational data is retained for the life of the tenant relationship; audit records (ClickHouse) are retained for a minimum of 7 years to support procurement compliance requirements common in enterprise buyers' own audit obligations; exact retention periods per data category should be confirmed against the customer's contractual and regulatory requirements rather than assumed from this default.

I. Version Compatibility Matrix

Minimum supported versions as stated in Section 2.4 (Technology Stack Summary). Upper bounds are not yet pinned platform-wide; treat 'x' as "newest tested minor within that major" until a formal compatibility test matrix is run and this table is updated with confirmed ranges.

Component	Minimum Version	Notes
PostgreSQL	15.x	PGVector extension requires 15+

Component	Minimum Version	Notes
Kafka	3.5.x	KRaft mode assumed (no ZooKeeper dependency)
Kubernetes	1.28.x	HPA v2 and CSI secret store driver assumed available
Redis	7.x	—
Neo4j	5.x	—
ClickHouse	23.x	—
Wasmtime	14.x	Plugin Runtime sandbox
OpenTelemetry Collector	1.20.x	—
Keycloak	22.x	—

I.1 Compatibility Policy

The matrix above states supported versions; this states the rules for how compatibility is maintained as those versions and the platform's own contracts change.

- **API compatibility:** governed by the versioning policy in 19.4 — additive changes stay within a major version, breaking changes increment it, with a minimum 6-month deprecation window.
- **Event compatibility:** consumers must tolerate unknown additional fields in an event payload (forward compatibility); removing or renaming a field is a breaking change requiring a new event name (e.g. `PurchaseOrderIssuedV2`) rather than mutating `PurchaseOrderIssued` in place, since Kafka topics may have multiple consumer versions in flight.
- **Plugin compatibility:** governed by the manifest-based dependency resolution in 14.3 — a plugin declares the minimum platform version it requires, and the Marketplace refuses installation rather than allowing a runtime mismatch.
- **Database migration strategy:** schema migrations are additive and backward-compatible within a release (new nullable columns, new tables); destructive migrations (column removal, type changes) ship in a separate release after the owning service has stopped reading the old shape, following an expand-contract pattern rather than a single in-place breaking migration.

J. Error Code Catalog

Representative error codes following the `OpenS2PError` hierarchy defined in Section 21.1. This is illustrative, not exhaustive — the authoritative list for a given service is its OpenAPI specification; keep this appendix in sync as new domain error codes are added.

Code	HTTP Status	Error Class	Meaning
AGGREGATE_NOT_FOUND	404	AggregateNotFoundError	Generic not-found for any aggregate root
SOURCING_EVENT_NOT_F	404	AggregateNotFoundError	Sourcing event does not

Code	HTTP Status	Error Class	Meaning
OUND			exist
SUPPLIER_NOT_QUALIFIED	409	BusinessRuleViolationError	Action requires a qualified supplier
SUPPLIER_BLOCKED	409	BusinessRuleViolationError	Supplier is inactive or suspended
CONTRACT_NOT_FOUND	404	AggregateNotFoundError	Contract does not exist
PURCHASE_ORDER_INVALID_STATE	422	BusinessRuleViolationError	Requested transition not valid from current PO state
INVOICE_MATCH_FAILED	422	BusinessRuleViolationError	Three-way match failed; see 8.x for detail
AI_CONFIDENCE_LOW	422	BusinessRuleViolationError	Agent action requires human approval
UNAUTHORISED	401	UnauthorisedError	Missing or expired token
FORBIDDEN	403	ForbiddenError	Authenticated but not authorised (OPA denial)
DATABASE_ERROR	503	DatabaseError	Underlying data store unavailable
MESSAGING_ERROR	503	MessagingError	Kafka unavailable or publish failed
INTERNAL_ERROR	500	OpenS2PError (generic)	Unhandled exception